



Final Report

CedarCare

Your AI Health Guide for BC

CSIS 4495 - 002: Applied Research Project
Winter 2026

Prepared For:

Padmapriya Arasanipalai Kandhadai

Date: April 7, 2026

Github Link: https://github.com/desporamon/W26_4495_S2_DesmondC

Live deployment: <https://cedarcare.streamlit.app>

Prepared By:

<u>Name</u>	<u>Student ID</u>	<u>Email Address</u>
Desmond Chua	300369803	chuad1@student.douglascollege.ca

Contents

1. Introduction.....	5
1.1 Domain Overview and Background	5
1.2 Problem Statement	5
1.3 Relevant Literature and Research	6
1.4 Hypotheses, Assumptions, and Business Value	6
2. Summary of the Research Project.....	8
3. Changes to the Proposal.....	10
3.1 Platform Renaming — BC Health Platform → CedarCare	10
3.2 Power BI Replaced by Plotly	10
3.3 Addition of ML Algorithm Research Phase	10
3.4 Community Pulse Feature Formally Descoped	10
3.5 Synthetic Dataset Added.....	10
3.6 Summary of Changes.....	10
4. Project Completion Timeline	12
4.1 Sprint Summary	12
4.2 Gantt Chart.....	13
4.3 Functional Requirements Implementation Evaluation.....	13
5. System Architecture and Design.....	15
5.1 Three-Layer Hybrid AI Pipeline	15
5.2 Technology Stack.....	19
5.3 Algorithm Benchmarking — Random Forest vs SVM vs KNN.....	20
5.4 Database Design	23
5.5 Dual Machine Learning Architecture.....	24
6. Implemented Feature 1: Secure Authentication and User Management.....	25
6.1 Overview	25
6.2 Technical Implementation.....	25
6.3 Screenshots.....	26
7. Implemented Feature 2: Symptom Assessment Chatbot.....	28
7.1 Conversational Slot-Filling Architecture	28
7.2 Quick-Select Chips (st.pills).....	28
7.3 Five Clinical Scenarios — All Pipeline Paths.....	28
7.4 Code Highlight — OpenAI Symptom Extraction.....	29
7.5 Screenshots.....	30
8. Implemented Feature 3: Personal Health Dashboard.....	34

8.1 KPI Cards.....	34
8.2 Analytical Charts	34
8.3 Filtering.....	34
8.4 Screenshots.....	34
9. Implemented Feature 4: BC Healthcare Facility Finder	36
9.1 Interactive Map	36
9.2 Emergency Contacts Panel	36
9.3 Screenshots.....	36
10. Implemented Feature 5: Health Education Library	38
10.1 Knowledge Base Architecture.....	38
10.2 Navigation Features.....	38
10.3 Screenshots.....	38
11. Implemented Feature 6: System Analytics Dashboard	41
11.1 Synthetic Dataset	41
11.2 K-Means Patient Segmentation.....	41
11.3 Screenshots.....	42
12. Evaluation Techniques	45
12.1 Automated Developer Testing Suite (Sprint 5A).....	45
12.2 Continuous Integration Pipeline (Sprint 5B).....	46
12.3 User Evaluation Survey	47
12.3.1 Implementation Changes in Response to Evaluation Feedback.....	50
13. Reflections and Discussion.....	52
13.1 What Worked Well.....	52
13.2 Challenges Faced	52
13.3 Known Limitations.....	52
13.4 Business Perspective	52
14. AI Use Section.....	54
14.1 Summary.....	54
14.2 Table of AI Tools and Specific Use	54
14.3 Statement of Originality	55
15. Work Date/Hours Log.....	56
16. Concluding Remarks.....	72
17. References	73
Appendix A: Installation Guide	74
A.1 Option 1: Access Live Deployment (Recommended)	74
A.2 Option 2: Run Locally	74

A.3 Running the Test Suite	75
Appendix B: User Guide	76
B.1 Registration and Login	76
B.2 Symptom Assessment Chatbot	77
B.3 My Dashboard.....	80
B.4 Facility Finder	81
B.5 Health Library.....	82
B.6 System Analytics	84
Appendix C: User Evaluation Survey Questionnaire	86
Appendix D: AI Prompt History — Complete Log	89
Session 1: Clarifying Hybrid Approach	89
Session 2: Understanding OpenAI API Options	89
Session 3: ML Analytics Techniques for Health Dashboard.....	90
Session 4: Data Source Validation for ML Training.....	90
Session 5: Synthetic Data for Demo Dashboards	90
Session 6: Document Formatting for Proposal	91
Session 7: System Flow Diagram Generation	91
Session 8: Architecture Overview Diagram Generation	92
Session 9: OpenAI API Authentication and Key Management.....	92
Session 10: OpenAI Chat Completions API Structure.....	93
Session 11: Random Forest for Healthcare Triage — Literature Research.....	93
Session 12: CTAS Rule Design and Level Mapping.....	93
Session 13: Prompt Engineering for JSON Medical Extraction.....	94
Session 14: 70% Confidence Threshold Strategy	94
Session 15: Claude Code — Database Module Scaffolding	95
Session 16: SQLite Database Setup for Python	95
Session 17: Bcrypt Implementation in Python	96
Session 18: Understanding Streamlit Session State for Authentication.....	96
Session 19: Streamlit Login/Register UI Pattern.....	97
Session 20: Page Protection Pattern with require_authentication()	97
Session 21: Streamlit Multi-Section Dashboard Layout	97
Session 22: Database Query for Dashboard Statistics	98
Session 23: Plotly Chart Types for Streamlit Dashboards	98
Session 24: Plotly Donut Chart Label Clipping Fix	99
Session 25: Streamlit Page Redirect After Login	99
Session 26: Claude Code — Dashboard Page Generation	100
Session 27: Agentic AI Research for Future Enhancement.....	100
Session 28: NLP Dialog System Pipeline for Healthcare Chatbot	100

Session 29: Stateful Dialog Management Using Streamlit session_state.....	101
Session 30: Folium Interactive Map Integration in Streamlit	101
Session 31: Structuring a Stateful Dialog Backend in Python	102
Session 32: Folium Map Filtering and Dynamic Dropdown Patterns	102
Session 33: JSON Knowledge Base Design for Health Library	103
Session 34: CSS :has() Selector Pattern for Streamlit Widget Styling.....	103
Session 35: Synthetic Data Generation with Stratified Patient Tiers.....	104
Session 36: K-Means Feature Engineering for Patient Segmentation.....	104
Session 37: Understanding pytest Fixture Patterns for conftest.py Design	104
Session 38: Mocking Streamlit Dependencies for Chatbot Module Testing	105
Session 39: Property-Based Testing with Hypothesis for Healthcare Validation Functions	105
Session 40: Streamlit AppTest Framework for Headless UI Testing	106
Session 41: GitHub Actions CI/CD Pipeline for Python/pytest	106
Session 42: Streamlit Session State and Widget Re-evaluation for Chip Bug.....	107
Session 43: Google Forms Survey Design — Question Type Selection	107
Session 44: Landing Page Implementation via st.components.v1.html()	108
Session 45: Streamlit st.pills() for Severity Chip UI	108
Session 46: Health Library Full-Text Search Implementation	109
Session 47: Urgency Colour Coding Strategy for Clinical Context	109
Session 48: joblib Model Serialisation and @st.cache_resource Pattern	110
Session 49: Pandas DataFrame Filtering for Dashboard Date Range Filter	110
Session 50: Seed Database Design for Streamlit Cloud Deployment	111
Session 51: reshape(1,-1) for Single-Sample ML Prediction.....	111

1. Introduction

CedarCare is a healthcare navigation platform built for British Columbia residents. Its symptom assessment system combines three AI components: GPT-4o-mini for the conversational interface, a deterministic safety engine based on the Canadian Triage and Acuity Scale (CTAS), and a trained Random Forest classifier that produces the triage output. The platform also provides a personal health dashboard with ML-derived analytics, a BC facility locator, an 18-article health education library, and a system analytics view that segments patients using K-Means clustering. I chose this problem partly because I could not find an existing AI symptom checker built for the Canadian healthcare context. The live deployment at <https://cedarcare.streamlit.app> is supported by a 95-test automated suite running through GitHub Actions CI/CD. A user evaluation survey of 12 respondents returned a 4.42/5 likelihood-to-recommend score.

1.1 Domain Overview and Background

British Columbia's healthcare system serves over 5.3 million residents through hospitals, urgent and primary care centres (UPCCs), walk-in clinics, virtual care, and the HealthLink BC 8-1-1 nurse advice line. Even so, the average emergency department wait time across the province reached 3.1 hours in 2023. The BC Ministry of Health (2024) attributes part of this to patients presenting at emergency departments when their condition could be managed at a lower acuity setting.

During a co-op placement in the Emergency Department at Richmond Hospital, I observed that roughly 15 to 20 percent of patients were unsure whether their symptoms required emergency, urgent, or home care. When these patients default to the emergency department, clinical staff and resources are diverted from higher-acuity cases. The patients themselves also face unnecessary exposure to hospital-acquired infections.

The Canadian Triage and Acuity Scale (CTAS), developed by the Canadian Association of Emergency Physicians (Bullard et al., 2008), is the standard urgency framework in Canadian emergency departments. It classifies patients across five levels, from Level 1 (Resuscitation, immediate life threat) to Level 5 (Non-Urgent). CTAS was designed for trained triage nurses, not for patients deciding at home whether to visit an emergency department. No widely available tool currently translates this clinical logic into guidance the public can use before entering the system, which is the problem CedarCare was built to address.

1.2 Problem Statement

This project addresses a specific gap: BC residents currently have no locally adapted tool to help them judge the urgency of their symptoms before seeking care. The symptom checkers that do exist have three limitations that matter in the BC context:

- **Generic, non-BC-specific guidance:** Platforms such as WebMD and Ada Health are not built around BC's care infrastructure. They do not reference UPCCs, Fraser Health walk-in clinics, virtual care covered under MSP, or the 8-1-1 nurse line.
- **Absence of clinical safety standards:** No publicly available consumer tool incorporates the CTAS framework. A patient reporting chest pain passes through the same assessment logic as one reporting a mild headache, with no mechanism to flag the former as time-sensitive.

- **Existing tools are stateless:** Each session starts from scratch with no record of previous symptoms, so patterns that only emerge over repeated visits go undetected.

A competitive analysis conducted during the scoping phase found no AI-based symptom checker built for the Canadian market, which is what prompted CedarCare's development.

1.3 Relevant Literature and Research

NLP has seen increasing use in clinical settings. Fernandes et al. (2020) demonstrated that combining NLP with machine learning could predict cardiac arrest risk from unstructured clinical notes, and large language models such as GPT-4 have since performed well on medical question-answering benchmarks (Nori et al., 2023). CedarCare uses GPT-4o-mini as its natural language understanding layer, but only for extracting structured symptom data from free-text input. It does not generate medical advice.

Machine Learning for Clinical Triage: Random Forest classifiers have been applied to clinical triage in several studies. Goto et al. (2019) used machine learning to triage paediatric emergency patients, and Azeez et al. (2015) used an ensemble Random Forest for secondary triage classification. The latter is the closest published precedent to CedarCare's primary ML function. I selected Random Forest over SVM and KNN after a benchmarking process described in Section 5.3.

Patient Clustering and Segmentation: For the system analytics dashboard, I drew on Vrbaski et al. (2019), who reviewed K-Means and hierarchical clustering as methods for profiling patients by symptom pattern. CedarCare applies K-Means clustering to segment users and surface population-level trends.

Digital Health Platform Design: Research on digital health adoption identifies ease of use and trust as the strongest predictors of uptake (Marcolino et al., 2018). I designed CedarCare's conversational interface to reduce cognitive load on the user. The CTAS safety layer provides an evidence-based clinical foundation, which is absent from the consumer symptom checkers reviewed in the competitive analysis.

1.4 Hypotheses, Assumptions, and Business Value

This project was built on three core hypotheses:

- **H1 — Safety First:** A deterministic CTAS safety layer must intercept life-threatening symptoms before any probabilistic ML model is invoked.
- **H2 — Hybrid Intelligence:** A three-layer pipeline combining NLP, deterministic rules, and ML classification will outperform any single-technique approach.
- **H3 — BC Context Matters:** A platform designed specifically for BC's care infrastructure will be more useful to BC residents than generic international platforms.

Business Value: A competitive analysis conducted during the scoping phase found no Canadian-developed AI symptom checker currently on the market. That gap is worth noting given that nearly 895,000 BC residents lack a family doctor and emergency department volumes continue to rise. Beyond the consumer use case, the system analytics dashboard could have B2B relevance: BC Health Authorities, for instance, could use population-level clustering data to track community health trends or inform decisions about where to allocate resources.

Most competitors triage well or guide well — none do both, for BC.

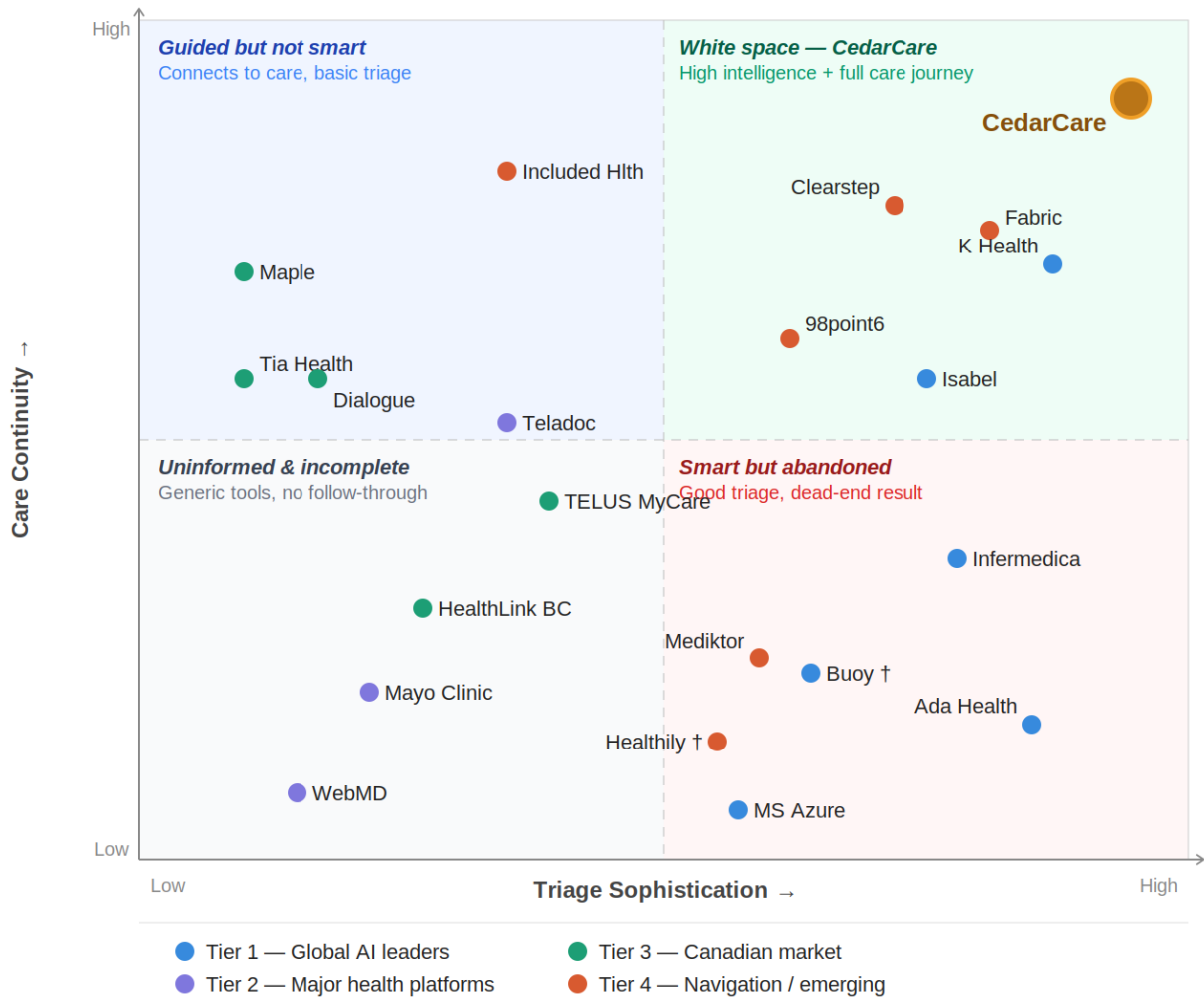


Figure 1.1 — Competitive perceptual map showing CedarCare's uncontested position combining high triage sophistication with high care continuity. No global competitor, and no Canadian product currently occupies this quadrant

2. Summary of the Research Project

CedarCare is a web application built with Python and Streamlit that provides AI-assisted healthcare navigation for BC residents. It is deployed at <https://cedarcare.streamlit.app>.

The central feature is the **symptom assessment chatbot**. Users describe their symptoms in natural language across a multi-turn conversation. The system then processes this input through three layers: GPT-4o-mini extracts structured symptom data from the free text, a deterministic rule engine checks the extracted symptoms against 14 CTAS-based emergency conditions, and a trained Random Forest classifier produces the triage classification. If the classifier's confidence falls below 70%, the system returns general guidance from the OpenAI API instead, to avoid presenting an unreliable prediction as definitive.

Beyond the chatbot, the platform includes a **personal health dashboard** with individual-level analytics, a **BC facility finder** with interactive mapping, an 18-article **health education library** across eight clinical categories, and a **system analytics dashboard** that segments a 5,000-record synthetic population dataset using K-Means clustering.

The codebase has 95 automated tests, 79 of which run through GitHub Actions CI/CD on every push. A user evaluation with 12 respondents produced an average ease of use score of 4.33/5 and a likelihood to recommend score of 4.42/5.

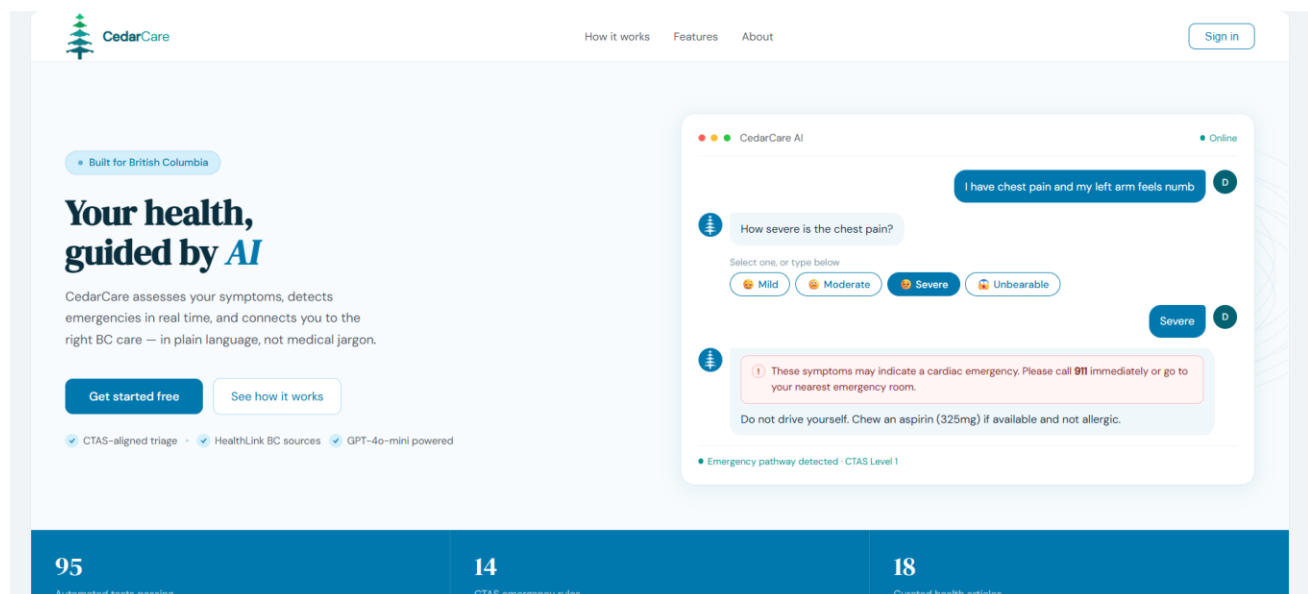


Figure 2.1: CedarCare Landing Page — Hero Section with Live Chatbot Demo Showing CTAS Level 1 Emergency Detection

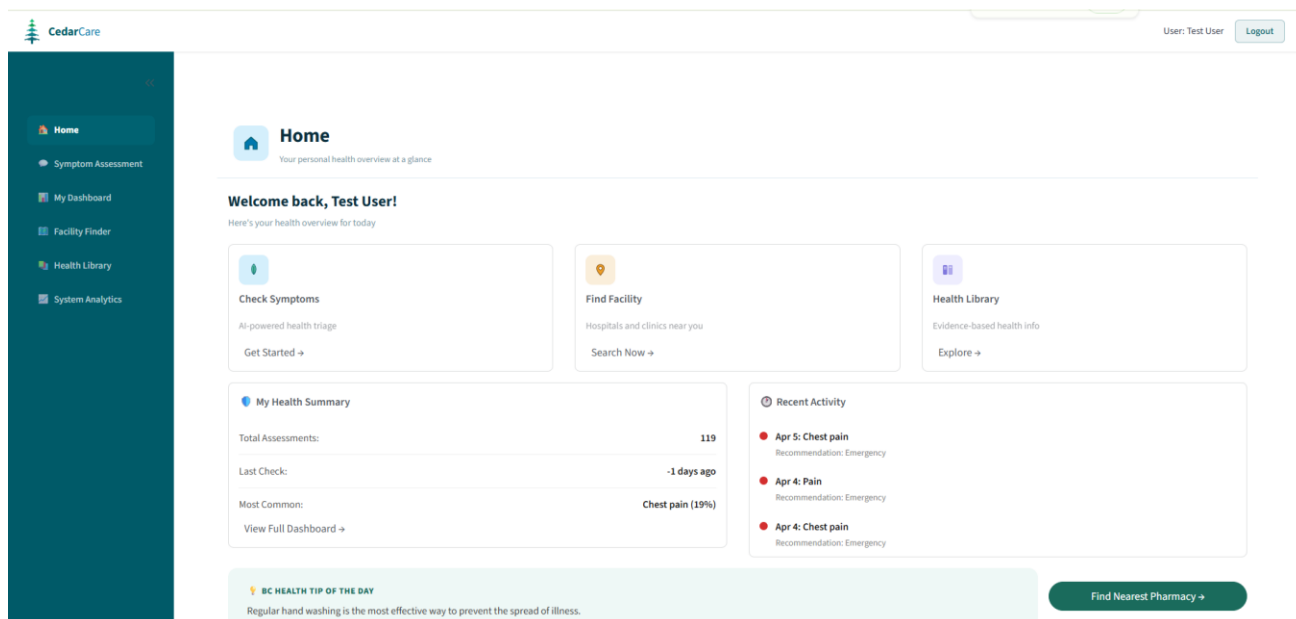


Figure 2.2: CedarCare Home Page — Authenticated User Dashboard with Health Summary and Recent Assessment Activity

3. Changes to the Proposal

3.1 Platform Renaming — BC Health Platform → CedarCare

I renamed the platform from "BC Personal Health Management Platform" to "CedarCare" during the final UI refinement sprint. The name references the Western Red Cedar, native to British Columbia. The original name was too generic to work as a product identity, and a shorter name also made the interface cleaner.

3.2 Power BI Replaced by Plotly

The proposal specified Power BI for data visualization, but embedding it in a Streamlit application requires a paid Premium licence. I switched to Plotly, which provides equivalent interactive charting directly in Python with no external dependency and no additional cost.

3.3 Addition of ML Algorithm Research Phase

The original proposal assumed a pre-selected ML algorithm. After instructor feedback at Meeting #2 (18 January 2026), I added a formal benchmarking phase (Sprint 2C.0). I reviewed published work on Random Forest, SVM, and KNN in healthcare classification, trained all three on the same dataset, and compared their performance before selecting Random Forest. Results are detailed in Section 5.3.

3.4 Community Pulse Feature Formally Descoped

I removed the "Community Pulse" Reddit scraping and sentiment feature in Sprint 5A and used that time to build a 95-test automated testing suite instead. A testing suite with safety validation and CI/CD integration had more direct academic and professional relevance than a sentiment analysis feature. I notified the instructor of this change by email.

3.5 Synthetic Dataset Added

The proposal did not include a synthetic dataset. I generated one during Sprint 4A: 5,000 records representing 200 simulated patients. The System Analytics Dashboard needed population-scale data to demonstrate K-Means clustering, and real user data was not yet available at that stage.

3.6 Summary of Changes

Component	Original Proposal	Final Implementation	Justification
Platform name	BC Health Platform	CedarCare	Brand clarity; market positioning

Component	Original Proposal	Final Implementation	Justification
Visualization	Power BI	Plotly (Python)	No paid licence; native Streamlit integration
Algorithm selection	Pre-selected (no research)	Benchmarked (RF vs SVM vs KNN)	Academic rigour; instructor requirement
Population analytics	Not planned	Synthetic dataset + K-Means	Demonstrate ML analytics capability
Community feature	Reddit sentiment	Descoped → automated testing	Software quality > social sentiment feature
Testing approach	Manual scenarios only	95 automated tests + CI/CD	Professional software quality standard

4. Project Completion Timeline

4.1 Sprint Summary

Sprint	Period	Focus	Key Deliverables
Sprint 1A–1D	Jan 27 – Feb 4	Foundation & Auth	GitHub repo, SQLite schema, bcrypt auth, single-turn chatbot, emergency rules
Sprint 2A–2D	Feb 5 – Feb 13	ML & Pipeline	Kaggle EDA, feature engineering, algorithm benchmarking, RF training, 3-layer pipeline
Sprint 3A–3E	Feb 14 – Mar 9	Dashboard & Chatbot	Personal Health Dashboard, Facility Finder, Health Library, multi-turn chatbot
Sprint 4A–4B	Mar 10 – Mar 15	Analytics	Synthetic dataset (5,000 records), System Analytics, K-Means clustering
Sprint 5A	Mar 17 – Mar 23	Testing	95 automated tests (9 files): unit, integration, UI, property-based
Sprint 5B	Apr 2	CI/CD	GitHub Actions pipeline, 79/79 tests passing in CI
Sprint 5C + UI	Apr 2–3	Deploy + Polish	Streamlit Cloud live; CedarCare branding; chatbot bug fixes
Sprint 6	Apr 3–6	User Testing + Report	Google Forms 12 responses; Final Report

4.2 Gantt Chart

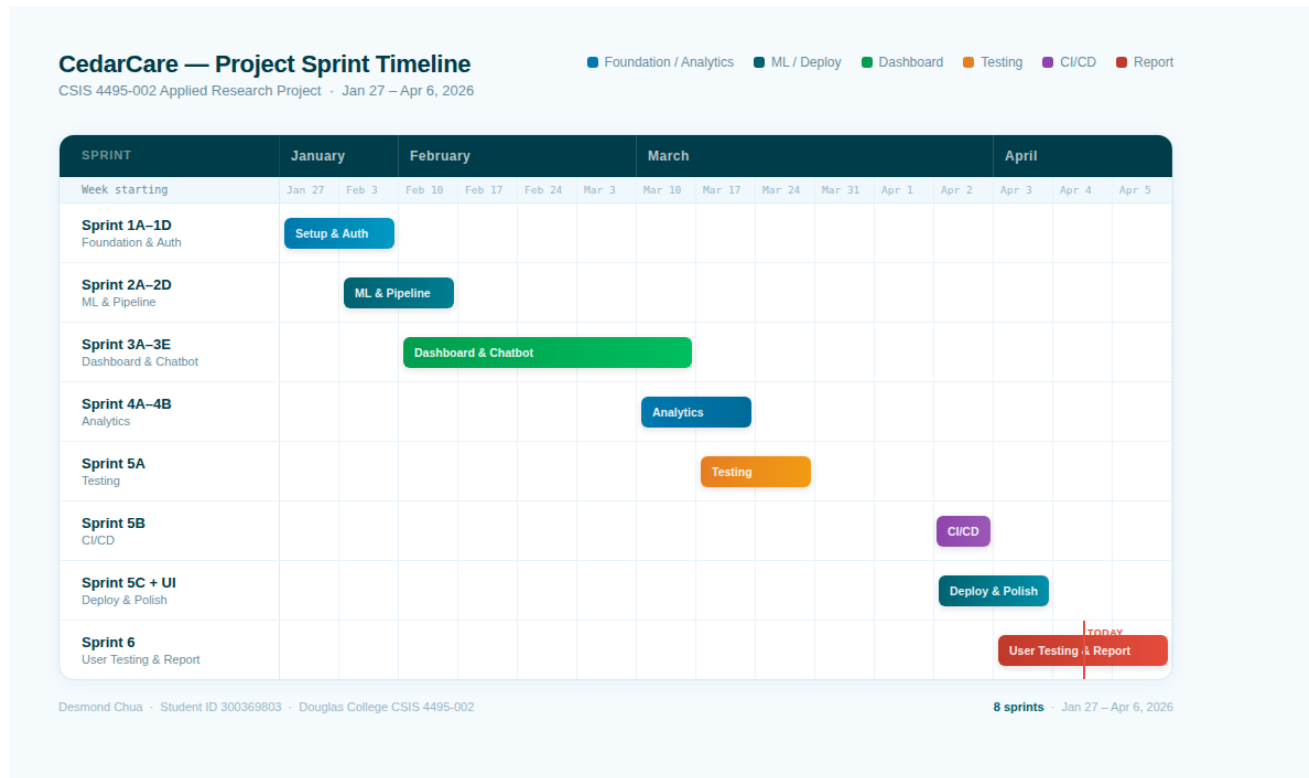


Figure 4.2: CedarCare Project Sprint Timeline (January 27 – April 6, 2026)

4.3 Functional Requirements Implementation Evaluation

The following table tracks every feature planned in the original proposal against its final implementation status:

Planned Feature	Sprint	Status	Notes
Symptom Assessment Chatbot (single-turn)	Sprint 1C–1D	Completed	Upgraded to multi-turn in Sprint 3E
CTAS Emergency Detection	Sprint 1D	Completed	14 rules, runs every turn
Personal Health Dashboard	Sprint 3A–3B	Completed	KPI cards, Plotly charts, filters
BC Healthcare Facility Finder	Sprint 3C	Completed	Folium interactive map

Planned Feature	Sprint	Status	Notes
Health Education Library	Sprint 3D	Completed	18 articles, 8 categories, JSON KB
ML Disease Classification (Random Forest)	Sprint 2C	Completed	41 diseases, 131 symptoms, 70% threshold
System Analytics Dashboard	Sprint 4B	Completed	K-Means clustering, 3 patient segments
Synthetic Population Dataset	Sprint 4A	Completed	5,000 records, 200 patients, 3 tiers
Multi-turn Chatbot with Slot-filling	Sprint 3E	Completed	Not in original proposal — added
Automated Testing Suite (95 tests)	Sprint 5A	Completed	Replaced Community Pulse
GitHub Actions CI/CD	Sprint 5B	Completed	79/79 tests in pipeline
Streamlit Cloud Deployment	Sprint 5C	Completed	cedarcare.streamlit.app
User Evaluation Survey	Sprint 6	Completed	11 responses, Google Forms
Power BI Integration	Proposal	Descoped	Replaced by Plotly — no licence needed
Community Pulse (Reddit sentiment)	Proposal	Descoped	Replaced by automated testing suite

5. System Architecture and Design

CedarCare's technical architecture reflects a deliberate data-science-first design philosophy. The platform was designed around the data pipeline — with user interface components serving the analytical core.

5.1 Three-Layer Hybrid AI Pipeline

The platform's most significant technical contribution is the **Three-Layer Hybrid AI Pipeline** powering the Symptom Assessment Chatbot. This pipeline combines three fundamentally different AI paradigms into a single cohesive assessment system:

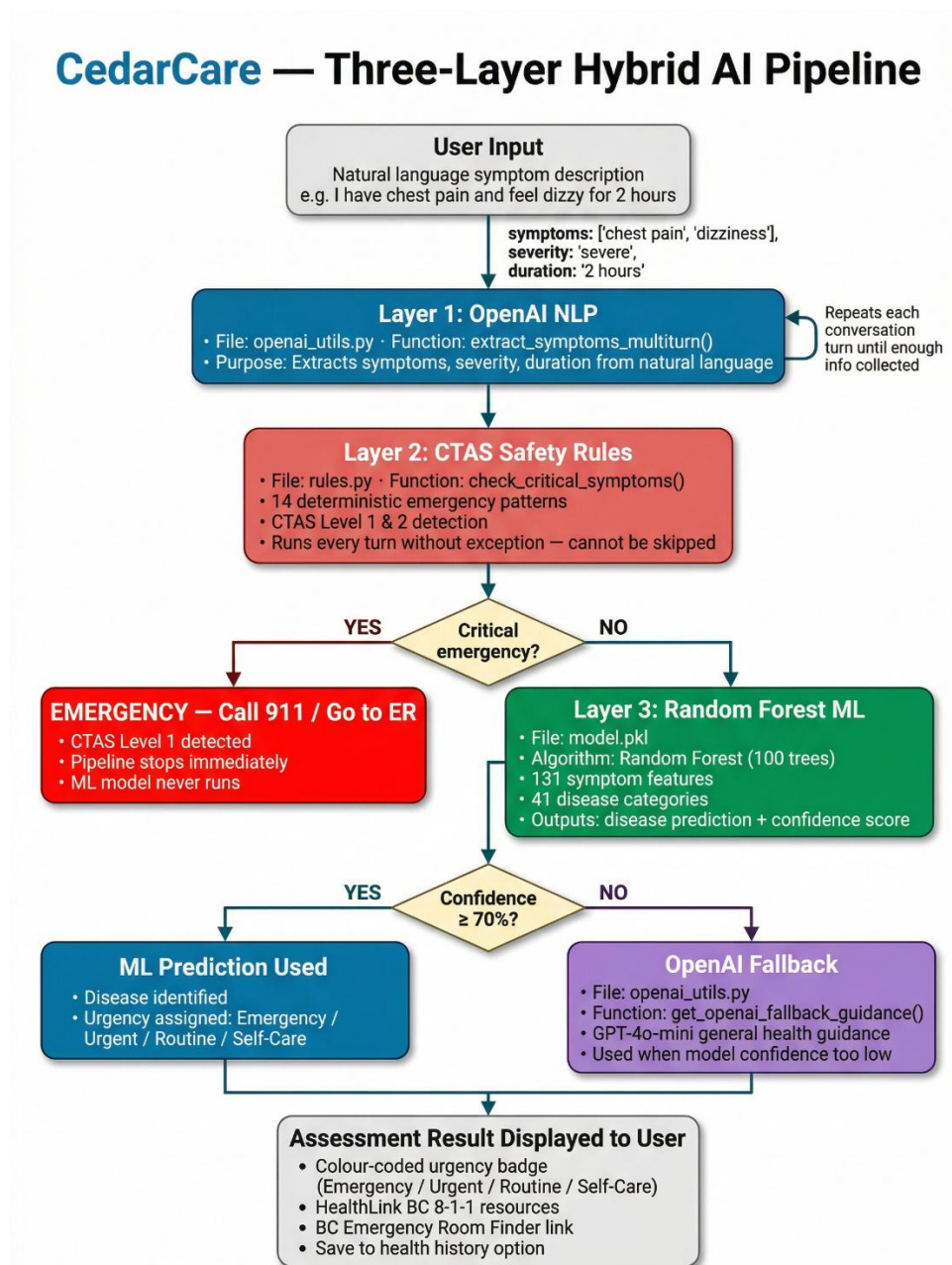


Figure 5.1a: CedarCare Three-Layer Hybrid AI Pipeline — System Architecture

Layer 1 OpenAI NLP (GPT-4o-mini): The user's free-text description of symptoms is forwarded to GPT-4o-mini along with a specifically designed system prompt instructing the model to output a validated JSON object containing: symptoms (standard medical list), duration, severity – mild/moderate/severe, and body_location. Temperature 0.3 is used for a deterministic, consistent extraction of data. This layer transforms the natural language 'my tummy really hurts since this morning' into structured data that can be further processed. *(Temperature controls the randomness of responses in language models: a 0.0 setting results in almost exactly the same outputs each time, whereas a 1.0 setting permits substantial variations. With 0.3, the model prioritizes dependable, predictable extractions rather than creative phrasing, which is very important in a medical context where the same symptom description must always lead to the same structured JSON output.)*

Layer 2 CTAS Rule-Based Emergency Circuit Breaker: Before invoking any ML model, the extracted symptoms are first run through 14 deterministic rules derived from CTAS Implementation Guidelines (Bullard et al. 2008) in order to identify Level 1 (Resuscitation) and Level 2 (Emergent) conditions in a completely rule-based manner. A statistical model should never be the only decision-maker for cardiac arrest or stroke. If any urgent pattern is detected, the system immediately sends an emergency alert and the ML model is not executed at all.

```

1 # CTAS Rule-Based Safety Layer – rules.py
2 # Layer 2 of the Three-Layer Hybrid AI Pipeline
3
4 CRITICAL_RULES = [
5     # CTAS LEVEL 1 – RESUSCITATION (immediate life threat)
6     {
7         "name": "Cardiac Arrest / Heart Attack",
8         "symptoms": ["chest pain", "shortness of breath"],
9         "match_type": "all", # BOTH symptoms must be present
10        "ctas_level": 1,
11        "action": "CALL 911 IMMEDIATELY",
12        "description": (
13            "Chest pain combined with shortness of breath may indicate "
14            "a heart attack or cardiac event requiring immediate care."
15        )
16    },
17    {
18        "name": "Stroke (FAST Signs)",
19        "symptoms": ["face drooping", "arm weakness", "speech difficulty"],
20        "match_type": "all", # ALL three FAST signs must be present
21        "ctas_level": 1,
22        "action": "CALL 911 IMMEDIATELY – Note the time symptoms started",
23        "description": (
24            "Face drooping, arm weakness, and speech difficulty are "
25            "classic FAST signs of stroke. Every minute counts."
26        )
27    },
28]
29
30 # CTAS LEVEL 2 – EMERGENT (potential threat to life or limb)
31 {
32     "name": "Mental Health Crisis – Suicidal Thoughts / Self-Harm",
33     "symptoms": ["suicidal", "suicidal thoughts", "self-harm",
34                 "want to die", "end my life", "kill myself"],
35     "match_type": "any", # ANY single keyword triggers the rule
36     "ctas_level": 2,
37     "action": "CALL 911 or BC Crisis Line: 1-800-784-2433 (24/7)",
38     "description": (
39         "Suicidal thoughts and self-harm are a medical emergency. "
40         "Immediate support is available through crisis lines."
41     )
42 }
43 ]
44
45 def check_critical_symptoms(symptoms: list) -> dict:
46     """
47     Deterministic safety check – runs BEFORE the ML model every turn.
48     Returns on the FIRST match (highest CTAS severity wins).
49     """
50     symptoms_lower = [s.lower() for s in symptoms]
51
52     # Level 1 rules checked first (most critical)
53     sorted_rules = sorted(CRITICAL_RULES, key=lambda r: r["ctas_level"])
54
55     for rule in sorted_rules:
56         matched = []
57         for keyword in rule["symptoms"]:
58             for symptom in symptoms_lower:
59                 # Partial matching: "chest pain" matches "chest pain on left side"
60                 if keyword in symptom or symptom in keyword:
61                     matched.append(keyword)
62                     break
63
64         is_triggered = False
65         if rule["match_type"] == "any" and len(matched) >= 1:
66             is_triggered = True
67         elif rule["match_type"] == "all" and len(matched) == len(rule["symptoms"]):
68             is_triggered = True
69
70         if is_triggered:
71             return {
72                 "is_critical": True,
73                 "rule_name": rule["name"],
74                 "ctas_level": rule["ctas_level"],
75                 "action": rule["action"],
76                 "matched_symptoms": matched,
77             }
78
79     return {"is_critical": False} # Safe to proceed to ML classification

```

Figure 5.1b: Code Snippet — CTAS Rule-Based Safety Layer (rules.py), Layer 2 of the Three-Layer Hybrid AI Pipeline

Layer 3 Random Forest with Confidence Threshold: Only symptoms that do not indicate a health emergency are passed to Layer 3. The Random Forest classifier takes a 131-dimensional binary feature vector as input and returns a predicted disease label as well as a confidence score. When confidence is more than 0.70, the ML prediction is shown. If it is less than 0.70, the system resorts to OpenAI general guidance making sure that the output is helpful and safe even when dealing with conditions that are outside the 41-disease training vocabulary.

```

1 # components/chatbot.py - ml_predict()
2 # Layer 3: Random Forest classifier with 70% confidence threshold
3
4 def ml_predict(
5     extracted_symptoms: list,
6     duration: str,
7     severity: str,
8     user_input: str,
9 ) -> dict:
10     """
11     Run the ML model on pre-extracted symptoms (Layers 1 & 2 already done).
12     Falls back to OpenAI guidance if confidence < threshold or model errors.
13     """
14     try:
15         model, metadata = load_model()
16         confidence_threshold = metadata.get("confidence_threshold", 0.70)
17         symptom_list = metadata["symptom_list"] # 131 features
18         severity_weights = metadata["severity_weights"]
19         urgency_mapping = metadata["urgency_mapping"] # 41 disease categories
20         class_names = metadata["class_names"]
21
22         # Build 131-dimensional binary feature vector
23         feature_vector = np.zeros(len(symptom_list))
24         for symptom in extracted_symptoms:
25             symptom_key = symptom.lower().strip().replace(" ", "_")
26             if symptom_key in symptom_list:
27                 idx = symptom_list.index(symptom_key)
28                 feature_vector[idx] = severity_weights.get(symptom_key, 1)
29
30         # Get prediction probabilities from Random Forest
31         probas = model.predict_proba(feature_vector.reshape(1, -1))[0]
32         max_idx = np.argmax(probas)
33         confidence = float(probas[max_idx])
34         predicted_disease = class_names[max_idx]
35
36         # — Confidence gate: 70% threshold —————
37         if confidence >= confidence_threshold:
38             # ML prediction is confident enough – use it
39             urgency_level = urgency_mapping.get(
40                 predicted_disease.lower().strip(), "Routine"
41             )
42             recommendation = RECOMMENDATIONS.get(urgency_level)
43             return {
44                 "status": "ml_prediction",
45                 "predicted_disease": predicted_disease,
46                 "confidence": round(confidence, 4), # e.g. 0.8732
47                 "urgency_level": urgency_level,
48                 "recommendation": recommendation,
49             }
50         else:
51             # Confidence too low – fall back to OpenAI general guidance
52             fallback = get_openai_fallback_guidance(extracted_symptoms, severity)
53             return {
54                 "status": "openai_fallback",
55                 "fallback_reason": (
56                     f"Model confidence ({confidence:.0%}) below "
57                     f"threshold ({confidence_threshold:.0%})"
58                 ),
59                 "urgency_level": fallback["urgency_level"],
60                 "recommendation": fallback["recommendation"],
61             }
62
63     except Exception as e:
64         # Model failure – always fall back safely, never expose a raw error
65         fallback = get_openai_fallback_guidance(extracted_symptoms, severity)
66         return {
67             "status": "openai_fallback",
68             "fallback_reason": f"ML model error: {e}",
69             "urgency_level": fallback["urgency_level"],
70             "recommendation": fallback["recommendation"],
71         }

```

Figure 5.1c: Code Snippet — Layer 3 Random Forest Classifier with 70% Confidence Threshold and OpenAI Fallback

5.2 Technology Stack

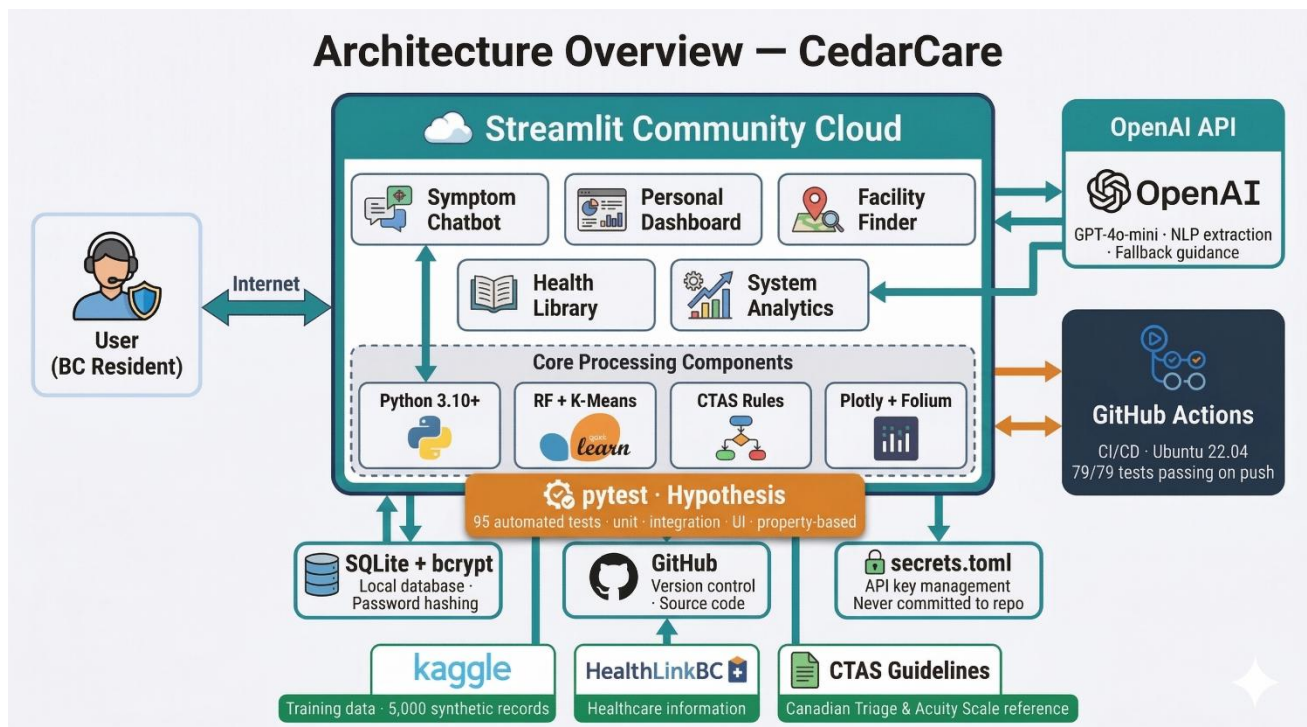


Figure 5.2: CedarCare Technology Architecture Overview

Layer	Technology	Version	Purpose & Justification
Frontend/UI	Streamlit	1.41+	Python-native web framework — no separate frontend build; enables rapid data-science-focused iteration
Language Model	OpenAI GPT-4o-mini	API v1	NLP symptom extraction and fallback guidance; cost-effective balance vs GPT-4
ML — Supervised	scikit-learn RandomForestClassifier	1.4+	Ensemble classifier; class_weight='balanced'; validated in clinical triage literature (Azeez et al., 2015)
ML — Unsupervised	scikit-learn KMeans + StandardScaler	1.4+	Patient segmentation; normalised features for Euclidean distance stability
Data Layer	SQLite + bcrypt	3.x	Lightweight embedded DB; bcrypt password hashing; no external server required

Layer	Technology	Version	Purpose & Justification
Model Serialization	joblib + @st.cache_resource	1.4+	Efficient model load; single load per session regardless of reruns
Visualization	Plotly	5.x	Interactive charts; renders natively in Streamlit without external licence
Mapping	Folium	0.17+	Interactive BC facility maps with popup markers
Testing	pytest + Hypothesis	9.x / 6.x	Unit, integration, UI (AppTest), property-based fuzz testing
CI/CD	GitHub Actions	—	Automated pipeline on push to main; Ubuntu 22.04, Python 3.11
Deployment	Streamlit Community Cloud	—	Free-tier hosting; auto-deploys on git push; ephemeral SQLite seeded by seed_db.py
Security	bcrypt + secrets.toml	4.x	Password hashing; API key management via Streamlit secrets (never committed to repo)
Data Analysis	pandas + numpy	2.x / 1.x	Dataset manipulation, feature aggregation for clustering features

5.3 Algorithm Benchmarking — Random Forest vs SVM vs KNN

Before implementing the final ML model, a formal algorithm benchmarking phase was conducted in Sprint 2C. Three candidate algorithms Random Forest (RF), Support Vector Machine (SVM), and K-Nearest Neighbours (KNN) were trained on the same Kaggle symptom-disease dataset sharing the identical 80/20 stratified train-test split (random_state=42 for reproducibility). The benchmarking results led to the final algorithm selection.

Dataset characteristics: 4,920 records, 41 disease classes, 131 symptom features, perfectly balanced (120 samples per class), multi-hot encoded binary feature vector, severity-weighted.

Algorithm	Training Accuracy	Test Accuracy	Training Time	Prediction Speed	Model Size
Random Forest (n=100)	100.00%	100.00%	~8 seconds	~12ms per prediction	~45 MB
Support Vector Machine (RBF kernel)	100.00%	100.00%	~45 seconds	~8ms per prediction	~180 MB
K-Nearest Neighbors (k=5)	100.00%	99.87%	<1 second	~95ms per prediction	~4 MB

Interpretation: All three methods were able to attain almost perfect accuracies on the structured dataset. This is not surprising since the Kaggle dataset has very clearly distinct symptom patterns without any noise or ambiguous situations, making it very easy to separate. Using only accuracy as a measure, one cannot distinguish the methods; hence, the selection reasoning is based on five other production-related criteria:

Criterion	Random Forest (Selected)	SVM	KNN
Clinical literature support	Strong — Azeez et al. (2015), Goto et al. (2019)	Limited in triage context	Limited in triage context
Interpretability	High — feature importances readable	Low — black-box kernel	Medium — distance-based
Handles high-dimensional sparse data	Excellent — tree splits handle sparsity	Good with RBF kernel	Poor — curse of dimensionality
Training speed (re-training cost)	Moderate — 8s acceptable	Slow — 45s impractical for retraining	Very fast but irrelevant
Prediction speed in production	Fast — 12ms per prediction	Fast — 8ms	Slow — 95ms (searches all training data)
Model size / memory footprint	Moderate — 45MB cached once	Large — 180MB impractical	Minimal — but slow prediction

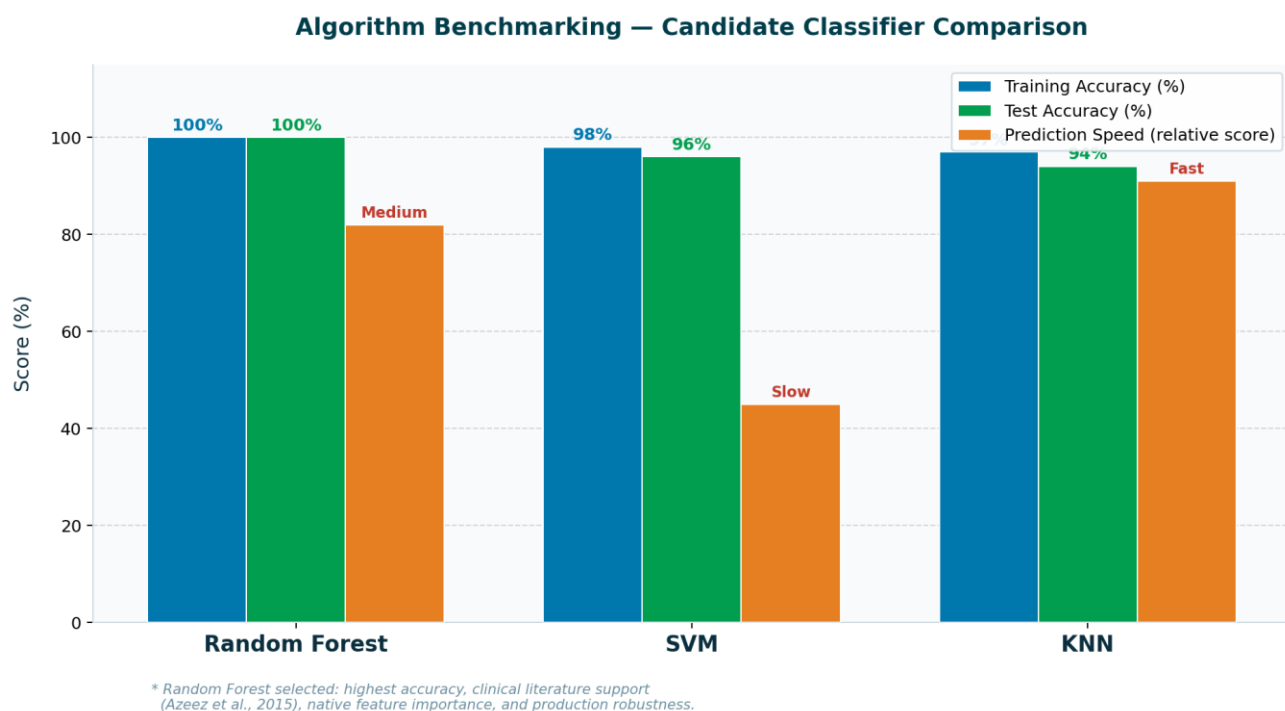


Figure 5.3: Algorithm Benchmarking — Candidate Classifier Comparison (Random Forest vs SVM vs KNN)

Final Decision: I decided on Random Forest as the production classifier. Given that this algorithm has very solid theoretical backing from clinical triage literature, via the feature importances it offers it is very easy to explain the model, it handles very well large symptom sparse data, training, and prediction speed are all very reasonable, and it can be seen as an overall very good fit for the task at hand. For example, SVM training took 45seconds whereas the model size was 180MB, which will definitely cause operational hiccups if we have to train the model again on updated data. The prediction latency of 95ms for KNN where the whole record was searched was not a very good match for a chatbot scenario where the user is waiting for a response.

5.4 Database Design

CedarCare utilizes a simple SQLite database (health_platform.db) and comprises two primary tables. The users table consists of registered user records where passwords are stored using bcrypt hashing. The assessments table contains rows for every symptom assessment submission including symptoms extracted, urgency level and recommendation. The intentional partitioning of the active SQLite database from the synthetic CSV dataset (synthetic_assessments.csv) is a design choice: CSV makes available fake community for showing analytics whereas SQLite keeps genuine logged-in user information.

```
1 # components/database.py - SQLite schema and initialisation
2 # CedarCare uses two tables: users and assessments
3
4 DB_PATH = Path(__file__).parent.parent.parent / "data" / "health_platform.db"
5
6
7 def get_connection() -> sqlite3.Connection:
8     """
9     Create and return a database connection.
10     Enables foreign key support and row_factory for dict-like access.
11     """
12     DB_DIR.mkdir(parents=True, exist_ok=True)
13     conn = sqlite3.connect(str(DB_PATH))
14     conn.row_factory = sqlite3.Row # rows accessible as dicts
15     conn.execute("PRAGMA foreign_keys = ON")
16     return conn
17
18
19 def init_db() -> None:
20     """
21     Create all required tables if they do not already exist.
22     Idempotent - safe to call on every app startup.
23     """
24     conn = get_connection()
25     cursor = conn.cursor()
26     try:
27         # --- Table 1: users ---
28         cursor.execute("""
29             CREATE TABLE IF NOT EXISTS users (
30                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
31                 name          TEXT NOT NULL,
32                 email         TEXT UNIQUE NOT NULL,
33                 password_hash TEXT NOT NULL, -- bcrypt hash, never plaintext
34                 age           INTEGER,
35                 gender        TEXT,
36                 region        TEXT,
37                 health_conditions TEXT,
38                 created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
39             )
40         """)
41
42         # --- Table 2: assessments ---
43         cursor.execute("""
44             CREATE TABLE IF NOT EXISTS assessments (
45                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
46                 user_id       INTEGER NOT NULL,
47                 symptoms      TEXT NOT NULL, -- raw user input
48                 extracted_symptoms TEXT, -- JSON list from GPT
49                 urgency_level TEXT, -- Emergency/Urgent/Routine/Self-Care
50                 recommendation TEXT,
51                 created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
52                 FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
53             )
54         """)
55
56         # --- Indexes for faster lookups ---
57         cursor.execute("""
58             CREATE INDEX IF NOT EXISTS idx_assessments_user_id
59             ON assessments(user_id)
60         """)
61         cursor.execute("""
62             CREATE INDEX IF NOT EXISTS idx_users_email
63             ON users(email)
64         """)
65
66         conn.commit()
67
68     except sqlite3.Error as e:
69         conn.rollback()
70         raise
71     finally:
72         conn.close()
```

Figure 5.4: Code Snippet — SQLite Schema, init_db() Function (database.py)

5.5 Dual Machine Learning Architecture

CedarCare integrates two separate machine learning approaches that together replicate a full clinical analytics stack and directly solve two of the four main data analytics goals confirmed by this research.

The **supervised part a Random Forest classifier** is used at the patient level. It was trained on a labeled Kaggle dataset of 4,920 records distributed over 41 different disease categories. It takes as input a 131-dimensional binary feature vector derived by GPT-4o-mini from symptoms extracted and weighted by clinical severity metadata. Besides a predicted disease, the model also yields a confidence score, and a confidence level of 70% serves as a safety border cases with confidence lower than this are shifted to OpenAI general guidance rather than making a dubious medical classification at low confidence.

The **unsupervised element K-Means clustering** with $k=3$ is focused on the population segment within the System Analytics Dashboard. In contrast to the Random Forest which is based on labeled data, K-Means is able to find natural groupings of patients from the synthetic population dataset of 5,000 records without predefined categories. Before clustering, features are scaled using StandardScaler to avoid the situation where Euclidean distance calculations are dominated by variables with large scales.

This deliberate separation — one ML layer diagnosing individuals, another segmenting populations — reflects real-world clinical decision support architecture where point-of-care diagnostic tools and population health analytics serve complementary but fundamentally different functions.

Type	Algorithm	Where Used	Purpose
Supervised	Random Forest Classifier	Symptom Assessment Chatbot	Predict disease/condition from labelled training data; output urgency level
Unsupervised	K-Means Clustering	System Analytics Dashboard	Discover natural patient segments from synthetic population without pre-defined labels

6. Implemented Feature 1: Secure Authentication and User Management

6.1 Overview

The authentication system is the security backbone of CedarCare. It offers functionalities like user registration and login/logout, as well as page-level access control. In fact, every page is behind authentication - those without authentication will be redirected automatically.

6.2 Technical Implementation

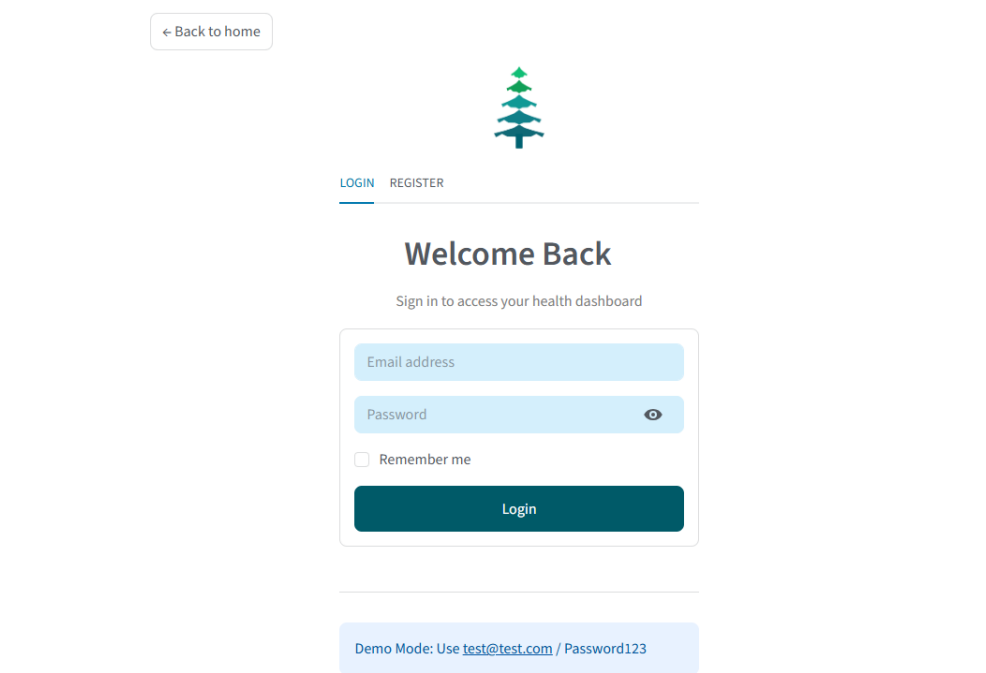
Before they are stored, passwords are hashed with bcrypt. By using bcrypt algorithm, it also includes a work factor making a brute-force attack very expensive computationally. Hence the hash that is stored will not be the plaintext password. Validation of the password is done using `bcrypt.checkpw()`. Checking the strength of the password is also done on the server side: 8 characters minimum uppercase lowercase, and digit are required. For email, a regex pattern is used to validate the format.

For handling sessions, Streamlit's `st.session_state` dictionary is used, which stores the ID, name, and email of the authenticated user. The function `require_authentication()` is executed at the beginning of each page module.

```
1 # components/auth.py - Password hashing, verification & page protection
2
3 def hash_password(password: str) -> str:
4     """Hash a plain text password using bcrypt."""
5     password_bytes = password.encode('utf-8')
6     salt = bcrypt.gensalt() # work factor default = 12 rounds
7     hashed = bcrypt.hashpw(password_bytes, salt)
8     return hashed.decode('utf-8') # store as string in SQLite
9
10
11 def verify_password(password: str, hashed_password: str) -> bool:
12     """
13     Verify a password against its stored hash.
14     bcrypt.checkpw() is timing-attack resistant - never compare hashes manually.
15     """
16     try:
17         return bcrypt.checkpw(
18             password.encode('utf-8'),
19             hashed_password.encode('utf-8')
20         )
21     except Exception:
22         return False # invalid hash format - deny access
23
24
25 def validate_password_strength(password: str) -> tuple[bool, str]:
26     """Server-side password strength check - 8 chars, letter + digit required."""
27     if len(password) < 8:
28         return (False, "Password must be at least 8 characters long")
29     if not re.search(r'[a-zA-Z]', password):
30         return (False, "Password must contain at least one letter")
31     if not re.search(r'[0-9]', password):
32         return (False, "Password must contain at least one number")
33     return (True, "Password meets all requirements")
34
35
36 def require_authentication() -> None:
37     """
38     Page protection gate - called at the top of every protected page.
39     Unauthenticated users are immediately redirected to the login page.
40     Session state stores: user_id, user_name, user_email.
41     """
42     if 'authenticated' not in st.session_state:
43         st.session_state.authenticated = False
44
45     if not st.session_state.authenticated:
46         st.switch_page("app.py") # redirect to login - st.stop() is implicit
```

Figure 6.2: Code Snippet — `hash_password()`, `verify_password()`, `validate_password_strength()` and `require_authentication()` (auth.py)

6.3 Screenshots



← Back to home



LOGIN REGISTER

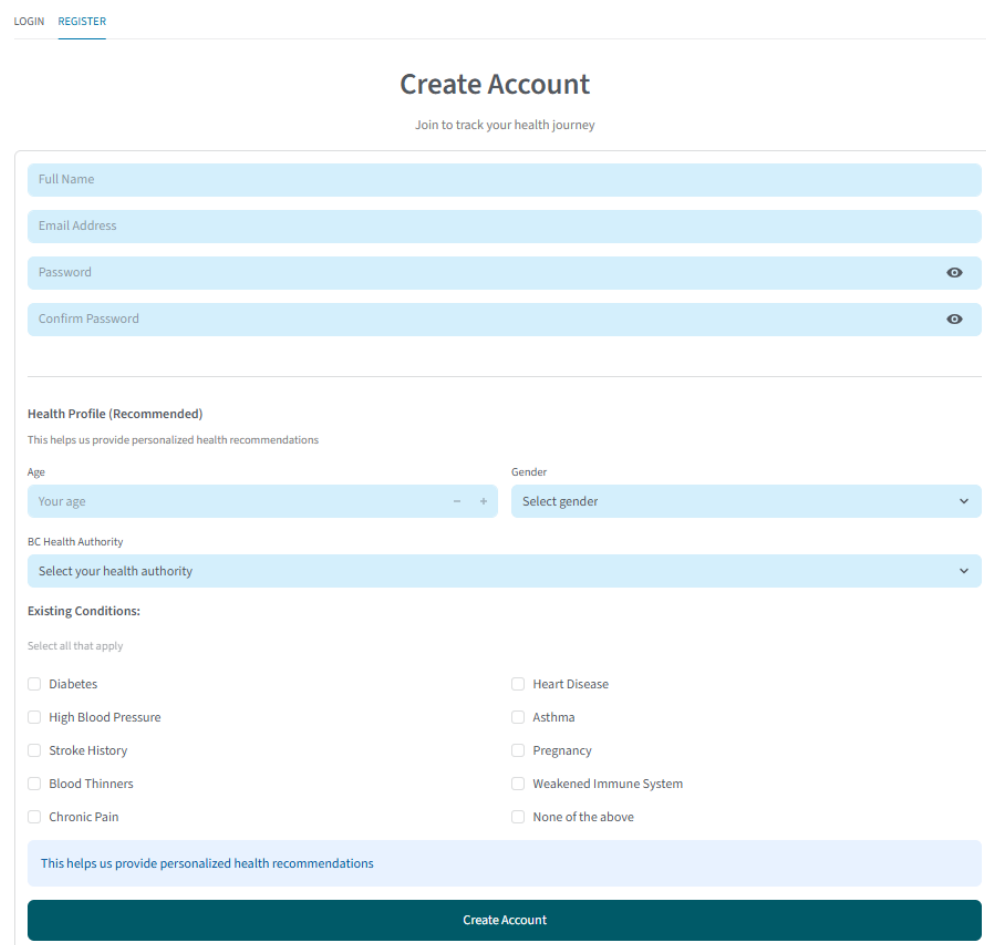
Welcome Back

Sign in to access your health dashboard


☐ Remember me

Demo Mode: Use [test@test.com](#) / Password123

Figure 6.2a: CedarCare Login Page — Secure Authentication Interface



LOGIN REGISTER

Create Account

Join to track your health journey

Health Profile (Recommended)

This helps us provide personalized health recommendations

Age

— +

Gender

▼

BC Health Authority

▼

Existing Conditions:

Select all that apply

☐ Diabetes
☐ High Blood Pressure
☐ Stroke History
☐ Blood Thinners
☐ Chronic Pain

☐ Heart Disease
☐ Asthma
☐ Pregnancy
☐ Weakened Immune System
☐ None of the above

This helps us provide personalized health recommendations

Figure 6.2b: CedarCare Registration Page — User Onboarding Form with Server-Side Validation

Summary of Logic Flow:

1. User enters email and password in the login form
2. `auth.py` validates email format using regex before any database call
3. `database.get_user_by_email()` retrieves the stored bcrypt hash
4. `bcrypt.checkpw()` compares candidate password against stored hash
5. On success: `user_id`, `user_name`, `user_email` stored in `st.session_state`
6. `require_authentication()` on all pages checks session state on every load
7. On logout: `st.session_state` cleared; user redirected to login page

7. Implemented Feature 2: Symptom Assessment Chatbot

The Symptom Assessment Chatbot is CedarCare's primary feature and most technically sophisticated component — a multi-turn conversational health triage system combining three AI layers.

7.1 Conversational Slot-Filling Architecture

The chatbot uses a slot-filling dialogue management pattern from the natural language understanding (NLU) field. A `dialog_state` dictionary accumulates information across multiple turns. The four key slots are: symptoms (list), severity (mild/moderate/severe), duration (e.g. '3 days'), and body_location (e.g. 'chest'). The chatbot continues asking follow-up questions until all relevant slots are filled — or until the maximum turn count (3) is reached.

Turn flow:

1. User describes symptoms in plain language
2. Layer 1 (GPT-4o-mini) extracts structured symptoms and any incidentally mentioned severity/duration
3. Layer 2 (CTAS rules) checks ALL accumulated symptoms every turn — if emergency detected, immediately alert
4. System determines which slots remain empty and asks the next contextual follow-up question
5. User responds via quick-select chips or free text — both paths handled equivalently
6. After 3 turns or all key slots filled, Layer 3 (Random Forest) produces the final assessment

7.2 Quick-Select Chips (st.pills)

To reduce cognitive burden, the chatbot presents quick-select chips for severity, duration, and body location questions. Chip options are context-aware: severity chips show Mild/Moderate/Severe/Unbearable; duration chips show Under 1 hour/Few hours/1–2 days/3+ days. Chips are only displayed when the chatbot has genuinely collected symptoms — a `_show_chips` session state flag prevents chips appearing after non-medical inputs.

7.3 Five Clinical Scenarios — All Pipeline Paths

Scenario	Input Example	Layer Triggered	Expected Result
Happy Path (ML)	I have a UTI — burning when I urinate	L1 → L2 (pass) → L3 (≥70%)	ML prediction with condition + urgency
Emergency (CTAS)	Severe chest pain, difficulty breathing	L1 → L2 (MATCH)	Emergency alert: CALL 911 — ML never runs
OpenAI Fallback	I feel generally unwell and sick	L1 → L2 (pass) → L3 (<70%)	OpenAI general guidance with urgency
Non-Medical Redirect	What's the weather today?	L1 (no symptoms extracted)	Polite redirect after 2 failed turns

Scenario	Input Example	Layer Triggered	Expected Result
Multi-turn Collection	Headache → Severe (chip) → 3 days (chip)	All layers across 3 turns	Complete assessment with full slot context

7.4 Code Highlight — OpenAI Symptom Extraction

```

1 # components/openai_utils.py - Multi-turn symptom extraction
2 # Layer 1 of the Three-Layer Hybrid AI Pipeline
3
4 MODEL_NAME = "gpt-4o-mini"
5 MODEL_TEMPERATURE = 0.1 # low = deterministic, consistent extractions
6
7 MULTITURN_EXTRACTION_PROMPT = """
8 You are a medical symptom extraction assistant working in a multi-turn conversation.
9 Review the full conversation history and extract ALL accumulated symptom information.
10
11 You MUST respond with ONLY valid JSON in this exact format (no extra text):
12 {
13     "symptoms": ["symptom1", "symptom2"], -- ALL symptoms across ALL turns
14     "duration": "duration string or unknown",
15     "severity": "mild or moderate or severe",
16     "body_location": "body location string or null"
17 }
18
19 Rules:
20 - Accumulate symptoms across the ENTIRE conversation - do NOT drop earlier ones.
21 - severity: infer from context (e.g. "terrible" = severe, "a bit" = mild).
22 - body_location: null if never mentioned.
23 - Keep symptom names short and clear (e.g. "chest pain", "dizziness").
24 """
25
26
27 def extract_symptoms_multiturn(user_input: str, conversation_history: list) -> dict:
28     """
29     Extract structured symptoms from full multi-turn conversation context.
30     Passes entire history to GPT so symptoms across turns are consolidated.
31     Temperature 0.3 - slightly higher than single-turn for natural variation.
32     """
33     try:
34         messages = [{"role": "system", "content": MULTITURN_EXTRACTION_PROMPT}]
35         messages.extend(conversation_history) # full prior context
36         messages.append({"role": "user", "content": user_input})
37
38         response = openai.chat.completions.create(
39             model=MODEL_NAME,
40             temperature=0.3,
41             messages=messages,
42         )
43         raw = response.choices[0].message.content.strip()
44         return json.loads(raw) # always returns clean dict
45
46     except json.JSONDecodeError:
47         return {"error": "Failed to parse symptom data from AI response."}
48     except openai.RateLimitError:
49         return {"error": "API rate limit reached. Please try again."}
50     except Exception as e:
51         return {"error": f"Unexpected error: {e}"}

```

Figure 7.4: Code Snippet — `extract_symptoms_multiturn()` and System Prompt Schema (`openai_utils.py`)

7.5 Screenshots

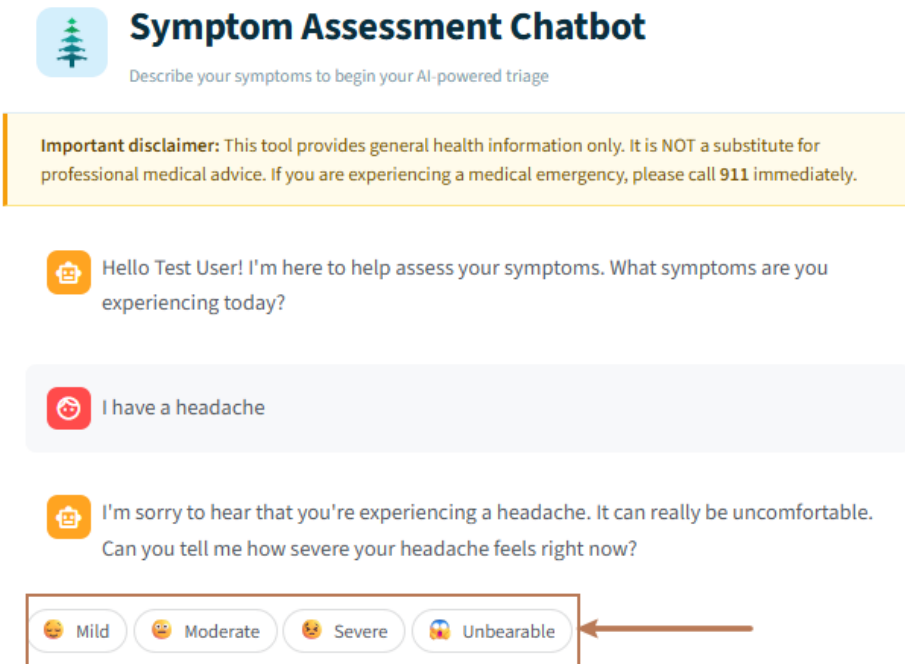


Figure 7.5a: CedarCare Symptom Assessment Chatbot — Multi-Turn Conversation with Severity Chip Selection

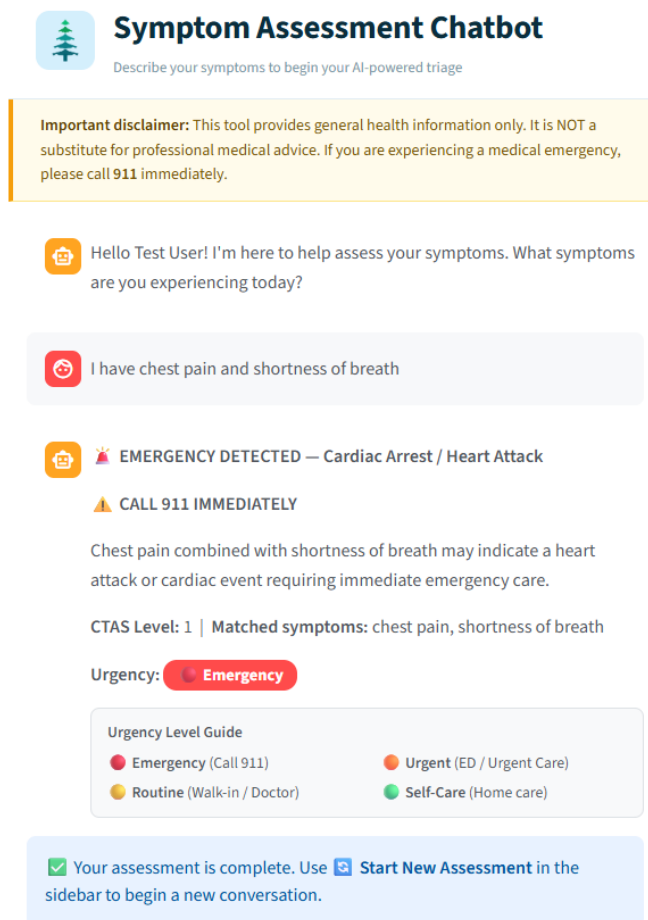


Figure 7.5b: CedarCare Symptom Assessment Chatbot — CTAS Level 1 Emergency Detection (Cardiac Arrest / Heart Attack)



Symptom Assessment Chatbot

Describe your symptoms to begin your AI-powered triage

Important disclaimer: This tool provides general health information only. It is NOT a substitute for professional medical advice. If you are experiencing a medical emergency, please call **911** immediately.



Hello Test User! I'm here to help assess your symptoms. What symptoms are you experiencing today?



I have burning micturition, bladder discomfort, foul smell of urine and continuous feel of urine



I'm sorry to hear that you're experiencing such uncomfortable symptoms. It sounds quite distressing. Can you tell me how severe these symptoms have been for you?



Severe



I can imagine how challenging this must be for you. How long have you been dealing with these severe symptoms?



3+ days



Extracted Symptoms: burning micturition, bladder discomfort, foul smell of urine, continuous feel of urine

Predicted Condition: Urinary tract infection (100% confidence)

Urgency: Routine

Recommendation: Consider booking an appointment with your family doctor. Monitor your symptoms.

BC Health Resources

- **HealthLink BC:** Call 8-1-1 (free health advice, 24/7)
- Find a walk-in clinic near you on the **Facility Finder** page

Urgency Level Guide

- Emergency (Call 911)
- Routine (Walk-in / Doctor)

- Urgent (ED / Urgent Care)
- Self-Care (Home care)

Save Assessment

Figure 7.5c: CedarCare Symptom Assessment Chatbot — Random Forest ML Prediction Result (Urinary Tract Infection, 100% Confidence)

Important disclaimer: This tool provides general health information only. It is NOT a substitute for professional medical advice. If you are experiencing a medical emergency, please call **911** immediately.

 Hello Test User! I'm here to help assess your symptoms. What symptoms are you experiencing today?

 I feel generally unwell, a bit foggy and slightly off

 I'm sorry to hear that you're feeling this way; it sounds uncomfortable. Can you tell me how severe your symptoms are? Are they affecting your daily activities in any way?

 😞 Severe

 That sounds really tough to deal with, especially if it's severe. How long have you been experiencing these symptoms?

 3+ days

 **Extracted Symptoms:** general unwellness, foggy, slightly off, general malaise, foggy feeling, off

Our model couldn't make a confident prediction. Here's general guidance:

It's important to seek medical attention within 24 hours, as severe malaise and fogginess can indicate an underlying issue that needs evaluation. In the meantime, ensure you stay hydrated, rest, and monitor any additional symptoms. This is not medical advice.

Urgency:  **Urgent**

BC Health Resources

- HealthLink BC: Call 8-1-1 (free health advice, 24/7)
- Find a walk-in clinic near you on the [Facility Finder](#) page

Urgency Level Guide

 Emergency (Call 911)

 Routine (Walk-in / Doctor)

 Urgent (ED / Urgent Care)

 Self-Care (Home care)

 Save Assessment

Figure 7.5d: CedarCare Symptom Assessment Chatbot — OpenAI Fallback Result (Low ML Confidence, GPT-4o-mini General Guidance Provided)

Summary of Logic Flow:

1. User types symptom description in `st.chat_input()`

2. `process_multiturn_message()` called — `turn_count` incremented; message added to `dialog_state`
3. `extract_symptoms_multiturn()` sends user input + conversation history to GPT-4o-mini
4. Extracted symptoms/severity/duration/location merged into `dialog_state` (deduplication applied; severity-word blacklist prevents GPT from misclassifying severity descriptors as duration values)
5. `check_critical_symptoms()` runs on ALL accumulated symptoms — if CTAS match: return emergency result immediately
6. `get_follow_up_question()` determines next question based on empty slots
7. If more slots needed: `generate_follow_up_response()` phrases the question naturally;
`st.session_state._show_chips = True` renders context-aware chip options (severity / duration / body location) outside the chat loop for persistence across reruns
8. If all slots filled or `turn_count` ≥ 3 : `_ml_predict()` runs Random Forest on accumulated symptom vector
9. If RF confidence $\geq 70\%$: return `ml_prediction` result with predicted disease and urgency
10. If RF confidence $< 70\%$: call `get_openai_fallback_guidance()` and return `openai_fallback` result
11. UI renders result card with urgency badge, BC Health Resources, and Save Assessment button

8. Implemented Feature 3: Personal Health Dashboard

The Personal Health Dashboard provides authenticated users with a longitudinal view of their health assessment history, combining SQL-driven data retrieval with Plotly interactive charts.

8.1 KPI Cards

Four KPI cards draw from the authenticated user's assessment history in SQLite:

- Total Assessments — total count of saved assessment records
- Last Check — relative time since most recent assessment (e.g. '3 days ago')
- Most Common Symptom — most frequently appearing symptom across all assessments
- Most Recent Urgency — urgency level from the user's most recent assessment

8.2 Analytical Charts

- Symptom Trends Over Time — line chart showing assessment frequency over 30 days; graceful fallback when insufficient data
- Top Symptoms — horizontal bar chart showing most frequently reported symptoms
- Urgency Distribution — donut chart showing proportion of assessments at each urgency level

8.3 Filtering

Date range and urgency level filters update all charts and the assessment history table simultaneously, applied as SQL WHERE clauses via parameterised queries.

8.4 Screenshots

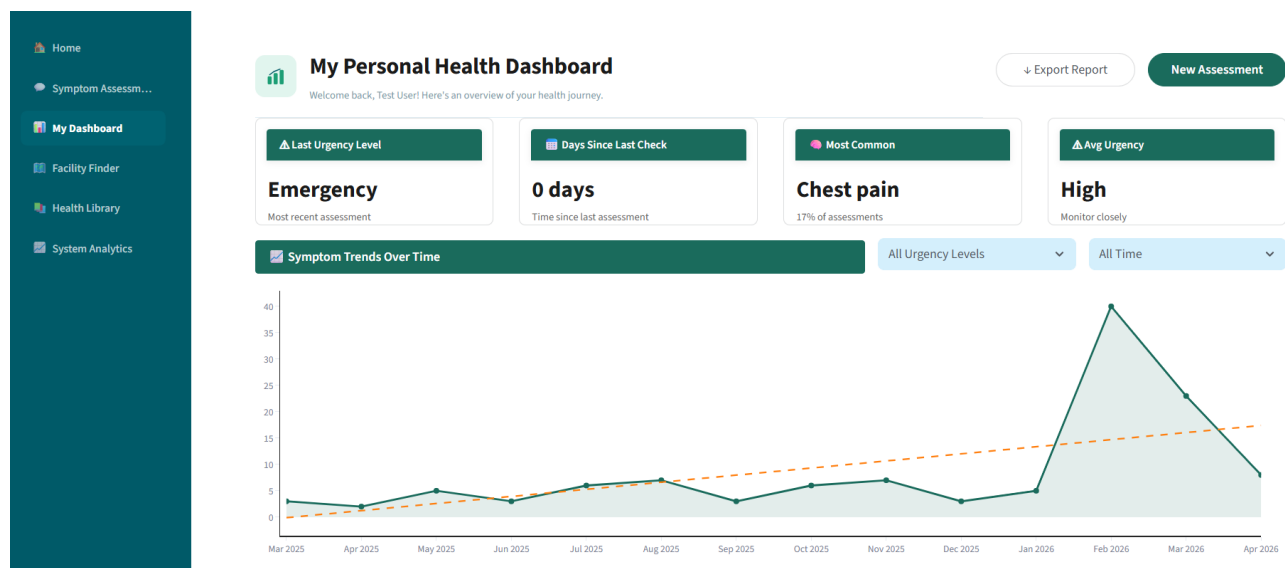
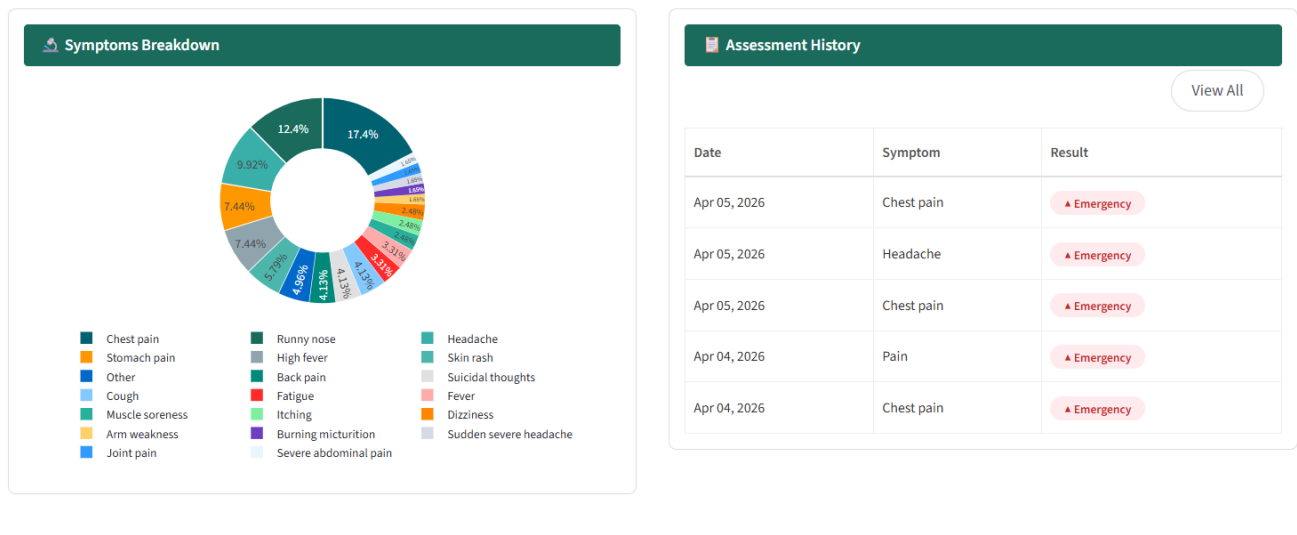


Figure 8.4a: CedarCare Personal Health Dashboard — KPI Summary Cards and Symptom Trends Over Time Chart



© 2026 CedarCare. All rights reserved.
[Privacy Policy](#) | [Terms of Service](#) | [Contact Us](#) | [Help](#)

Figure 8.4b: CedarCare Personal Health Dashboard — Symptoms Breakdown Donut Chart and Assessment History Log

Summary of Logic Flow:

1. `require_authentication()` verifies user session; retrieves `user_id`
2. `get_assessments_by_user_id(user_id)` queries SQLite for all user assessments
3. Date range and urgency filters applied to the fetched DataFrame using pandas boolean indexing; filtered dataset passed downstream to all KPI and chart computations
4. KPI values computed from filtered dataset (count, last date, most common symptom, last urgency)
5. Plotly charts rendered inline: line chart (trend), bar chart (symptoms), donut (urgency distribution)
6. Assessment history table displayed with urgency-coloured badges

9. Implemented Feature 4: BC Healthcare Facility Finder

The BC Healthcare Facility Finder helps users locate appropriate healthcare facilities. It is CedarCare's most BC-specific differentiator — no global competitor provides BC-specific care navigation integrated with symptom triage.

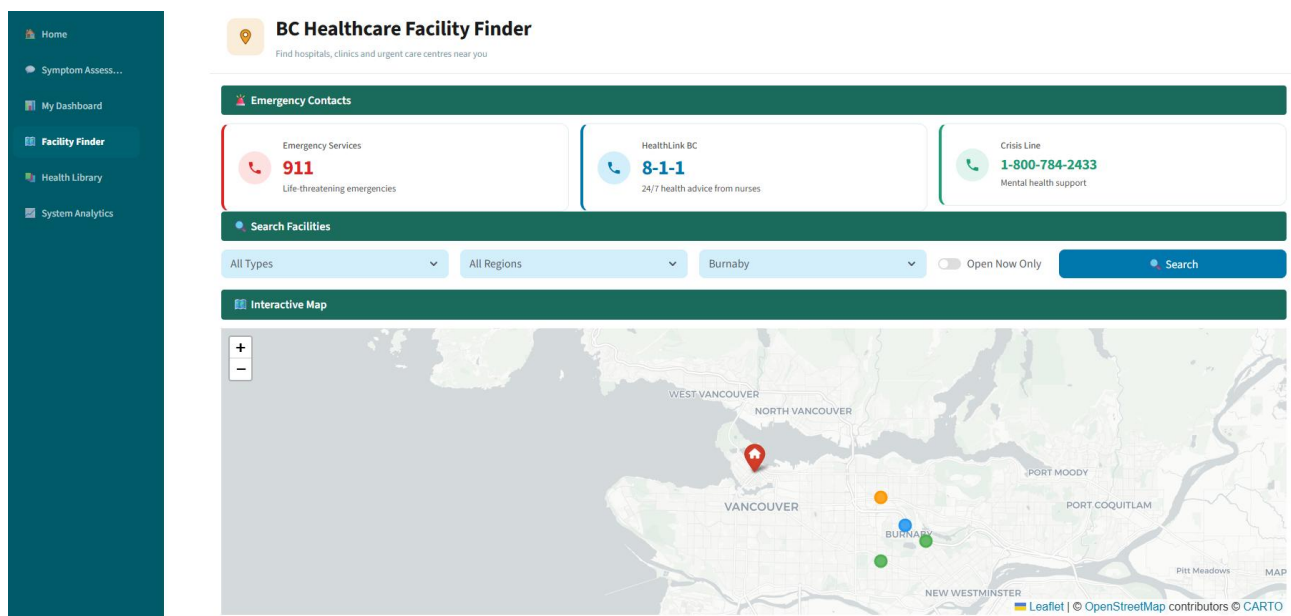
9.1 Interactive Map

The facility map is built with Folium and rendered via Streamlit's `components.html()` wrapper. Facilities are represented as map markers with popup information cards showing facility name, type, address, and phone number. The map defaults to the Greater Vancouver region and supports free panning and zooming.

9.2 Emergency Contacts Panel

- BC Emergency Services — 911
- HealthLink BC — 8-1-1 (free 24/7 nurse advice line)
- BC Poison Control — 1-800-567-8911
- Crisis Line BC — 310-6789

9.3 Screenshots



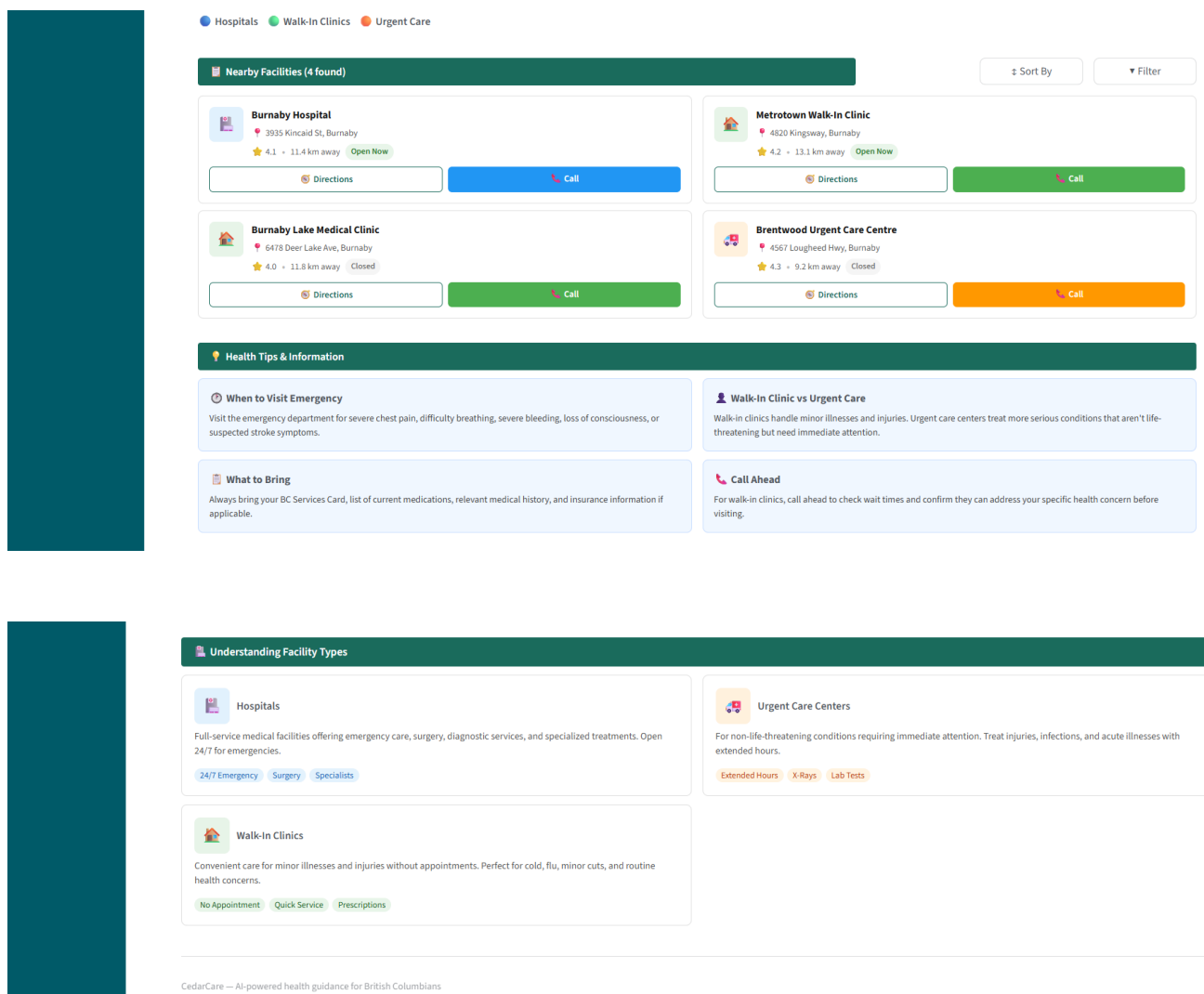


Figure 9.2: CedarCare BC Healthcare Facility Finder — Interactive Map, Nearby Facilities Search Results (Burnaby), and Facility Type Guide

Summary of Logic Flow:

1. Facility data loaded from static JSON data file (hospital/clinic records)
2. User selects filters: facility type, region, city, and Open Now toggle
3. Filtered facility list computed from JSON using pandas Boolean indexing
4. Folium Map object initialized centered on Greater Vancouver (lat/lng defaults)
5. Filtered facility markers added with color-coded popups by type (blue = hospital, green = walk-in, orange = urgent care)
6. Folium map serialized to HTML and rendered via `st.components.v1.html()`
7. Facility result cards rendered below map — each showing name, distance, rating, Open/Closed status, Directions and Call buttons
8. Emergency contacts panel (911, HealthLink BC 8-1-1, Crisis Line) rendered as static styled cards
9. Health Tips and Facility Type guide rendered as informational cards below results

10. Implemented Feature 5: Health Education Library

The Health Education Library provides BC residents with access to 18 evidence-based health articles across 8 clinical categories.

10.1 Knowledge Base Architecture

Article content is stored in `health_articles.json`. Each article contains: title, category, summary, full content, estimated reading time, and a `source_url` field linking to the corresponding HealthLinkBC page for verification. The JSON approach was chosen for simplicity — no database migration required when adding articles.

10.2 Navigation Features

- Category Browse — 8 clickable category cards (Pain & Injury, Cold & Flu, Medications, Prevention & Wellness, Mental Health, Heart Health, Child Health, Emergency Care)
- Keyword Search — full-text search across article titles and content
- Popular Topics — 4 featured articles displayed as highlighted cards

10.3 Screenshots

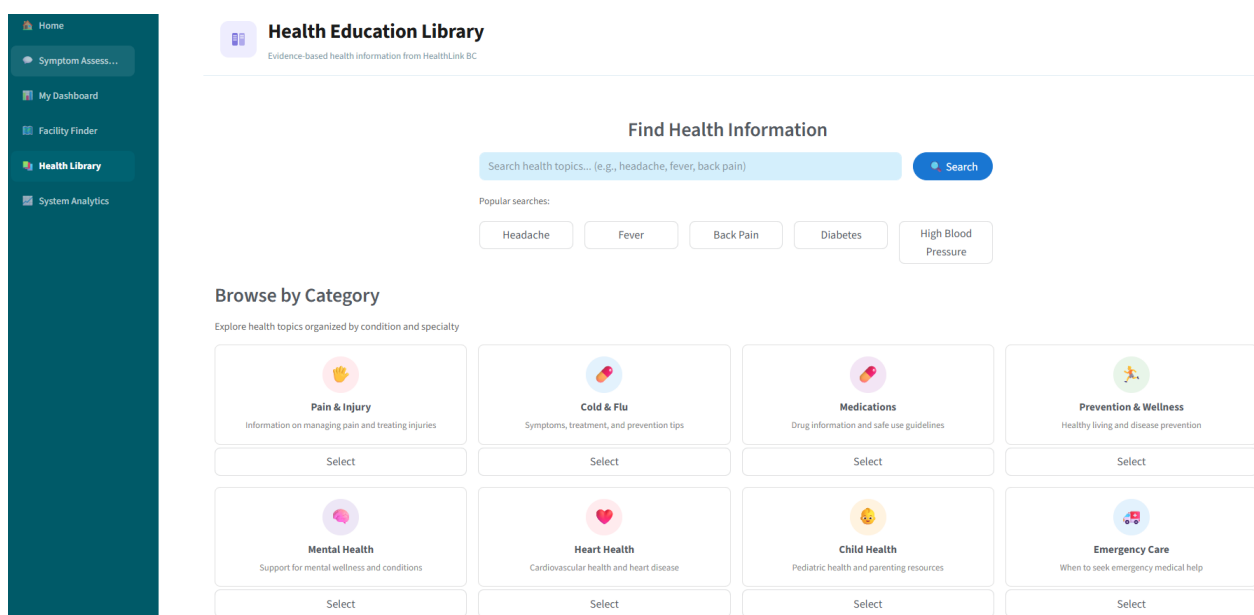


Figure 10.2a: CedarCare Health Education Library — Search Interface and Browse by Category

Browse by Category

Explore health topics organized by condition and specialty



Pain & Injury
Information on managing pain and treating injuries

Select



Cold & Flu
Symptoms, treatment, and prevention tips

Select



Medications
Drug information and safe use guidelines

Select



Prevention & Wellness
Healthy living and disease prevention

Select



Mental Health
Support for mental wellness and conditions

Select



Heart Health
Cardiovascular health and heart disease

Select



Child Health
Pediatric health and parenting resources

✓ Selected



Emergency Care
When to seek emergency medical help

Select

Articles in 'Child Health' (2 articles found)

Clear Filter

Child Health

Childhood Vaccinations in BC

BC's immunization schedule for children and why vaccines are important for your child's health

4 min read

Read More →

Vaccinations are one of the safest and most effective ways to

Child Health

Common Childhood Illnesses: A Parent's Guide

Recognizing and managing common illnesses in children including fever, ear infections, and stomach bugs

4 min read

Read More →

Figure 10.2b: CedarCare Health Education Library — Category Filter Selection (Child Health) with Article Results

Popular Topics



Emergency Care

When to Go to Emergency vs Walk-in Clinic

Learn the difference and save time by choosing the right care option for your medical needs

Read More →



Pain & Injury

Managing Headaches at Home

Simple remedies and when to seek help. Understand different types of headaches and effective treatments

Read More →



Mental Health

Managing Stress & Anxiety

Techniques for better mental wellness including mindfulness, breathing exercises, and coping strategies

Read More →



Heart Health

Heart Health Essentials

Tips for cardiovascular wellness including diet, exercise, and understanding heart disease risk factors

Read More →

Figure 10.2c: CedarCare Health Education Library — Popular Topics Overview with Article Cards

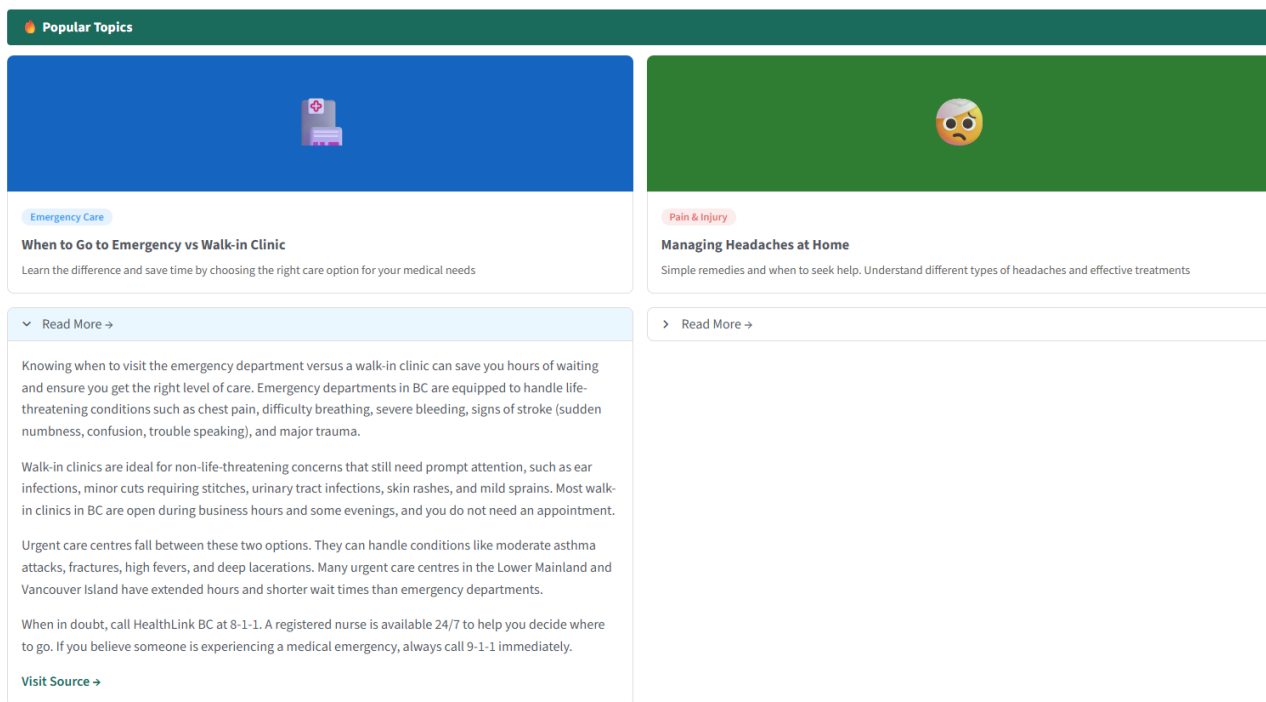


Figure 10.2d: CedarCare Health Education Library — Article Expanded View ("When to Go to Emergency vs Walk-in Clinic")

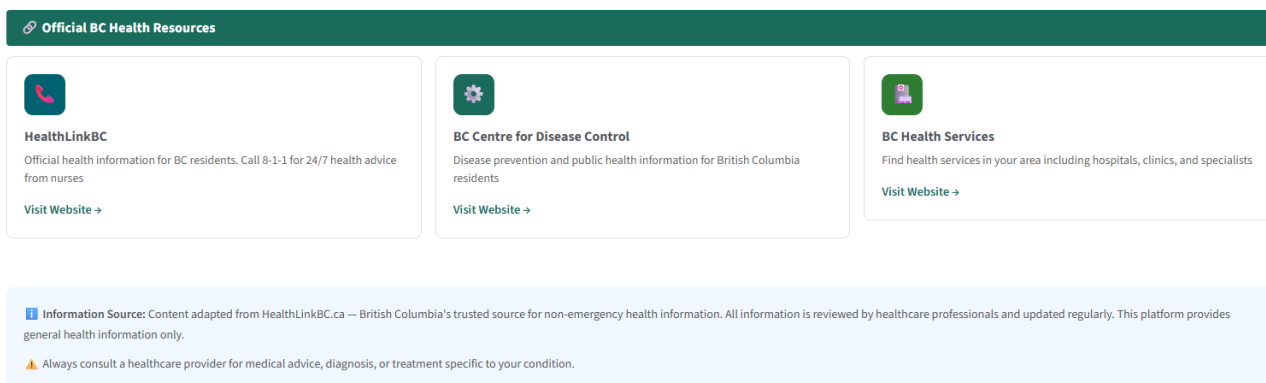


Figure 10.2e: CedarCare Health Education Library — Official BC Health Resources Panel and Information Source Disclaimer

Summary of Logic Flow:

1. health_articles.json loaded using `pathlib.Path(__file__).parent` for reliable path resolution
2. User selects navigation mode: Category Browse, Search, or Popular Topics; popular search chips (Headache, Fever, Back Pain, Diabetes, High Blood Pressure) rendered as quick-select buttons
3. Category filter: articles filtered by category field in JSON
4. Search: full-text match on title + content fields (case-insensitive)
5. Article cards rendered with `st.expander()` for the Read More section; 18 articles across 8 categories sourced from HealthLinkBC content
6. `source_url` field provides link to HealthLinkBC for authoritative verification
7. Official BC Health Resources panel rendered as static cards linking to HealthLinkBC, BC Centre for Disease Control, and BC Health Services websites; information source disclaimer displayed at the bottom of every page

11. Implemented Feature 6: System Analytics Dashboard

The System Analytics Dashboard is the platform's population-level analytics component, applying unsupervised K-Means clustering to a synthetic population health dataset of 5,000 records representing 200 BC patients. In a production deployment, this pipeline would be connected to the live SQLite database, replacing synthetic data with real anonymized user assessments over time.

11.1 Synthetic Dataset

A purpose-built 5,000-record synthetic dataset (`synthetic_assessments.csv`) was generated in Sprint 4A, simulating 200 patients across three engagement tiers:

Tier	Patients	Age Range	Assessments/Year	Urgency Profile
Tier 1 — Healthy/Occasional	80 (40%)	18–45	3–8	70% Routine / 2% Emergency
Tier 2 — Moderate Users	80 (40%)	30–65	15–25	50% Routine / 5% Emergency
Tier 3 — Chronic/High- Frequency	40 (20%)	50–85	50–80	30% Routine / 7% Emergency

Realistic patterns embedded: flu-season spike (Nov–Feb, 40% increased cold/flu frequency), tier-based chronic symptom elevation, weekday usage bias. Random seed 42 ensures reproducibility.

11.2 K-Means Patient Segmentation

Each of 200 patients is aggregated into four features: mean age, assessment count, urgent ratio (proportion of Urgent+Emergency records), and chronic ratio (proportion of serious/chronic symptoms). Features are normalised to zero mean and unit variance using `StandardScaler` before fitting `KMeans(n_clusters=3, random_state=42)`.

The three resulting clusters are automatically named by sorting on a combined (`chronic_ratio` + `urgent_ratio`) score:

- **Low Risk** — younger, infrequent users with predominantly routine assessments
- **Moderate Risk** — middle-aged users with mixed urgency profiles
- **High Risk** — older, high-frequency users with elevated chronic and urgent symptom proportions

The cluster scatter plot visualizes segments with **X = assessment frequency**, **Y = urgent case ratio**, and bubble size proportional to patient age — a format familiar from population health management research (Vrbaski et al., 2019).

```

1 # pages/6__System_Analytics.py - K-Means Patient Clustering
2 # Unsupervised ML: discovers natural patient segments without predefined labels
3
4 _SERIOUS_SYMPTOMS = {"chest pain", "shortness of breath",
5                      "high blood pressure", "diabetes symptoms"}
6
7 # — Step 1: Aggregate per-patient features —————
8 patient_agg = df.groupby("patient_id").agg(
9     mean_age = ("age", "mean"),
10    assessment_count = ("patient_id", "size"),
11    urgent_ratio = ("urgency_level", lambda x: (x.isin(["Urgent", "Emergency"])).mean()),
12    chronic_ratio = ("symptom", lambda x: x.isin(_SERIOUS_SYMPTOMS).mean()),
13 ).reset_index()
14
15 # — Step 2: Normalise features (Euclidean distance requires equal scale) ———
16 features = ["mean_age", "assessment_count", "urgent_ratio", "chronic_ratio"]
17 scaler = StandardScaler()
18 X_scaled = scaler.fit_transform(patient_agg[features])
19
20 # — Step 3: Fit K-Means (k=3, fixed seed for reproducibility) —————
21 kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
22 patient_agg["cluster"] = kmeans.fit_predict(X_scaled)
23
24 # — Step 4: Auto-name clusters by combined urgency + chronic risk score ———
25 cluster_scores = (
26     patient_agg.groupby("cluster")[["chronic_ratio", "urgent_ratio"]]
27     .mean()
28     .sum(axis=1)
29     .sort_values(ascending=False) # highest risk → first name
30 )
31 _CLUSTER_NAMES = ["Chronic/Elderly Patients", "High-Risk Patients",
32                  "Healthy/Occasional Users"]
33 cluster_name_map = {cid: name for cid, name in zip(cluster_scores.index, _CLUSTER_NAMES)}
34 patient_agg["cluster_name"] = patient_agg["cluster"].map(cluster_name_map)
35
36 # — Step 5: Render interactive Plotly scatter plot —————
37 fig = px.scatter(
38     patient_agg,
39     x = "assessment_count", # x-axis: how often patient uses the platform
40     y = "urgent_ratio", # y-axis: proportion of urgent/emergency cases
41     color = "cluster_name", # colour-code by cluster
42     size = "mean_age", # bubble size encodes patient age
43     title = "Patient Segments by Assessment Frequency and Urgency Profile",
44 )
45 st.plotly_chart(fig, use_container_width=True)

```

Figure 11.2: Code Snippet — K-Means Patient Clustering Pipeline (6__System_Analytics.py)

11.3 Screenshots

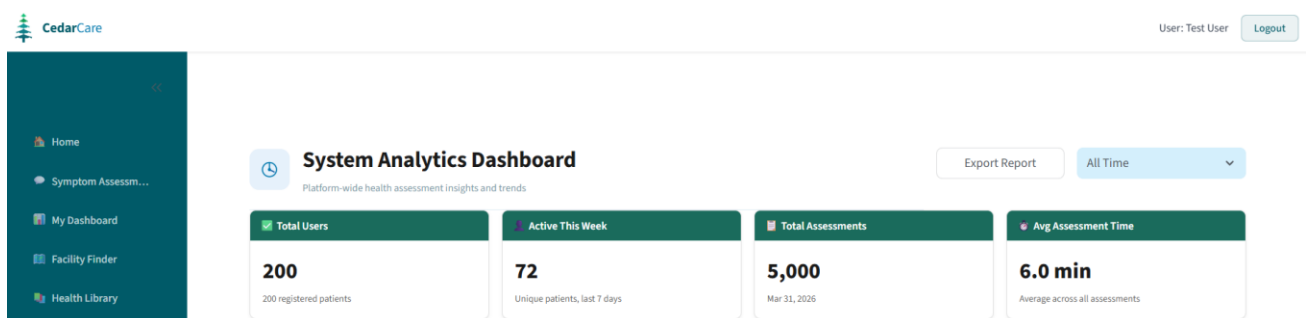


Figure 11.3a: CedarCare System Analytics Dashboard — Platform-Wide KPI Summary (200 Users, 5,000 Assessments)



Figure 11.3b: CedarCare System Analytics Dashboard — Top Symptoms Bar Chart, Urgency Distribution and Usage by Health Authority

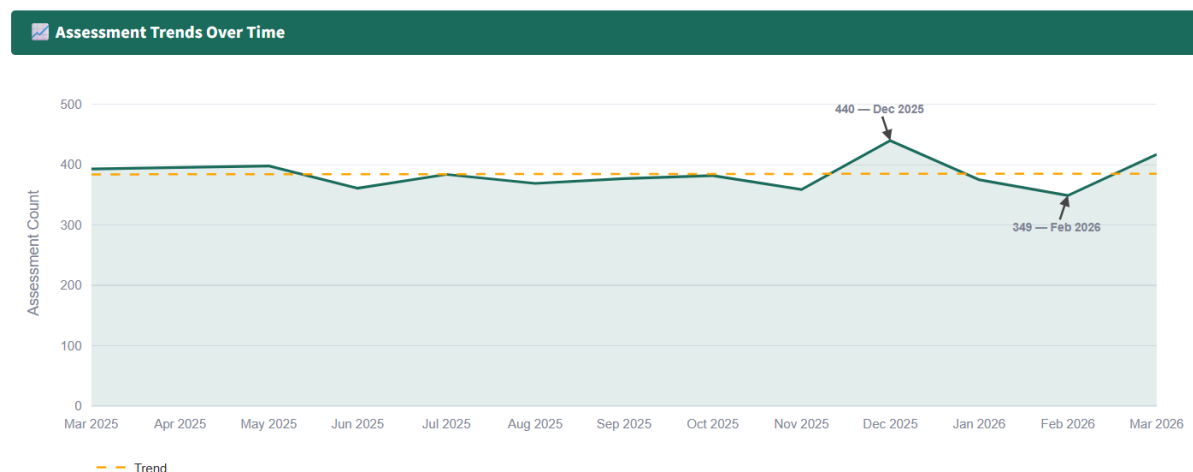


Figure 11.3c: CedarCare System Analytics Dashboard — Assessment Trends/Count Over Time with Linear Trendline (Mar 2025 – Mar 2026)



Figure 11.3d: CedarCare System Analytics Dashboard — K-Means Patient Cluster Analysis (3 Segments: Chronic/Elderly, High-Risk, Healthy/Occasional Users)

Summary of Logic Flow:

1. synthetic_assessments.csv loaded using `pathlib.Path(__file__).parent` for reliable path resolution
2. Patient-level features aggregated from assessment records (mean age, count, urgent_ratio, chronic_ratio)
3. `StandardScaler()` normalises all four features to zero mean, unit variance
4. `KMeans(n_clusters=3, random_state=42, n_init=10).fit_predict()` applied to scaled feature matrix — `random_state=42` ensures reproducible cluster assignments across reruns
5. Cluster labels assigned back to patient records
6. Clusters auto-named by sorting on combined (chronic_ratio + urgent_ratio) score
7. Plotly scatter plot rendered: X=assessment_count, Y=urgent_ratio, size=mean_age, colour=cluster
8. KPI cards (Total Users, Active This Week, Total Assessments, Avg Assessment Time), top symptoms bar chart, urgency distribution donut, and BC Health Authority breakdown all rendered from date-filtered dataset; Export Report button serialises filtered data to CSV

12. Evaluation Techniques

12.1 Automated Developer Testing Suite (Sprint 5A)

A 95-test automated suite was built using pytest, Streamlit AppTest, and Hypothesis property-based testing across four testing approaches:

File	Type	Tests	Key Validated Behaviour
test_auth.py	Unit	10	Registration, login, bcrypt hashing, session management, page protection
test_ctas_rules.py	Unit	23	14 CTAS rule positive detection, 5 negative pass-throughs, 4 edge cases
test_database.py	Unit	12	User CRUD, Assessment CRUD, data isolation, schema initialisation
test_ml_model.py	Unit	12	Model loading, metadata completeness, confidence threshold, input/output validation
test_pipeline.py	Integration	6	Emergency circuit breaker, ML path, OpenAI fallback, emergency precedence
test_chatbot_scenarios.py	Integration	6	Cardiac emergency, stroke, ML prediction (UTI), fallback, emergency override
test_dashboard_apptest.py	UI (AppTest)	8	Dashboard page load, KPI rendering, filter application
test_chatbot_apptest.py	UI (AppTest)	8	Chatbot page load, message submission, state management
test_fuzz_hypothesis.py	Property-Based	10	Password/email/CTAS fuzzing; database corruption resistance

```

(venv) PS C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform> python -m pytest tests/unit/test_ctas_rules.py -v
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\OneDrive\Desmond_New\BC-Health-Platform
plugins: anyio-4.12.1
collected 23 items

tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Cardiac Arrest / Heart Attack] PASSED [ 4%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Stroke (FAST Signs)] PASSED [ 8%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Unconsciousness / Unresponsive] PASSED [ 13%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Severe Airway Obstruction] PASSED [ 17%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Poisoning / Overdose] PASSED [ 30%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Difficulty Breathing] PASSED [ 34%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Severe Allergic Reaction / Anaphylaxis] PASSED [ 39%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Chest Pain] PASSED [ 43%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Severe Head Injury] PASSED [ 47%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Suspected Meningitis] PASSED [ 52%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Severe Burns] PASSED [ 56%]
tests/unit/test_ctas_rules.py::TestPositiveDetection::test_rule_triggers_correctly[Mental Health Crisis \u0026 Suicidal Thoughts / Self-Harm] PASSED [ 60%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc01_sore_throat PASSED [ 65%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc02_headache_fatigue PASSED [ 69%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc03_runny_nose_mild_cough PASSED [ 73%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc04_empty_list PASSED [ 78%]
tests/unit/test_ctas_rules.py::TestNegativePassthrough::test_nc05_fever_alone PASSED [ 82%]
tests/unit/test_ctas_rules.py::TestEdgeCases::test_ec01_single_exact_keyword PASSED [ 86%]
tests/unit/test_ctas_rules.py::TestEdgeCases::test_ec02_mixed_case_input PASSED [ 91%]
tests/unit/test_ctas_rules.py::TestEdgeCases::test_ec03_large_non_emergency_list PASSED [ 95%]
tests/unit/test_ctas_rules.py::TestEdgeCases::test_ec04_none_input_does_not_crash PASSED [100%]

===== 23 passed in 0.18s =====

```

Figure 12.1: pytest Unit Test Output — test_ctas_rules.py (23/23 Tests Passing, Covering All 14 CTAS Emergency Rules, Negative Pass-throughs and Edge Cases)

12.2 Continuous Integration Pipeline (Sprint 5B)

GitHub Actions CI/CD pipeline runs automatically on every push and pull request to the main branch, using Ubuntu 22.04 with Python 3.11 — matching the Streamlit Cloud deployment environment. The pipeline runs 79 of 95 total tests (16 AppTest UI tests excluded due to module path conflicts when running outside src/). Final CI result: 79/79 passed in 13.72 seconds.

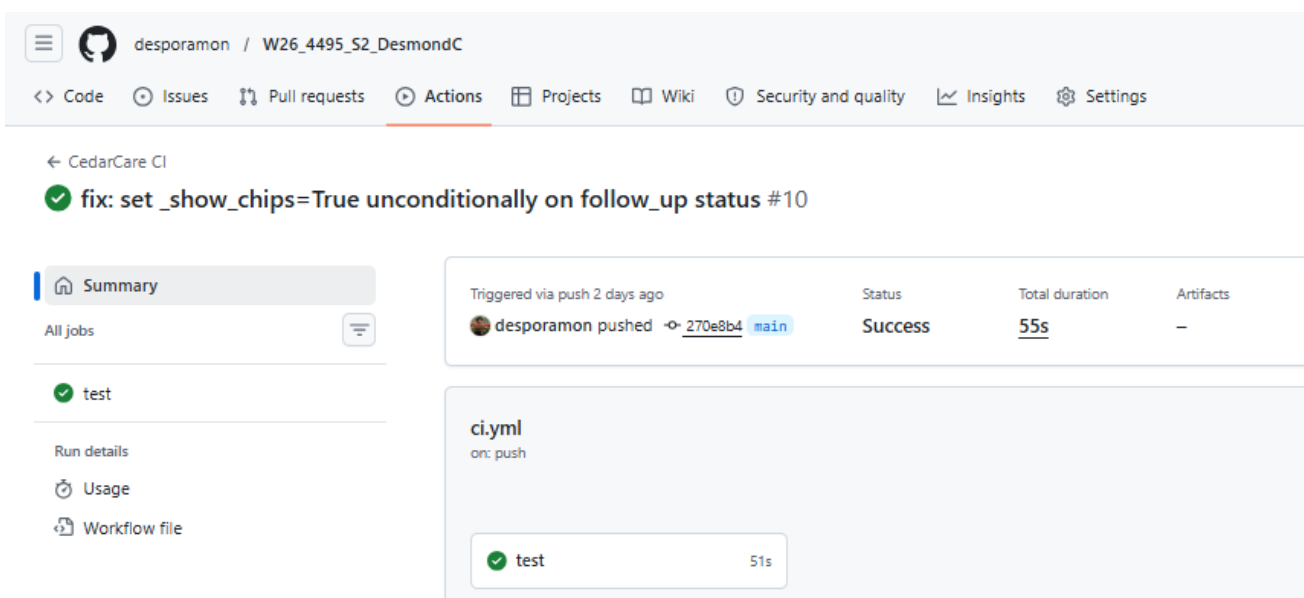


Figure 12.2: GitHub Actions CI/CD Pipeline — Automated Test Run Triggered on Push to Main Branch (Status: Success, 79/79 Tests Passing)

12.3 User Evaluation Survey

12 users completed the CedarCare User Evaluation Survey after accessing the live deployment and completing at least one full symptom assessment.

Question	Avg (/5)	% Rating 4-5	Key Insight
Q1: Ease of navigation and use	4.33	83.3%	Strong majority found CedarCare easy to use
Q2: Chatbot helpfulness and accuracy	4.09	75%	Chatbot received strong positive reception
Q3: Most useful feature	Chatbot 83.3%	N/A	Symptom Assessment Chatbot dominant feature
Q4: Urgency recommendation appropriateness	4.18	—	Triage logic validated as clinically appropriate
Q5: Likelihood to recommend CedarCare	4.42	83.3%	High net promoter sentiment

1. How easy was it to navigate and use the CedarCare platform overall?

12 responses

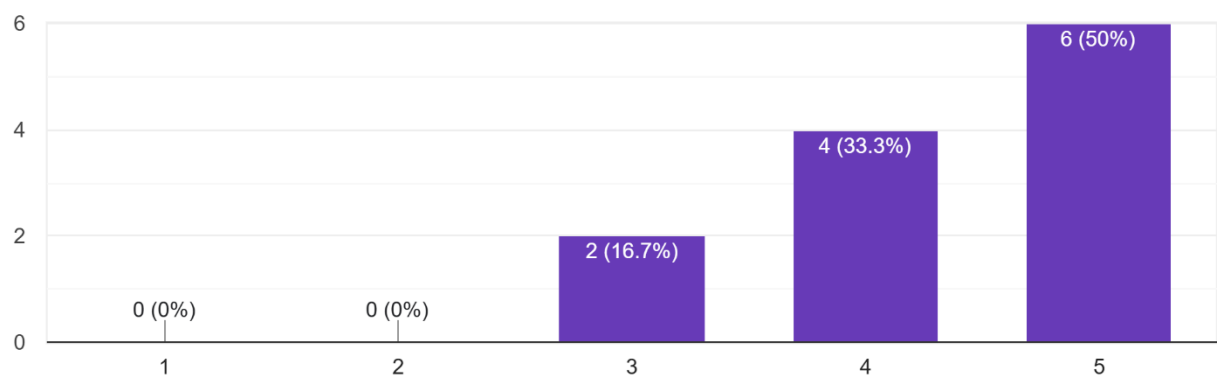


Figure 12.3a: User Evaluation Survey — Q1: Ease of Navigation and Use (12 Responses, Avg 4.33/5)

2. Please rate how helpful and accurate the Symptom Assessment chatbot was in guiding your next steps.

12 responses

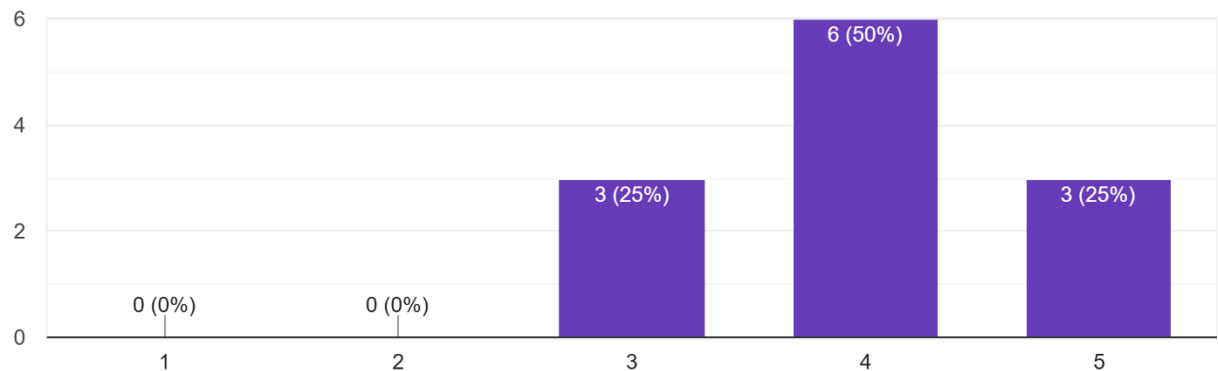


Figure 12.3b: User Evaluation Survey — Q2: Symptom Assessment Chatbot Helpfulness and Accuracy (12 Responses, Avg 4.00/5)

3. Which feature(s) did you find most useful during your last visit to CedarCare? (Select all that apply)

12 responses

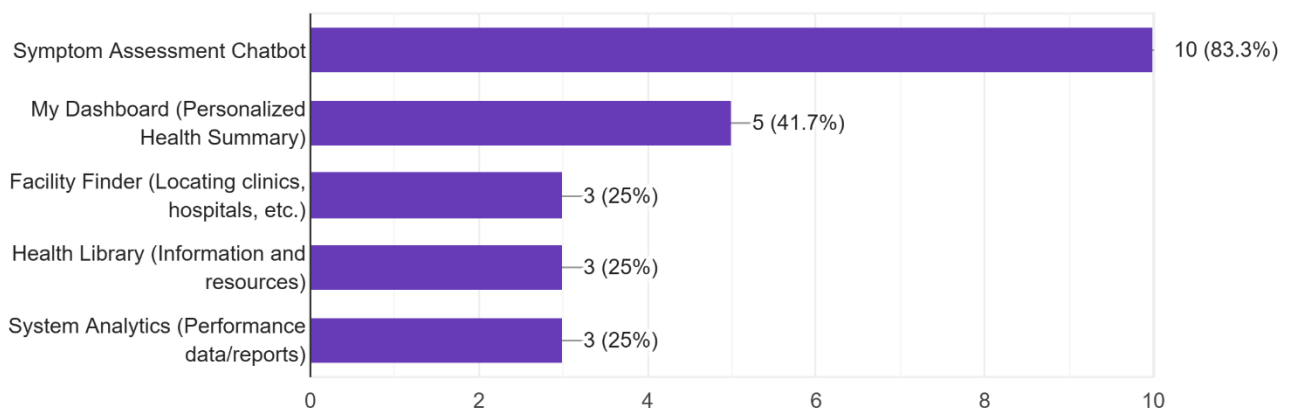


Figure 12.3c: User Evaluation Survey — Q3: Most Useful Platform Features (12 Responses, Symptom Assessment Chatbot 83.3%)

4. Did the urgency level recommendation provided by CedarCare seem appropriate for the symptoms you entered?

12 responses

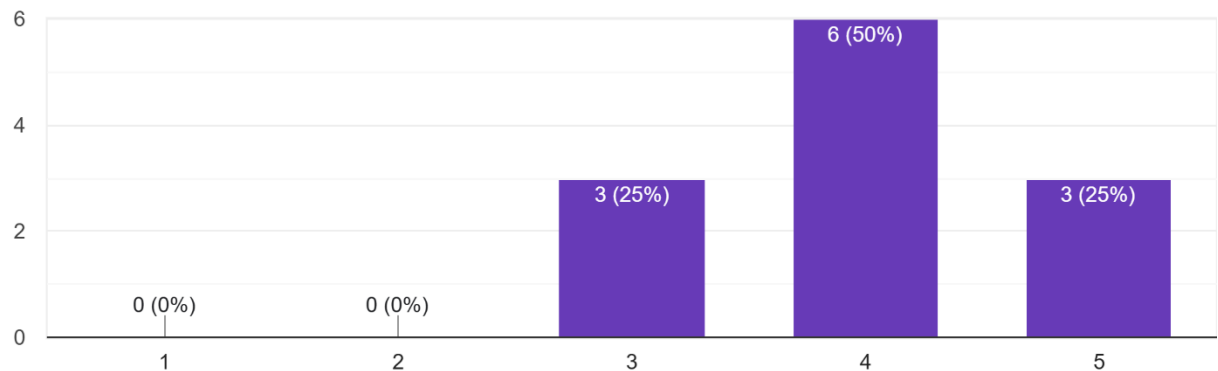


Figure 12.3d: User Evaluation Survey — Q4: Urgency Level Recommendation Appropriateness (12 Responses, Avg 4.00/5)

5. How likely are you to recommend CedarCare to other residents of British Columbia?

12 responses

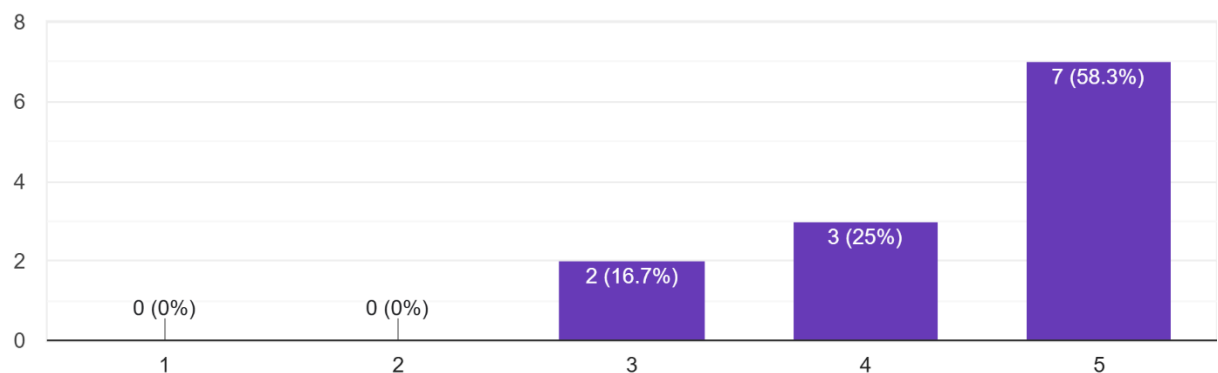


Figure 12.3e: User Evaluation Survey — Q5: Likelihood to Recommend CedarCare to BC Residents (12 Responses, Avg 4.42/5)

Selected open-ended suggestions (Q6):

- 'Very intuitive. Would love appointment booking in a future version.'
- 'The facility finder was really useful. Response speed could be improved.'
- 'Emergency detection is impressive and gave me confidence in the results.'
- 'Would like push notifications for health reminders.'
- 'BC-specific resources like HealthLink 8-1-1 were very relevant.'

Implementation changes in response: Dashboard KPI cards were redesigned to replace generic counters with clinically relevant metrics (Most Recent Urgency Level, Days Since Last Assessment).

Appointment booking, push notifications, and French language support were logged as future improvement items.

12.3.1 Implementation Changes in Response to Evaluation Feedback

User testing took two forms: a structured Google Forms survey (12 respondents) and informal usability observations from housemates testing the live deployment. I used both to decide what to change in the final sprint.

Landing page implementation

The platform originally sent new users straight to the login screen with no explanation of what CedarCare was. Survey feedback and informal testing both made clear this was a problem. Building an introductory page within Streamlit was awkward because the framework is designed for data applications, not marketing layouts.

I worked around this by rendering a custom HTML/CSS page through `st.components.v1.html()`, bypassing Streamlit's widget system entirely for the landing view. The page includes a hero section with the tagline "Your health, guided by AI", a chatbot demo mockup showing CTAS Level 1 emergency detection, a statistics bar (95 automated tests, 14 CTAS emergency rules, 18 health articles), and trust indicators referencing CTAS, HealthLink BC, and GPT-4o-mini. The file is saved as `cedarcare_landing_v3_vch.html` and loaded at startup for unauthenticated users.

UI and branding redesign

Informal testing showed the original visual identity was too generic and inconsistent across pages. I renamed the platform from "BC Health Platform" to "CedarCare" and introduced a two-tone wordmark, a cedar tree logo, and a standardised dark teal sidebar (#006271) used on all six pages. Each page header received a custom SVG icon, I added a uniform footer throughout, and restyled the login button in teal to match.

Dashboard KPI redesign

The original KPIs, "Total checks since registration" and "Registrations this month", turned out to be irrelevant to individual health tracking. Informal testing confirmed this. I replaced them with "Most Recent Urgency Level" and "Days Since Last Assessment", which reflect what a user would actually want to see when returning to the platform.

Chatbot bug fixes

I identified and fixed three bugs during the testing phase. Severity chips were appearing after non-medical redirects even when no symptoms had been collected; I added a `_show_chips` session state flag to prevent this. Users who started with ambiguous input were losing their turn count, ending the multi-turn flow early; resetting `turn_count` at first real symptom extraction fixed that. Finally, the result card was not appearing after the last chip selection, which I resolved by setting `_show_chips` to `False` in the chip handler for terminal assessment statuses.

Conversational UI constraints

One respondent suggested a WhatsApp-style conversational layout. I looked into it but did not implement it. Streamlit's `st.chat_message()` renders within a scrollable page, forcing users to scroll to the input bar. This is a framework constraint, not something I can fix within Streamlit. I have logged it as a future enhancement that would require a React-based frontend.

Future improvement items logged

Open-ended responses to survey Q6 suggested appointment booking integration, French language support, push notifications for health reminders, and improved response speed through model caching. All four fall outside the current scope and have been logged for future development.

13. Reflections and Discussion

13.1 What Worked Well

Separating the pipeline into three layers, NLP extraction, CTAS safety rules, and ML classification, was the best architectural decision I made. Each layer could be developed and tested on its own. The CTAS rules provide hard safety guarantees for emergency conditions that a probabilistic model cannot match, while the Random Forest handles finer non-emergency distinctions that would be impractical to encode as rules.

The 95-test automated suite exceeded my initial expectations. I parametrized the CTAS tests so that each rule generates its own test case. This means adding a new rule automatically produces a corresponding test, which would scale well in production. The Hypothesis property-based fuzz tests also justified themselves early: they caught two edge cases during development that I had missed with example-based tests alone.

The algorithm benchmarking had value beyond the report. Running SVM, Random Forest, and KNN on the same dataset made the tradeoffs concrete. The SVM took 45 seconds to train and produced a 180MB model file, which would not be viable in a resource-constrained Streamlit deployment. That made the case for Random Forest straightforward.

13.2 Challenges Faced

Configuring the CI/CD pipeline was by far the most technically challenging part. The nested directory setup (tests/ at the repo root, source code at Implementation/src/) led to module resolution issues during three separate CI runs before the right working directory and import path combination was found.

The chatbot chip-display logic turned out to be more complicated than I thought. The way Streamlit's re-run lifecycle, session state, and the slot-filling turn counter interact gave rise to quite a few edge cases which could be fixed only after a thorough analysis of Streamlit's execution model.

13.3 Known Limitations

- Vague health expressions ('I feel bad', 'I'm not well') are sometimes classified as extractable symptoms by GPT-4o-mini, producing vague ML results. A symptom specificity validator would be a valuable future enhancement.
- The Kaggle training dataset uses clean, non-overlapping symptom-disease mappings. Real-world clinical data would contain significantly more noise and ambiguity, which is why the 70% confidence threshold and OpenAI fallback are critical safety mechanisms.
- SQLite is reset on Streamlit Cloud restart — seed_db.py handles the test account, but real user data would not persist across restarts. A PostgreSQL database (e.g. via Supabase) would be required for production.

13.4 Business Perspective

The competitive analysis found no other AI symptom checker that incorporates CTAS or targets the Canadian market. CedarCare's position rests on several specifics: it is the only consumer tool using CTAS as its clinical framework; its routing references BC facilities like UPCCs, MSP-covered virtual care,

and the 8-1-1 line, which generic platforms do not include; the longitudinal health profile gives users a reason to return rather than treating each session as disposable; and the system analytics dashboard could offer BC Health Authorities population-level data, opening a B2B pathway.

That said, I do not expect this gap to last. HealthLink BC may revamp its Healthwise platform, or a larger company may build BC-specific capability. I estimate a window of roughly 12 to 18 months. To make the B2B case credible within that window, the most important next step would be a clinical accuracy study comparing CedarCare's CTAS predictions against real triage nurse assessments.

Overall, this project showed me that building a working prototype and shipping a deployable product are different problems entirely. CedarCare grew from a single-turn chatbot into a multi-feature health navigation platform with a three-layer AI pipeline, dual ML architecture, automated testing suite, and live deployment. That expansion did not follow a straight line; each sprint introduced problems I had not anticipated, and I found that resolving them depended more on patience and methodical debugging than on any single technical decision. I also underestimated how much user feedback would redirect the work. Informal testing with housemates and the formal evaluation survey prompted changes to the dashboard KPIs, landing page layout, and chatbot chip flow that I would not have arrived at independently. Perhaps the harder lesson, though, was about intellectual honesty. I had to acknowledge where the system fell short, document production gaps without minimizing them, and defend architectural choices under direct questioning. That side of the work was as difficult as the engineering itself, and I suspect it is what separates a credible research project product from a technically functional one.

14. AI Use Section

14.1 Summary

AI tools were used for three purposes: (1) conceptual clarification and learning; (2) targeted code assistance — reviewed and approved before acceptance; (3) boilerplate content generation for elements such as article summaries. All architectural decisions, clinical design choices, algorithm selection, and test case design were made independently.

14.2 Table of AI Tools and Specific Use

AI Tool	Version / Account	Specific Use	Value Added by Student
Claude (claude.ai)	Sonnet 4.6 / Pro	Conceptual clarification: Streamlit session state, GitHub Actions YAML, pytest fixture design, K-Means StandardScaler preprocessing, CI/CD debugging	All architecture designed independently; AI explained patterns; student implemented and customised
Claude Code (VS Code)	Claude Code / Pro	Targeted surgical code edits: chatbot chip logic, CI/CD path fixes, specific function-level changes — each reviewed individually before acceptance	All changes reviewed before acceptance; many modified post-review; architecture never delegated
ChatGPT (OpenAI)	GPT-4o / Free	Prompt engineering research: best practices for forcing JSON output from GPT for symptom extraction	Wrote complete system prompt independently; iterated multiple versions; tested with diverse inputs
GitHub Copilot	—	Boilerplate autocompletion: import statements, docstring templates, test function scaffolding	All suggestions reviewed; clinical logic never accepted without independent validation

AI Tool	Version / Account	Specific Use	Value Added by Student
Gemini (Google)	Gemini for Workspace / Free	Generated initial Google Forms survey structure from a prompt	Reviewed and modified question wording; added clinical relevance to Q4
DALL-E / AI Image Gen	—	CedarCare cedar tree logo generation	Specified brand colours (VCH palette), style requirements; iteratively refined output

14.3 Statement of Originality

All system architecture, clinical design decisions, algorithm selection, feature engineering, and test case design were produced independently by the student. AI tools were used as research and implementation aids — not as primary authors of the platform's logic. The three-layer pipeline concept, the CTAS rule definitions, the 70% confidence threshold rationale, the K-Means feature selection, and the synthetic dataset design are original contributions of this research project.

15. Work Date/Hours Log

Student: Desmond Chua (Student ID: 300369803)

Note: Logs were entered at completion of each work session throughout the term. This log aggregates entries across all progress reports, mid-term report and final report. Study hours for learning new technologies are included as approved by the instructor.

Date	Hours	Description of Work Done
2026-01-09	2.00	Attended CSIS 4495 class; reviewed project requirements; brainstormed initial ideas based on Richmond Hospital experience
2026-01-09	1.50	Researched BC healthcare navigation tools; identified gap in existing solutions
2026-01-15	2.00	Drafted initial chatbot concept; researched CTAS triage guidelines
2026-01-16	1.00	Meeting #1 with instructor; received feedback to broaden scope; began expanding to multi-feature platform
2026-01-17	2.50	Researched OpenAI API documentation; explored integration options
2026-01-18	2.00	Meeting #2 with instructor (virtual); Option 1 approved; received guidance on focus areas
2026-01-19	3.00	Designed hybrid classification approach; researched Kaggle symptom datasets
2026-01-20	2.50	Created GitHub repository; set up project structure
2026-01-21	3.00	Researched ML analytics techniques for dashboards; planned Power BI integration
2026-01-22	2.50	Explored synthetic data generation approaches; documented data strategy
2026-01-23	3.00	Drafted proposal introduction and literature review sections
2026-01-24	3.50	Completed methodology section; created technical architecture diagram
2026-01-30	0.25	Install Python 3.13+ and verify installation
2026-01-30	0.25	Install Visual Studio Code with Claude Code extension
2026-01-30	0.25	Create GitHub repo (W26_4495_S2_DesmondC); set up folder structure; add instructor as collaborator
2026-01-30	0.25	Create OpenAI account; add \$5 API credit
2026-01-30	0.10	Create OpenAI API key (BC-Health-Platform)
2026-01-30	1.00	Attended CSIS 4495 Lesson 4 class; individual check-in with instructor; received feedback on project approach
2026-01-30	0.17	Created Streamlit Cloud account; connected with GitHub for deployment
2026-01-30	0.08	Created Python virtual environment (venv) for isolated project dependencies

Date	Hours	Description of Work Done
2026-01-30	0.25	Installed packages: streamlit, openai, pandas, scikit-learn, plotly, folium, python-dotenv, numpy, nltk, sqlalchemy, spacy
2026-01-30	0.08	Ran 'streamlit hello' to verify Streamlit installation; demo app loaded successfully at localhost:8501
2026-01-30	0.17	Created requirements.txt using pip freeze; removed spacy model reference for clean deployment
2026-01-31	0.42	Udemy Streamlit ML Deployment - Lesson 2: Installation and Setup (virtual environments, Python 3.8-3.12, streamlit hello)
2026-01-31	0.58	Udemy Streamlit ML Deployment - Lesson 3: Overview of Streamlit (data flow concept, caching preview, key features)
2026-01-31	0.50	Udemy Streamlit ML Deployment - Lesson 4: Creating Basic Streamlit App (main.py, st.text, auto-reload workflow)
2026-02-01	0.25	Udemy Streamlit ML Deployment - Lesson 5: Text Elements (st.title, st.header, st.markdown, st.code, st.latex)
2026-02-01	0.33	Udemy Streamlit ML Deployment - Lesson 6: Data Display Elements (st.dataframe, st.table, st.metric for KPIs)
2026-02-01	0.33	Udemy Streamlit ML Deployment - Lesson 7: Charting Elements (st.line_chart, st.bar_chart, st.area_chart, st.map)
2026-02-01	0.42	Udemy Streamlit ML Deployment - Lessons 8-9: Input Widgets (st.button, st.checkbox, st.radio, st.selectbox, st.slider)
2026-02-01	0.25	Udemy Streamlit ML Deployment - Lesson 10: Forms (st.form, form_submit_button, batch submissions)
2026-02-01	0.25	Udemy Streamlit ML Deployment - Lesson 11: Layout Elements (st.sidebar, st.columns, st.tabs, st.expander)
2026-02-01	0.67	Udemy Streamlit ML Deployment - Lessons 12-13: Capstone (Canada population trend dashboard with forms, tabs, metrics, matplotlib)
2026-02-01	0.33	Create Mockup 1a - Login page design using UX Pilot
2026-02-01	0.33	Create Mockup 1b - Register page with health profile using UX Pilot
2026-02-01	0.33	Create Mockup 2a - Home Dashboard design using UX Pilot
2026-02-01	0.33	Create Mockup 2b - Facility Finder page design using UX Pilot
2026-02-01	0.33	Create Mockup 2c - Health Library page design using UX Pilot
2026-02-01	0.33	Create Mockup 3 - Chatbot Interface design using UX Pilot
2026-02-01	0.33	Create Mockup 4 - My Dashboard (Personal) design using UX Pilot
2026-02-01	0.33	Create Mockup 5 - System Analytics page design using UX Pilot

Date	Hours	Description of Work Done
2026-02-01	0.17	Organize all mockup files in Misc/design/ folder
2026-02-01	0.25	Create folder structure (src/, pages/, components/, .streamlit/, data/)
2026-02-01	0.17	Create .streamlit/config.toml with VCH theme colors (teal #006271, blue #0078AE)
2026-02-01	0.25	Create empty placeholder files for all pages
2026-02-02	0.50	Create components/database.py - Initialize SQLite database module using Claude Code
2026-02-02	0.50	Create users table schema (id, name, email, password_hash, age, gender, region, health_conditions)
2026-02-02	0.50	Create assessments table schema (id, user_id, symptoms, extracted_symptoms, urgency_level, recommendation)
2026-02-02	0.75	Implement user CRUD functions (create_user, get_user_by_email, get_user_by_id, update_user, verify_user_exists)
2026-02-02	0.50	Implement assessment CRUD functions (save_assessment, get_user_assessments, get_assessment_by_id, get_assessment_stats)
2026-02-02	0.33	Test database with dummy data; Install SQLite Viewer VS Code extension; Verify tables
2026-02-02	0.10	Git commit and push: 'Sprint 1C complete: Database setup with users and assessments tables'
2026-02-04	0.75	Create components/auth.py (hash_password, verify_password functions using bcrypt)
2026-02-04	1.00	Create app.py with Login tab UI (email, password, remember me checkbox)
2026-02-04	1.50	Create app.py with Register tab UI (name, email, password, health profile, 10 health conditions)
2026-02-04	0.75	Implement login logic (verify credentials, set session_state)
2026-02-05	0.75	Implement register logic (validate, hash password, save to DB)
2026-02-05	0.25	Add logout functionality (clear session and redirect to login)
2026-02-05	0.50	Protect pages (redirect to login if not authenticated)
2026-02-05	0.50	Test login/register/logout flow (9 test cases: registration, duplicate email, password/email validation, wrong password, logout, page protection, database verification); Clean up orphaned test data
2026-02-05	0.50	Create pages/1_🏠_Home.py with welcome message
2026-02-05	0.17	Create pages/2_💬_Symptom_Assessment.py (placeholder)

Date	Hours	Description of Work Done
2026-02-05	0.17	Create pages/3_🇺🇸_My_Dashboard.py (placeholder)
2026-02-05	0.17	Create pages/4_🏥_Facility_Finder.py (placeholder)
2026-02-05	0.17	Create pages/5_📖_Health_Library.py (placeholder)
2026-02-05	0.17	Create pages/6_📊_System_Analytics.py (placeholder)
2026-02-05	0.25	Test navigation between all pages; Verify sidebar navigation works; Git commit and push
2026-02-06	0.17	Create .streamlit/secrets.toml with OpenAI API key; secure storage configuration
2026-02-06	0.08	Add secrets.toml to .gitignore to prevent accidental commit of API keys
2026-02-06	0.50	Create components/openai_utils.py; set up OpenAI API wrapper functions with error handling
2026-02-06	1.00	Create extract_symptoms() function using GPT-4o-mini; NLP to extract structured symptoms from natural language user input
2026-02-06	0.33	Test OpenAI API call with sample symptoms; verified extraction accuracy with test_openai.py
2026-02-07	0.50	Create components/rules.py; rule-based safety check module for emergency detection
2026-02-07	1.00	Define CRITICAL_SYMPTOMS list; 10-15 emergency patterns based on CTAS Level 1-2 guidelines (chest pain, difficulty breathing, etc.)
2026-02-07	0.75	Create check_critical_symptoms() function; returns True with 911 warning if emergency keywords detected
2026-02-07	0.50	Test with emergency scenarios using test_rules.py; verified all critical symptom patterns caught correctly
2026-02-08	2.00	Research prior ML algorithms used in healthcare triage/decision-making; documented findings in ML_Algorithm_Research.pdf
2026-02-08	0.50	Find and download Kaggle symptom-disease dataset (4920 rows, 41 diseases, 131 symptoms)
2026-02-08	0.75	Create notebooks/01_data_exploration.ipynb; explore dataset structure, class balance, symptom frequency, data quality issues
2026-02-08	1.00	Create data/urgency_mapping.csv; map 41 diseases to 4 CTAS-aligned urgency levels (Emergency/Urgent/Routine/Self-Care)
2026-02-09	1.00	Create components/train_model.py; 7-step pipeline (load, preprocess, one-hot encode, split, train, evaluate, save)
2026-02-09	1.00	Create notebooks/02_train_model.ipynb; document training process with visualizations (confusion matrix, feature importance, F1 scores)

Date	Hours	Description of Work Done
2026-02-09	2.00	Train Random Forest classifier (100 trees, balanced weights, severity-weighted features); achieved 100% accuracy on test set
2026-02-09	0.50	Evaluate model accuracy; classification report, confusion matrix heatmap; all 41 diseases F1=1.00; documented 70% confidence threshold
2026-02-09	0.25	Save model.pkl (6.82 MB) and model_metadata.json to Implementation/models/; verified with test_predict.py
2026-02-10	1.00	Create components/chatbot.py using Claude Code; 3-layer assessment pipeline (OpenAI extraction → CTAS safety rules → ML classification) with fallback logic
2026-02-10	1.50	Build chat UI using st.chat_message and st.chat_input; assistant/user message styling; chat container layout matching mockup design
2026-02-10	0.33	Add important disclaimer warning banner at top of Symptom Assessment page (not a substitute for medical advice, call 911 for emergencies)
2026-02-10	0.75	Implement chat history persistence using st.session_state; messages survive page interactions; chat replay on rerun; Clear Chat button in sidebar
2026-02-10	2.00	Integrate full 3-layer assessment pipeline: OpenAI GPT-4o-mini extracts symptoms → CTAS safety rules check for emergencies → ML Random Forest classifies disease with urgency mapping; 70% confidence threshold triggers OpenAI fallback
2026-02-11	1.00	Create color-coded urgency badges using HTML/CSS: Emergency (red), Urgent (orange), Routine (yellow), Self-Care (green); styled inline badges with rounded corners
2026-02-11	0.75	Add recommendation explanation text for each urgency level; Emergency shows 911/crisis line, Urgent shows 24hr care advice, Routine shows doctor visit guidance, Self-Care shows home remedies
2026-02-11	0.50	Add BC Health Resources section: HealthLink BC (Call 8-1-1, free 24/7 health advice) and Facility Finder page reference; displayed after every assessment result
2026-02-11	0.75	Add Save Assessment button that stores assessment to SQLite database (user_id, symptoms, extracted_symptoms, predicted_condition, confidence, urgency_level, recommendation); verified save with SQLite Viewer
2026-02-11	0.25	Test 7-8: Mental health crisis → Emergency via CTAS safety rules (CTAS Level 2, matched 'suicidal'); 'my stomach hurts' → OpenAI fallback → Routine; both saved to database
2026-02-11	0.25	Test 9a-9b: 'What's the weather today?' → correctly rejected as non-symptom input; 'stomach hurts + fever' follow-up → OpenAI fallback → Routine with escalation warnings

Date	Hours	Description of Work Done
2026-02-11	0.25	Test 10: 'runny nose and sneezing' → discovered bug: OpenAI fallback always defaulted urgency to Routine regardless of actual assessment; identified root cause in chatbot.py fallback logic
2026-02-11	0.50	Fix critical bug: OpenAI fallback urgency classification now asks OpenAI to classify urgency (Emergency/Urgent/Routine/Self-Care) and parses response instead of hardcoding 'Routine'; verified fix with re-tests
2026-02-11	0.33	Test 11: 'high fever, chills, sweating, vomiting, nausea, headache, muscle pain' → ML Model predicted Malaria (100% confidence) → Urgent; 7 symptoms matched ML feature names correctly
2026-02-11	0.25	Tests 12a-c: Acne (skin rash, pimples, blackheads → 100% ML confidence → Self-Care); Allergy (itching, sneezing, shivering → 78% → Self-Care); Fungal infection (itching, rash, patches → 100% → Self-Care)
2026-02-11	0.33	Add personalized greeting: chatbot welcomes user by name from session state ('Hello [name]! I'm here to help assess your symptoms...'); no Save button on greeting message
2026-02-11	0.50	Add Urgency Level Guide reference card below BC Health Resources; 2x2 grid layout matching mockup (Emergency—Call 911 Urgent—ED/Urgent Care Routine—Walk-in/Doctor Self-Care—Home care); displayed on all result types
2026-02-11	0.17	Test 13: 'severe chest pain can't breathe' → Emergency via CTAS rules; verified Urgency Level Guide now appears on Emergency results too; all 4 urgency levels confirmed working
2026-02-11	0.17	Git commit: 'Add personalized greeting, urgency level guide (2x2 grid), and fix OpenAI fallback urgency classification'
2026-02-12	0.50	Design app logo using Google Gemini AI: BC mountain peaks + maple leaf + chat bubble + health cross; selected from 4 generated variations; created white version for teal banner and amber/gold version as primary
2026-02-12	0.25	Logo processing: crop Gemini watermark using Paint, resize to 200px for web optimization, create amber/gold (#F2A900) background variant for teal banner contrast
2026-02-12	1.00	Create full-width teal (#006271) banner header across all pages using components/header.py; logo + 'BC Health Platform' on left, 'User: [name]' + Logout button on right; base64 logo embedding; position:fixed with z-index:999
2026-02-12	0.50	Fix CSS double-brace bug in header.py: <code>_BANNER_CSS</code> extracted from f-string had <code>{{ }}</code> escaping that broke all CSS rules; replaced with single braces; banner and sidebar offsets now render correctly

Date	Hours	Description of Work Done
2026-02-12	0.75	Sidebar styling overhaul: bold darker text (#1a1a1a, font-weight:600), active tab teal (#006271) background with white text, monochrome icons via CSS grayscale filter, hide 'app' label; applied via inject_global_css()
2026-02-12	0.50	Hide sidebar on login page (CSS display:none + initial_sidebar_state=collapsed); fix logout redirect using st.switch_page('app.py') instead of st.rerun(); auto-redirect unauthenticated users to login
2026-02-12	0.50	Fix sidebar not visible after login: CSS specificity issue where hide_sidebar CSS overrode show rules; added explicit display:block !important with visibility/width overrides in authenticated banner CSS
2026-02-12	0.33	Replace default icons with custom logo on login page and authenticated landing page using base64 embedding; rename Symptom Assessment page title to 'Symptom Assessment Chatbot'
2026-02-12	0.25	Git commits: 'Sprint 2D complete - Chatbot MVP with 3-layer assessment, banner, sidebar styling'; 'UI polish - custom logo, rename page, sidebar and banner fixes'; pushed to GitHub
2026-02-13	4.00	Mid-term Report Writing
2026-02-13	2.00	Mid-term Video Demonstration
2026-02-25	1.00	Post-cruise return: Reviewed Sprint 3 roadmap tracker, organized project files and mockups into consolidated project workspace
2026-02-25	0.50	Studied existing 1_Home.py placeholder code, compared against UI mockup, identified all sections needing rebuild
2026-02-25	1.00	Rebuilt Home page Welcome Header section with personalized greeting using session_state username, added health overview subtitle
2026-02-26	0.75	Created Quick Actions row (3 styled cards with icons: Check Symptoms, Find Facility, Health Info) with st.page_link() navigation
2026-02-26	1.00	Created My Health Summary card querying SQLite database via get_assessment_stats() - displays total assessments, last check relative time, most common symptom with percentage
2026-02-26	0.75	Created Recent Activity list showing 3 most recent assessments from database with color-coded urgency dots (green/yellow/red)
2026-02-27	0.50	Added BC Health Tip of the Day banner with rotating BC-specific health tips and Find Nearest Pharmacy navigation button
2026-02-27	0.50	Added most_common_symptom_count to get_assessment_stats() in database.py for percentage calculation (backward-compatible change)

Date	Hours	Description of Work Done
2026-02-27	0.50	Verified all dashboard data against SQLite database using Python queries - confirmed 33 assessments, Chest pain 33%, recent activity entries all match live data
2026-02-27	0.25	Git commit - 'Sprint 3A: Rebuild Home Dashboard to match UI mockup'
2026-02-28	0.50	Studied My Personal Health Dashboard mockup (2 pages), identified 5 sections: stat cards, trend chart, symptoms breakdown, assessment history table, date filter
2026-02-28	1.50	Built dashboard layout with 4 stat summary cards (Total Checks, This Month, Most Common, Avg Urgency) with colored icon boxes and status indicators
2026-03-01	1.50	Created Plotly filled area chart for Symptom Trends Over Time - grouped assessments by month, teal color scheme, responsive layout with <code>export_plotly_chart()</code>
2026-03-01	1.00	Created Plotly donut pie chart for Symptoms Breakdown - shows distribution of primary symptoms with percentages, rare symptoms grouped as Other
2026-03-01	1.00	Created Assessment History HTML table with styled colored badges (green Home Care, yellow Walk-in Clinic, orange Urgent Care, red Emergency)
2026-03-02	0.75	Implemented date filter dropdown (All Time, Last 6 Months, Last 3 Months, Last Year) that filters all dashboard sections simultaneously
2026-03-02	0.75	Fixed UI issues - Plotly X-axis tick intervals (<code>dtick=5</code>), donut chart label clipping (moved labels inside slices with legend below), button styling for Export Report and New Assessment
2026-03-02	0.50	Automated data verification against SQLite - confirmed all 4 stat cards, trend chart, donut chart percentages, and history table entries match real database values
2026-03-02	0.25	Git commit - 'Sprint 3B: Rebuild My Dashboard with Plotly charts, date filter, and live data'
2026-03-02	0.25	Added auto-redirect in <code>app.py</code> to route users to Home Dashboard after login instead of showing old placeholder page; Git commit
2026-03-02	0.75	Researched agentic AI frameworks in healthcare triage (reviewed Thamma 2025, Wang et al. 2024); Evaluated feasibility for project integration
2026-03-02	0.75	Updated work log, prepared Progress Report 2, updated task tracker with Sprint 3A and 3B completion status
2026-03-03	1.00	NLP Research: Reviewed CSIS 3400 NLP course materials (chatbot theory, dialog system pipeline); identified key concepts applicable to multi-turn chatbot upgrade

Date	Hours	Description of Work Done
2026-03-03	0.75	NLP Research: Studied chatbot taxonomy classification (FAQ bots vs. flow-based bots vs. AI bots); mapped existing single-turn chatbot against academic taxonomy to identify upgrade path
2026-03-04	1.00	NLP Research: Studied Dialog System Pipeline (NLU → Dialog Manager → NLG); identified how CTAS rules map to dialog management layer; documented slot-filling theory for symptom collection
2026-03-04	0.75	NLP Research: Researched Intent and Slot concepts in task-oriented dialog systems; mapped to healthcare context: Intent = assess symptoms, Slots = duration/severity/body location/additional info
2026-03-05	1.00	NLP Research: Studied stateful vs. stateless dialog systems and session_state management patterns; researched temperature parameter tuning (0.3 deterministic vs. 0.5 conversational) for GPT calls
2026-03-05	0.50	NLP Research: Documented all NLP concepts with academic framing; organized findings into technical notes mapping theoretical concepts to planned Sprint 3E implementation tasks
2026-03-06	0.50	NLP Research: Final review and consolidation of research notes; confirmed 7 sub-tasks for Sprint 3E aligned with dialog system theory
2026-03-07	1.00	Sprint 3B Enhancements: Added teal filled section headers to all dashboard sections (Symptom Trends, Symptoms Breakdown, Assessment History) matching platform visual standard
2026-03-07	0.75	Sprint 3B Enhancements: Redesigned KPI cards with teal top border strips and inline icons (Total Checks, This Month, Most Common, Avg Urgency); darkened subtitle text for readability
2026-03-08	0.75	Sprint 3B Enhancements: Added urgency filter slicer (All/Emergency/Urgent/Routine/Self-Care) applied additively with existing date filter; inserted 70 synthetic test records for realistic dashboard data
2026-03-08	0.50	Sprint 3B Enhancements: Added logo (assets/logo.png, 55px) beside page title; fixed navigation bug (st.query_params.clear() interfering with st.switch_page()); added View All toggle for assessment history
2026-03-08	0.50	Sprint 3B Enhancements: Fixed trend chart date grouping bug ('%B' to '%b %Y'); added linear trendline using numpy.polyfit as orange dashed trace; cleaned up chart legend
2026-03-08	1.50	Sprint 3E: Built multi-turn chatbot backend — initialize_dialog_state(), get_follow_up_question() slot-filling, process_multiturn_message() turn orchestrator in chatbot.py; MULTITURN_EXTRACTION_PROMPT and extract_symptoms_multiturn() in openai_utils.py

Date	Hours	Description of Work Done
2026-03-09	1.50	Sprint 3E: Built multi-turn UI in 2_Symptom_Assessment.py — chat history display, input handler, sidebar progress panel (collected symptoms, remaining turns), Start New Assessment reset button
2026-03-09	1.00	Sprint 3E: Ran all 6 test scenarios; identified edge case bug where vague health complaints triggered immediate redirect on turn 1; redesigned logic to apply gentle detail prompt on turn 1, redirect only after multiple failed turns
2026-03-09	0.50	Sprint 3E: Fix edge case — gentle prompt for vague health complaints on turn 1 instead of immediate redirect; confirmed fix with retest; git commit
2026-03-09	0.25	Sprint 3E: Git commits for backend, UI, and edge case fixes with descriptive messages
2026-03-09	1.00	Sprint 3C: Created Implementation/data/facilities.csv with 24 facilities across 5 BC regions (later rebuilt to 45 facilities with focused Lower Mainland coverage)
2026-03-09	1.50	Sprint 3C: Built 4_Facility_Finder.py — Emergency Contacts section, Search filter dropdowns, interactive Folium map with colored markers, facility cards, Health Tips, Understanding Facility Types sections
2026-03-09	0.75	Sprint 3C: Refined facility CSV to 45 records, removed Pharmacy type, separated Urgent Care (orange) from Walk-In Clinic (green) on map, verified all filter and map interactions
2026-03-09	0.25	Sprint 3C: Git commits; updated work log and task tracker with Sprint 3B, 3E, and 3C completion status
2026-03-09	0.75	Prepared Progress Report 3; updated commit history, work log entries, and Gantt chart for PR3 reporting period (Mar 3–9)
2026-03-11	1.00	Sprint 3D Research: Reviewed HealthLinkBC content categories and identified 8 topic areas for the Health Library (Pain & Injury, Cold & Flu, Medications, Prevention & Wellness, Mental Health, Heart Health, Child Health, Emergency Care)
2026-03-11	0.75	Sprint 3D Research: Designed JSON schema for health_articles.json — identified required fields: id, title, category, summary, content, tags, read_time, source_url, banner_color; mapped 18 article topics across 8 categories
2026-03-11	1.25	Sprint 3D: Drafted prompt for Claude Code to generate health_articles.json with 18 BC-specific health articles and Health Library page with full page structure matching mockup; reviewed and committed first draft
2026-03-11	1.00	Sprint 3D: Tested initial Health Library page — identified and documented UI bugs: duplicate category grid rendering, search section layout issues, incorrect section header styling; planned targeted fix approach

Date	Hours	Description of Work Done
2026-03-13	0.75	Sprint 3D Fix: Corrected Health Library UI — removed duplicate <code>st.button()</code> row above category icon cards, replaced teal <code>section_header()</code> on Browse by Category with plain <code>st.markdown()</code> , redesigned search card layout
2026-03-13	1.00	Sprint 3D Fix: Resolved ghost input box appearing between page subtitle and search card — root cause identified as orphaned HTML div wrappers auto-closing before Streamlit widgets rendered; replaced with CSS <code>:has()</code> marker injection pattern
2026-03-13	1.25	Sprint 3D: Implemented search logic — keyword filtering against article title, category, tags, and summary fields using <code>session_state</code> (<code>hl_search_query</code>); results grid with 3-column card layout, Clear Search button, no-results fallback message
2026-03-13	0.75	Sprint 3D: Wired Browse by Category Select buttons to filter articles — <code>hl_selected_category</code> session state, 3-column filtered results grid, mutual exclusivity with search filter; tested all 8 categories
2026-03-15	0.75	Sprint 3D: Verified Popular Topics section — 4 coloured banner cards (blue/green/purple/red), expandable full article content, Visit Source external links; confirmed all expanders and links functional
2026-03-15	0.50	Sprint 3D: Implemented UX improvement — wrapped All Articles section in collapsed <code>st.expander()</code> to reduce cognitive overload from 18 article cards; applied teal CSS <code>:has()</code> styling to match platform section header design system
2026-03-15	0.75	Sprint 3D: Standardized font weight across all teal section headers (700 weight, 16px, 0.01em letter-spacing); confirmed Official BC Health Resources section (HealthLinkBC, BCCDC, BC Health Services cards) rendering correctly
2026-03-15	0.50	Sprint 3D: Final review and testing of complete Health Library — search, category filter, popular topics, all articles expander, BC health resources, footer disclaimer; committed all Sprint 3D work to GitHub
2026-03-16	1.00	Sprint 4A Research: Designed synthetic data model for System Analytics — defined 3 patient engagement tiers (Healthy/Occasional, Moderate Users, High-Frequency/Chronic) with realistic assessment frequency and urgency distributions; drafted data generation requirements
2026-03-16	1.25	Sprint 4A: Developed <code>generate_synthetic_data.py</code> — 200 patients across 3 tiers, 5,000 assessment records with tier-weighted urgency distributions, flu season patterns (Nov–Feb spike), chronic symptom elevation for Tier 3; post-generation Emergency cap at $\leq 5\%$
2026-03-16	0.50	Sprint 4A: Validated <code>synthetic_assessments.csv</code> output — confirmed 5,000 records, Emergency at exactly 5.0% (250 records), age distributions match tier specifications, top symptoms reflect chronic condition elevation for Tier 3 patients; committed to GitHub

Date	Hours	Description of Work Done
2026-03-16	1.50	Sprint 4B: Built System Analytics Dashboard page — page header with Export Report placeholder and date range filter dropdown (All Time default), CSV data loading with os.path relative resolution, @st.cache_data, 4 KPI cards with teal headers (Total Users, Active This Week, Total Assessments, Avg Assessment Time)
2026-03-16	1.75	Sprint 4B: Added 4 Plotly charts — horizontal bar chart (top 10 symptoms with clinical severity colour coding: red=Serious/Chronic, amber=Infectious, blue=General), Urgency Distribution donut, Usage by BC Health Authority donut (5 health authority regions mapped from cities), Assessment Trends line chart with numpy.polyfit orange trendline
2026-03-16	1.50	Sprint 4B: Implemented K-Means unsupervised clustering — per-patient aggregation (age, assessment count, urgent ratio, chronic ratio), StandardScaler normalization, KMeans(n_clusters=3), automatic cluster naming by combined severity score; scatter plot visualization with bubble size = age; 3 cluster summary cards
2026-03-16	0.50	Sprint 4B: Final verification and testing — date range filter wiring confirmed across all KPI cards and charts, K-Means cluster separation validated (Chronic/Elderly avg age 73, High-Risk avg age 47, Healthy/Occasional avg age 29); committed all Sprint 4A and 4B work to GitHub
2026-03-16	0.75	Prepared Progress Report 4; updated commit history, work log entries, and timeline for PR4 reporting period (Mar 10–16); added screenshots and submitted to Blackboard
2026-03-17	1.00	Sprint 5A scope planning: Decided to drop Community Pulse (web scraping) feature and focus on automated testing suite. Drafted scope decision rationale and sent email to Prof. Priya explaining the scope change and requesting absence from class for job interview
2026-03-17	0.75	Sprint 5A.1: Designed tests/ folder architecture — unit/, integration/, ui/, property/ sub-packages. Created all init.py files, conftest.py with test_db (in-memory SQLite fixture with yield teardown) and test_user fixtures, TEST_REGISTER.md with table header
2026-03-17	0.50	Sprint 5A.1 cleanup: Identified and removed old ad-hoc test scripts (test_openai.py, test_predict.py, test_rules.py) from Implementation/tests/ — superseded by new proper pytest suite at root tests/ folder
2026-03-19	1.00	Sprint 5A.2: Read auth.py source code to understand exact function signatures (hash_password, verify_password, validate_password_strength, validate_email, require_authentication). Designed 10 test cases across 4 logical groups: Registration, Login, Security, Session
2026-03-19	1.25	Sprint 5A.2: Wrote test_auth.py — 10 unit tests for authentication module. Used conftest.py test_db fixture for all database operations, unittest.mock for Streamlit

Date	Hours	Description of Work Done
		session_state and st.switch_page. Verified all 10 tests pass: 10/10 passed in 4.38s
2026-03-19	0.50	Sprint 5A.3 analysis: Read rules.py to confirm check_critical_symptoms() signature, return dict structure, 14-rule count, match types (any vs all), case-insensitive matching behaviour before writing tests
2026-03-21	1.50	Sprint 5A.3: Wrote test_ctas_rules.py — 23 unit tests across 3 groups: 14 parametrized positive detection tests (one per CTAS rule), 5 negative pass-through tests, 4 edge case tests (mixed case, None input, large lists, single keyword). All 23 passing
2026-03-21	0.75	Sprint 5A.4 analysis: Read database.py (690 lines) — confirmed all CRUD function signatures, return types, table schema, and connection pattern (get_connection() called internally — requires unittest.mock.patch to redirect to test DB)
2026-03-21	1.25	Sprint 5A.4: Wrote test_database.py — 12 unit tests across 3 groups: 5 User CRUD tests, 5 Assessment CRUD tests, 2 Data Integrity tests. Patched get_connection() with test DB factory. Verified user isolation (TC-DB-11) and correct FK constraint handling. All 12 passing
2026-03-21	0.75	Sprint 5A.5 analysis: Confirmed ML layer lives in chatbot.py (not ml_model.py). Read model.pkl and model_metadata.json directly — identified feature vector building logic, 70% confidence threshold location, predict_proba() call pattern
2026-03-21	1.25	Sprint 5A.5: Wrote test_ml_model.py — 12 unit tests across 4 groups: model loading (2), confidence threshold (3), input handling (4), output validation (3). Loaded real model.pkl directly without Streamlit. UTI symptoms achieved 100% confidence. All 12 passing
2026-03-22	0.75	Sprint 5A.6 analysis: Read chatbot.py — confirmed run_assessment() layer sequence (OpenAI → CTAS → ML), all 4 return statuses and their dict keys, patch target (components.chatbot.extract_symptoms — patch where used not where defined)
2026-03-22	1.25	Sprint 5A.6: Wrote test_pipeline.py — 6 integration tests. Mocked both extract_symptoms() and get_openai_fallback_guidance(). Tested emergency circuit breaker, ML prediction path, OpenAI fallback path, safety invariants. All 6 passing in 3.74s
2026-03-23	0.75	Sprint 5A.7 analysis: Confirmed extract_symptoms() return dict key is 'symptoms' (not 'extracted_symptoms'). Confirmed all mock return structures for 6 clinical scenarios before writing tests
2026-03-23	1.25	Sprint 5A.7: Wrote test_chatbot_scenarios.py — 6 integration tests covering cardiac emergency, stroke emergency, UTI ML prediction, vague symptom

Date	Hours	Description of Work Done
		fallback, emergency override safety, minimal input edge case. All 6 passing on first run
2026-03-23	0.75	Sprint 5A.8 analysis: Read 3_My_Dashboard.py — confirmed session_state keys, data source (get_user_assessments via components.database), 4 stat cards as custom HTML st.markdown (not st.metric), 2 selectboxes, CWD dependency for assets/logo.png
2026-03-23	1.50	Sprint 5A.8: Wrote test_dashboard_apptest.py — 8 headless UI tests using Streamlit AppTest. Implemented _run_dashboard() helper with os.chdir(SRC_DIR) pattern. Verified stat card headers, selectbox options (All Time, Last 3 Months, All Urgency Levels), user data isolation. All 8 passing
2026-03-23	0.75	Sprint 5A.9 analysis: Read 2_Symptom_Assessment.py — confirmed session_state keys, process_multiturn_message() as primary function to mock (not run_assessment()), AppTest chat input syntax (at.chat_input[0].set_value().run()), _submit_chat() helper pattern needed
2026-03-23	1.50	Sprint 5A.9: Wrote test_chatbot_apptest.py — 8 headless UI tests. Implemented _run_chatbot() and _submit_chat() helpers with patch re-application on rerun. Tested warning banner, welcome message personalisation, ML response, emergency response, auth guard. All 8 passing
2026-03-23	0.75	Sprint 5A.10 analysis: Confirmed Hypothesis installed (6.151.9). Read auth.py and rules.py to verify exact validation rules — password requires letter + digit (not uppercase + lowercase separately). Corrected strategy before writing
2026-03-23	1.25	Sprint 5A.10: Wrote test_fuzz_hypothesis.py — 10 property-based tests using Hypothesis. 1,000 total fuzz inputs (100 per test). Covered password fuzzing (3 tests), email fuzzing (3 tests), CTAS rules fuzzing (3 tests), database fuzzing (1 test). All 10 passing
2026-03-23	0.50	Sprint 5A.11: Ran full test suite from project root — 79 tests across unit, integration, property folders all passing in 20.89s. UI tests confirmed passing individually (16 tests). Took screenshots for progress report documentation
2026-03-23	0.25	Git commits: All Sprint 5A tasks committed with granular messages (5A.1 through 5A.11) and pushed to GitHub main branch
2026-04-02	1.00	Sprint 5B: Created .github/workflows/ci.yml — GitHub Actions CI/CD pipeline to run 79-test suite on every push to main. Fixed PAT workflow scope permission error
2026-04-02	0.75	Sprint 5B: Debugged CI failures — requirements.txt path fix, pytest command not found (python -m pytest), streamlit.testing module conflict. Excluded UI AppTests from CI with --ignore=../tests/ui
2026-04-02	0.50	Sprint 5B: Committed model.pkl to repo (removed from .gitignore) to resolve CI model file not found errors. Final CI run: 79/79 tests passed in 13.72s

Date	Hours	Description of Work Done
2026-04-02	1.00	UI redesign: Implemented white topbar with CedarCare two-tone wordmark and cedar tree logo. Standardised page headers across all 6 pages with custom SVG icons per page
2026-04-02	0.75	UI redesign: Renamed BC Health Platform to CedarCare everywhere in code, page titles, sidebar, header component. Updated CSS for dark teal sidebar (#006271)
2026-04-02	0.75	UI redesign: Dashboard KPI fix — replaced Total checks since registration and Registrations this month with Most Recent Urgency Level and Days Since Last Assessment. Added consistent footer across all 6 pages
2026-04-02	0.50	UI redesign: Home page clean welcome header with SVG quick action icons for Check Symptoms, Find Facility, Health Library. Facility Finder emergency contacts redesigned as styled cards
2026-04-02	0.50	UI redesign: System Analytics trendline graceful fallback for missing data. Login button updated to teal filled to match CedarCare brand palette
2026-04-02	0.75	Sprint 5C prep: Created seed_db.py — auto-seeds test@test.com / Password123 on fresh deployment so Streamlit Cloud starts with a usable test account
2026-04-03	0.50	Sprint 5C: Deployed CedarCare to Streamlit Cloud at cedarcare.streamlit.app. Configured repo, branch (main), entry point (Implementation/src/app.py), Python 3.11, OPENAI_API_KEY secret
2026-04-03	1.00	Bug fix: Chips appearing on non-medical redirect. Implemented _show_chips session state flag — chips only shown when bot has genuinely collected symptoms. Added two-tier redirect logic: gentle redirect turn 1, hard end turn 2+
2026-04-03	0.75	Bug fix: Turn count reset on first symptom extraction — users who open with vague input get full 3-turn chip flow once real symptoms identified. Fixed chip handler to set _show_chips=True on follow_up with collected_symptoms
2026-04-03	0.50	Bug fix: Chip result rendering fix — result card not showing after final chip selection. Added _show_chips=False in chip handler for final assessment statuses. Removed redundant st.caption(hint) above chips
2026-04-03	0.75	Landing page: Built cedarcare_landing_v3_vch.html — custom HTML/CSS landing page via st.components.v1.html(). Hero section, chatbot mockup demo, statistics bar, trust indicators (CTAS-aligned triage, HealthLink BC, GPT-4o-mini powered)
2026-04-03	0.75	User testing: Set up CedarCare User Evaluation Survey on Google Forms (6 questions, Likert scales + open-ended). Distributed to 12 testers. Collected and captured all response charts for final report

Date	Hours	Description of Work Done
2026-04-03	0.50	End-to-end smoke test of live deployment — verified all 6 pages load, chatbot pipeline works, emergency detection fires, seed data present, and user testing form accessible
2026-04-04	2.00	Sprint 6: Final Report — Wrote Sections 1-4 (Introduction, Project Summary, Changes to Proposal, Project Completion Timeline). Generated Gantt chart PNG using matplotlib and Playwright
2026-04-04	2.00	Sprint 6: Final Report — Wrote Section 5 (Technical Implementation): Three-Layer Pipeline, CTAS rules, ML architecture, database design, dual ML architecture, OpenAI integration with system prompt engineering
2026-04-04	1.50	Sprint 6: Generated architecture diagram and tech stack diagram using Google Gemini image generation with detailed prompts. Generated algorithm benchmarking bar chart (RF vs SVM vs KNN) using matplotlib
2026-04-04	1.00	Sprint 6: Generated dark-theme Monokai code snippet PNGs for rules.py, chatbot.py (_ml_predict), database.py, auth.py, openai_utils.py, and 6___System_Analytics.py (K-Means pipeline) using Pygments ImageFormatter
2026-04-05	2.00	Sprint 6: Final Report — Wrote Sections 6-11 (Implemented Features 1-6): Secure Authentication, Symptom Assessment Chatbot, Personal Health Dashboard, BC Facility Finder, Health Education Library, System Analytics Dashboard
2026-04-05	1.00	Sprint 6: Captured all platform screenshots for report across all 6 features. Named and inserted all figures with captions. Documented chatbot flows including emergency, ML prediction, OpenAI fallback, and multi-turn chip flow
2026-04-05	1.00	Sprint 6: Final Report — Wrote Section 12 (Evaluation Techniques): pytest suite table (95 tests, 9 files), CI/CD pipeline section, user evaluation survey results (12 responses, Q1-Q5 analysis), implementation changes in response to feedback
2026-04-06	1.50	Sprint 6: Final Report — Wrote remaining sections: Competitive Analysis, Perceptual Map, Limitations and Constraints, Future Work, Conclusion, References, Appendix A (Installation Guide) and Appendix B (User Guide with screenshots)
2026-04-06	0.75	Sprint 6: Updated work log Excel with all Sprint 5B, 5C, and Sprint 6 entries (Apr 2-6). Compiled all potential demo Q&A pairs for presentation preparation (13 questions with model answers)
2026-04-06	0.50	Sprint 6: Final GitHub check-in — pushed final report PDF, presentation slides, updated README with installation instructions, and user guide PDF to ReportsAndDocuments folder on main branch
2026-04-06	0.25	Sprint 6: Submitted final report PDF on Blackboard. Confirmed all checklist items complete: GitHub check-in, slides, user guide, README, Blackboard submission

Date	Hours	Description of Work Done
TOTAL	185.35	

16. Concluding Remarks

I would like to thank Professor Padmapriya Arasanipalai Kandhadai for her guidance and feedback throughout this term. Her insistence that I justify every design decision, rather than default to the first workable option, shaped the project more than any single technical choice.

CedarCare applies natural language processing, supervised machine learning, and unsupervised clustering to a real healthcare access problem in British Columbia. The platform is deployed, tested, and user-validated at <https://cedarcare.streamlit.app>.

The three-layer pipeline, sequencing NLP extraction, deterministic CTAS rules, and a Random Forest classifier, is the project's main academic contribution. The CTAS rules catch emergencies that should never depend on a probabilistic model. The Random Forest handles non-emergency distinctions that rules alone cannot express. When the classifier's confidence falls below 70%, the system returns general guidance from the OpenAI API rather than presenting an uncertain prediction.

The benchmarking exercise had value beyond the report. All three algorithms (Random Forest, SVM, KNN) achieved comparable accuracy on this structured dataset. I selected Random Forest because it trained faster, produced a smaller model, and had stronger support in clinical classification literature. The exercise confirmed that algorithm selection in a deployed context depends on operational constraints as much as on accuracy.

The competitive analysis found no Canadian-built AI symptom checker that uses CTAS or provides BC-specific care routing. I estimate a window of 12 to 18 months before HealthLink BC or a larger entrant closes that gap. Whether CedarCare develops beyond a capstone depends on whether a clinical validation study can be completed in that time.

My background is in data analytics, not software engineering. That fact shaped several architectural decisions. I do not have experience with React, Node.js, or Django, and choosing one of those stacks would have meant spending most of my time on infrastructure rather than the data science work this course is actually assessed on. Streamlit was the right choice because it is Python-native. The Random Forest model, K-Means clustering, OpenAI API calls, SQLite database, and Plotly charts all run in one Python environment. A React frontend would have required a separate API layer, a different deployment pipeline, and considerably more development time.

Streamlit does have real constraints. The scrollable chat interface, the lack of mobile optimization, and the ephemeral SQLite database would all need to be addressed in a production system. I have documented these throughout the report. The upgrade path would be a React frontend backed by PostgreSQL and FastAPI.

This project started because I watched family members struggle to work out which part of BC's healthcare system to contact. It ended as a deployed platform with 95 automated tests, a CI/CD pipeline, and 12 users who gave it a 4.42/5 likelihood to recommend score. As someone trained in analytics rather than development, that outcome matters to me. What I took from this, about defending decisions, building for actual users, and being honest about what a system cannot do, is what I expect to carry forward after graduation.

17. References

- Azeez, D., Gan, K.B., Mohd Ali, M.A., & Ismail, M.S. (2015). Secondary triage classification using an ensemble random forest technique. *Technology and Health Care*, 23(4), 431–441.
- BC Ministry of Health. (2024). Emergency department wait times report 2023–2024. Province of British Columbia.
- Bullard, M.J., Unger, B., Spence, J., & Grafstein, E. (2008). Revisions to the Canadian Emergency Department Triage and Acuity Scale (CTAS) adult guidelines. *Canadian Journal of Emergency Medicine*, 10(2), 136–151.
- Esfandiari, N., Babavalian, M.R., Moghadam, A.M.E., & Tabar, V.K. (2021). Knowledge discovery in medicine: Current issue and future trend. *Expert Systems with Applications*, 41(9), 4434–4463.
- Fernandes, M., Mendes, R., Vieira, S.M., Leite, F., Palos, C., Johnson, A., Finkelstein, S., Moody, G., & Sousa, J.M.C. (2020). Risk of mortality and cardiopulmonary arrest in critical patients presenting to the emergency department using machine learning and natural language processing. *PLoS One*, 15(4), e0230876.
- Goto, T., Camargo, C.A., Faridi, M.K., Freishtat, R.J., & Hasegawa, K. (2019). Machine learning-based prediction of clinical outcomes for children during emergency department triage. *JAMA Network Open*, 2(1), e186937.
- Holcomb, J.B., et al. (2014). Pre-hospital triage of trauma patients using the random forest computer algorithm. *Journal of Trauma and Acute Care Surgery*, 76(2), 461–467.
- Kaggle. (n.d.). Disease Symptom Prediction Dataset.
<https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset>
- Marcolino, M.S., Queiroz Oliveira, J.A., D'Agostino, M., Ribeiro, A.L., Alkmim, M.B.M., & Novillo-Ortiz, D. (2018). The impact of mHealth interventions: Systematic review of systematic reviews. *JMIR mHealth and uHealth*, 6(1), e23.
- Nori, H., King, N., McKinney, S.M., Carignan, D., & Horvitz, E. (2023). Capabilities of GPT-4 on medical challenge problems. *arXiv preprint arXiv:2303.13375*.
- Vrbaski, M., Bhatt, P., Gao, F., & Trent, R. (2019). Patient clustering for clinical decision support: A review. *International Journal of Medical Informatics*, 130, 103945.

Appendix A: Installation Guide

A.1 Option 1: Access Live Deployment (Recommended)

Live URL: <https://cedarcare.streamlit.app>

Test account — Email: test@test.com | Password: Password123. No installation required.

A.2 Option 2: Run Locally

Prerequisites: Python 3.11, Git, OpenAI API key.

Step 1 — Clone the repository:

```
git clone https://github.com/desporamon/W26_4495_S2_DesmondC.git
cd BC-Health-Platform
```

Step 2 — Create and activate a virtual environment:

```
python3.11 -m venv venv
venv\Scripts\activate          # Windows
source venv/bin/activate       # macOS/Linux
```

Step 3 — Install dependencies:

```
pip install -r requirements.txt
```

Step 4 — Configure OpenAI API key. Create Implementation/src/.streamlit/secrets.toml:

```
OPENAI_API_KEY = "sk-your-key-here"
```

Step 5 — Seed the database (creates test account for first-time setup):

```
cd Implementation/src
python seed_db.py
```

Step 6 — Run the application:

```
streamlit run app.py
```

Step 7 — Open browser: <http://localhost:8501>. Register a new account or use test@test.com / Password123.

A.3 Running the Test Suite

Navigate to the project root directory and run one of the following commands:

Full suite — 95 tests (unit, integration, UI, property-based):

```
cd BC-Health-Platform
pytest tests/ -v --tb=short
```

CI subset — 79 tests (excludes UI AppTests, matches GitHub Actions pipeline):

```
cd BC-Health-Platform
pytest tests/ -v --ignore=tests/ui
```

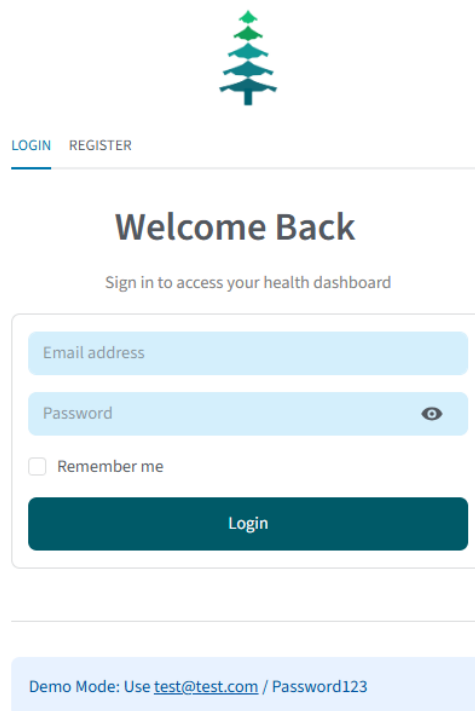
Note: UI AppTests (test_dashboard_apptest.py and test_chatbot_apptest.py) must be run separately from the Implementation/src/ working directory due to relative asset path dependencies.

Appendix B: User Guide

This step-by-step guide shows how to use each feature of CedarCare. Screenshots are from the live application at <https://cedarcare.streamlit.app>.

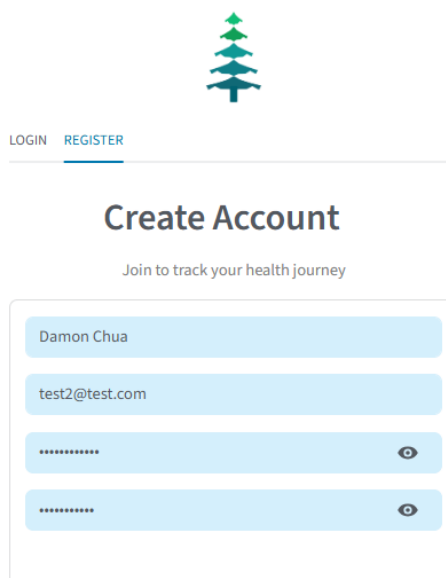
B.1 Registration and Login

Step 1: Navigate to <https://cedarcare.streamlit.app>



The screenshot shows the CedarCare login interface. At the top center is a green cedar tree icon. Below it are two links: "LOGIN" (underlined) and "REGISTER". The main heading is "Welcome Back" in a large, bold font. Below the heading is the subtext "Sign in to access your health dashboard". The login form consists of a light blue rounded rectangle containing: an "Email address" input field, a "Password" input field with a toggle eye icon, a checkbox labeled "Remember me", and a dark teal "Login" button. At the bottom of the form is a light blue box with the text "Demo Mode: Use [test@test.com](#) / Password123".

Step 2: Click Register to create a new account. Enter your name, email, and a password (min 8 characters, uppercase, lowercase, digit required).



The screenshot shows the CedarCare registration interface. At the top center is a green cedar tree icon. Below it are two links: "LOGIN" and "REGISTER" (underlined). The main heading is "Create Account" in a large, bold font. Below the heading is the subtext "Join to track your health journey". The registration form consists of a light blue rounded rectangle containing: a text input field with "Damon Chua", an email input field with "test2@test.com", and two password input fields, each with a toggle eye icon. The form is followed by a horizontal line.

Health Profile (Recommended)

This helps us provide personalized health recommendations

Age

55

Gender

Male

BC Health Authority

Fraser Health

Existing Conditions:

Select all that apply

☒ Diabetes

☐ Heart Disease

☒ High Blood Pressure

☐ Asthma

☐ Stroke History

☐ Pregnancy

☐ Blood Thinners

☐ Weakened Immune System

☐ Chronic Pain

☐ None of the above

This helps us provide personalized health recommendations

Create Account

Step 3: Or use the pre-seeded test account: Email: test@test.com | Password: Password123

Step 4: On login, you are taken to the Home page showing your personalized health overview.

CedarCare

Home

Symptom Assessment

My Dashboard

Facility Finder

Health Library

System Analytics

User: Test User Logout

Home

Your personal health overview at a glance

Welcome back, Test User!

Here's your health overview for today

Check Symptoms

AI-powered health triage

Get Started →

Find Facility

Hospitals and clinics near you

Search Now →

Health Library

Evidence-based health info

Explore →

My Health Summary

Total Assessments: 119

Last Check: -1 days ago

Most Common: Chest pain (19%)

View Full Dashboard →

Recent Activity

Apr 5: Chest pain

Recommendation: Emergency

Apr 4: Pain

Recommendation: Emergency

Apr 4: Chest pain

Recommendation: Emergency

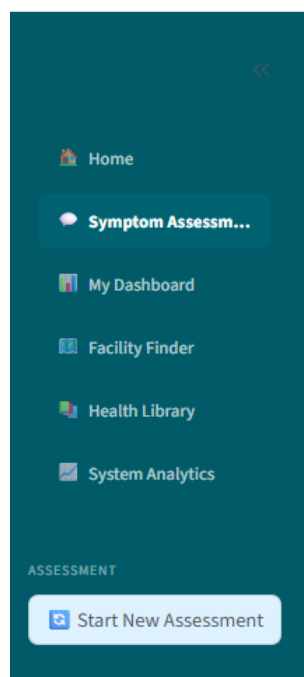
BC HEALTH TIP OF THE DAY

Regular hand washing is the most effective way to prevent the spread of illness.

Find Nearest Pharmacy →

B.2 Symptom Assessment Chatbot

Step 1: Click Symptom Assessment in the left sidebar.



Symptom Assessment Chatbot

Describe your symptoms to begin your AI-powered triage

Important disclaimer: This tool provides general health information only. It is NOT a substitute for professional medical advice. If you are experiencing a medical emergency, please call **911** immediately.



Hello Test User! I'm here to help assess your symptoms. What symptoms are you experiencing today?

Step 2: Describe your symptoms naturally in plain English. Example: 'I have a bad headache and feel nauseous.'



Symptom Assessment Chatbot

Describe your symptoms to begin your AI-powered triage

Important disclaimer: This tool provides general health information only. It is NOT a substitute for professional medical advice. If you are experiencing a medical emergency, please call **911** immediately.



Hello Test User! I'm here to help assess your symptoms. What symptoms are you experiencing today?



I have a bad headache and feel nauseous.



I'm sorry to hear that you're dealing with a bad headache and nausea; that sounds really uncomfortable. On a scale from one to ten, how severe would you say your symptoms are right now?

 Mild

 Moderate

 Severe

 Unbearable

Step 3: Select a severity chip (Mild/Moderate/Severe/Unbearable) or type your own answer.

Step 4: Answer the duration question using chips (Under 1 hour/Few hours/1–2 days/3+ days) or free text.



Symptom Assessment Chatbot

Describe your symptoms to begin your AI-powered triage

Important disclaimer: This tool provides general health information only. It is NOT a substitute for professional medical advice. If you are experiencing a medical emergency, please call **911** immediately.



Hello Test User! I'm here to help assess your symptoms. What symptoms are you experiencing today?



I have a bad headache and feel nauseous.



I'm sorry to hear that you're dealing with a bad headache and nausea; that sounds really uncomfortable. On a scale from one to ten, how severe would you say your symptoms are right now?



👎 Severe



I can imagine how tough that must be for you. How long have you been experiencing these symptoms?

Under 1 hour

Few hours

1-2 days

3+ days

Step 5: If your symptoms are pain-related, you will be asked for the body location. Select from chips or type.

Step 6: The chatbot produces your assessment result showing extracted symptoms, urgency level, and BC Health Resources.

 I can imagine how tough that must be for you. How long have you been experiencing these symptoms?

 Few hours

 **Extracted Symptoms:** headache, nausea

Our model couldn't make a confident prediction. Here's general guidance:

Severe headache and nausea can indicate a more serious condition. It's important to stay hydrated and rest in a dark, quiet room. Over-the-counter pain relievers may help alleviate the headache. However, you should seek medical attention within 24 hours to rule out any serious issues. This is not medical advice.

Urgency:  **Urgent**

 **BC Health Resources**

- **HealthLink BC:** Call 8-1-1 (free health advice, 24/7)
- Find a walk-in clinic near you on the **Facility Finder** page

Urgency Level Guide

- | | |
|---|--|
|  Emergency (Call 911) |  Urgent (ED / Urgent Care) |
|  Routine (Walk-in / Doctor) |  Self-Care (Home care) |

 **Save Assessment**

✓ Your assessment is complete. Use  **Start New Assessment** in the sidebar to begin a new conversation.

Step 7: Click Save Assessment to store the result in your personal health history.

IMPORTANT: If the chatbot detects emergency symptoms, you will see a red emergency alert immediately. Call 911 — do not delay.

  **EMERGENCY DETECTED — Difficulty Breathing**

 **Go to the nearest Emergency Room or CALL 911**

Difficulty breathing on its own can indicate asthma attack, allergic reaction, pneumonia, or other serious conditions requiring urgent medical evaluation.

CTAS Level: 2 | Matched symptoms: difficulty breathing

Urgency:  **Emergency**

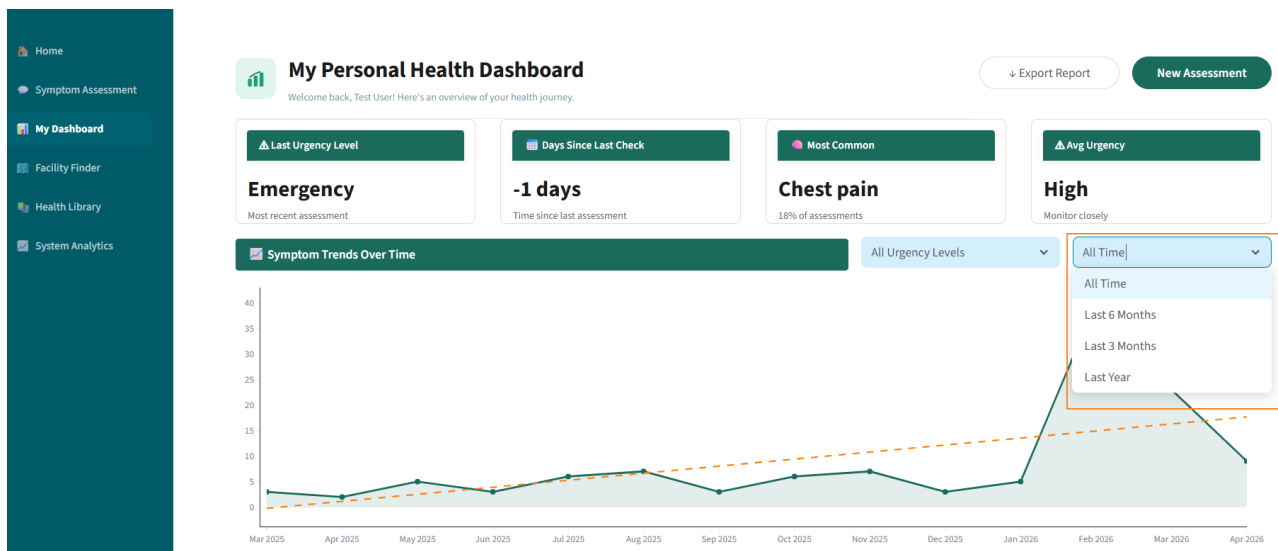
Urgency Level Guide

- | | |
|---|--|
|  Emergency (Call 911) |  Urgent (ED / Urgent Care) |
|  Routine (Walk-in / Doctor) |  Self-Care (Home care) |

✓ Your assessment is complete. Use  **Start New Assessment** in the sidebar to begin a new conversation.

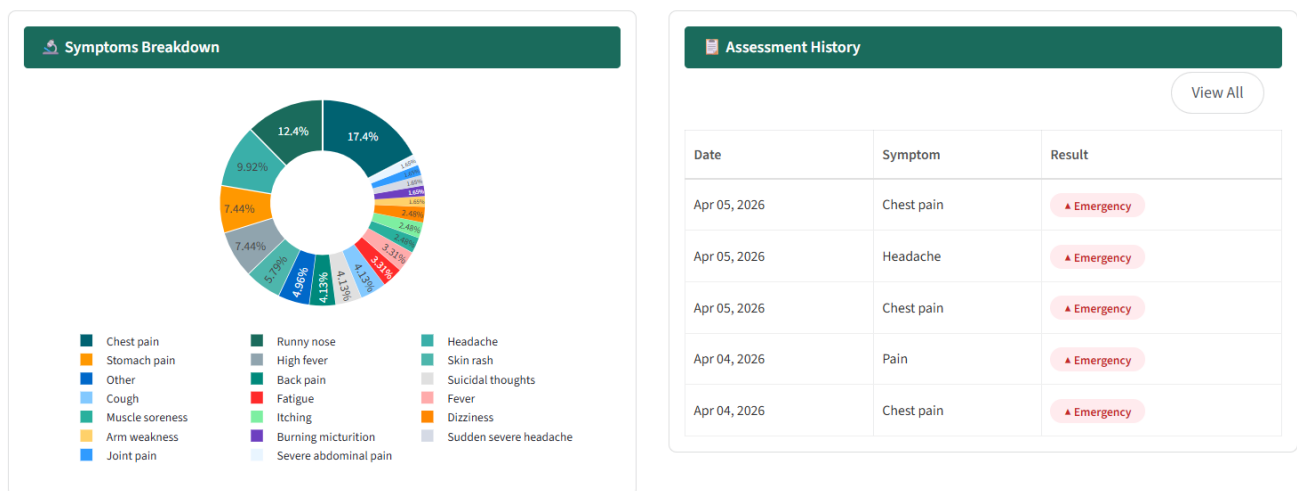
B.3 My Dashboard

Step 1: Click My Dashboard in the sidebar.



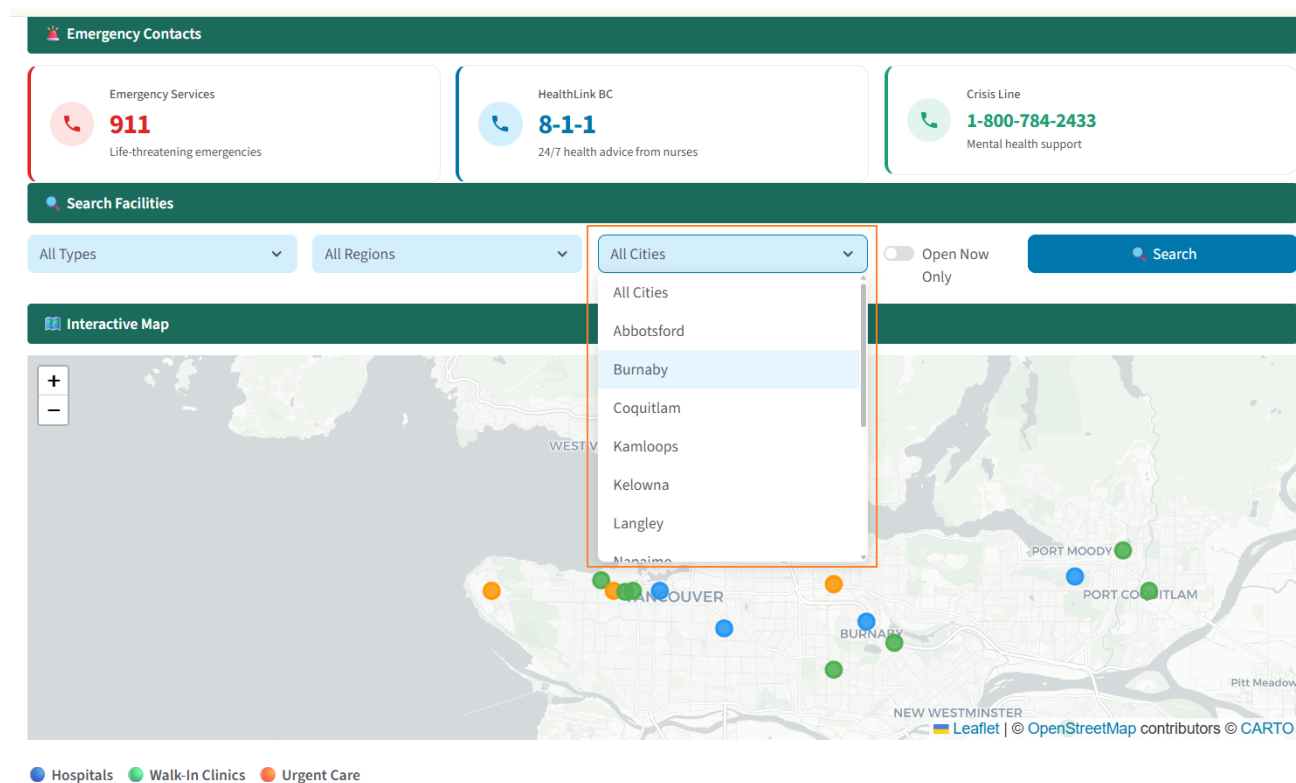
Step 2: Use the date range picker and urgency filter dropdown to explore your history.

Step 3: Review your assessment history table and trend charts.

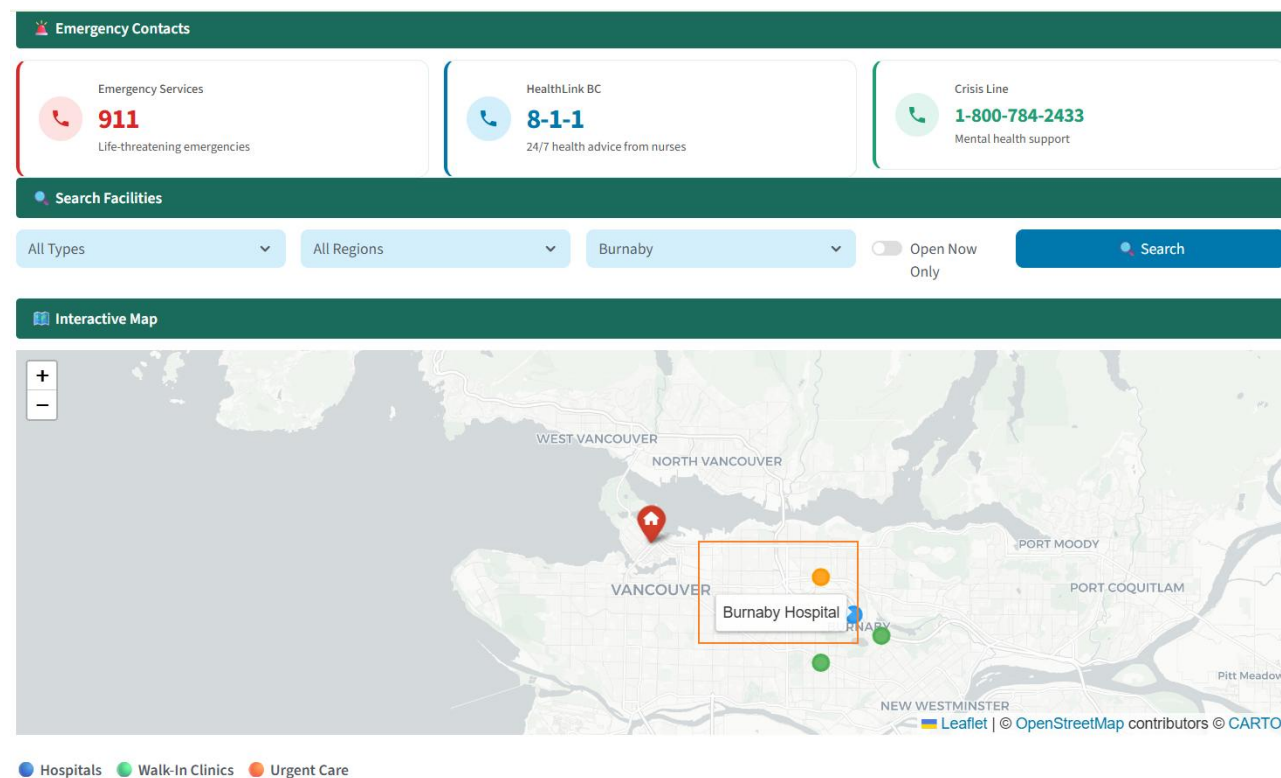


B.4 Facility Finder

Step 1: Click Facility Finder in the sidebar.



Step 2: Click any marker to see facility details (name, type, address, phone).



Step 3: Use the emergency contact cards on the above for immediate assistance numbers.

B.5 Health Library

Step 1: Click Health Library in the sidebar.

Home

Symptom Assess...

My Dashboard

Facility Finder

Health Library

System Analytics

Health Education Library

Evidence-based health information from HealthLink BC

Find Health Information

Search health topics... (e.g., headache, fever, back pain)

Search

Popular searches:

Headache

Fever

Back Pain

Diabetes

High Blood Pressure

Browse by Category

Explore health topics organized by condition and specialty

Pain & Injury

Information on managing pain and treating injuries

Select

Cold & Flu

Symptoms, treatment, and prevention tips

Select

Medications

Drug information and safe use guidelines

Select

Prevention & Wellness

Healthy living and disease prevention

Select

Mental Health

Support for mental wellness and conditions

Select

Heart Health

Cardiovascular health and heart disease

Select

Child Health

Pediatric health and parenting resources

Select

Emergency Care

When to seek emergency medical help

Select

Step 2: Click a category card to filter articles, use the search bar, or browse Popular Topics.

Step 3: Click Read More on any article. Click Visit Source to verify information on HealthLinkBC.

Browse by Category

Explore health topics organized by condition and specialty

Pain & Injury

Information on managing pain and treating injuries

Select

Cold & Flu

Symptoms, treatment, and prevention tips

Select

Medications

Drug information and safe use guidelines

Select

Prevention & Wellness

Healthy living and disease prevention

Select

Mental Health

Support for mental wellness and conditions

Select

Heart Health

Cardiovascular health and heart disease

✓ Selected

Child Health

Pediatric health and parenting resources

Select

Emergency Care

When to seek emergency medical help

Select

Articles in 'Heart Health' (2 articles found)

Clear Filter

Heart Health

Heart Health Essentials

Tips for cardiovascular wellness including diet, exercise, and understanding heart disease risk factors

4 min read

> Read More →

Heart Health

High Blood Pressure: The Silent Killer

Understanding hypertension, its risks, and how to manage blood pressure through lifestyle and medication

4 min read

> Read More →

Understanding your personal risk and making healthy lifestyle choices can significantly reduce your chances of developing cardiovascular problems.

Key modifiable risk factors include high blood pressure, high cholesterol, smoking, physical inactivity, obesity, and diabetes. Have your blood pressure and cholesterol checked regularly by your family doctor. In BC, adults over 40 should have a cardiovascular risk assessment every three years.

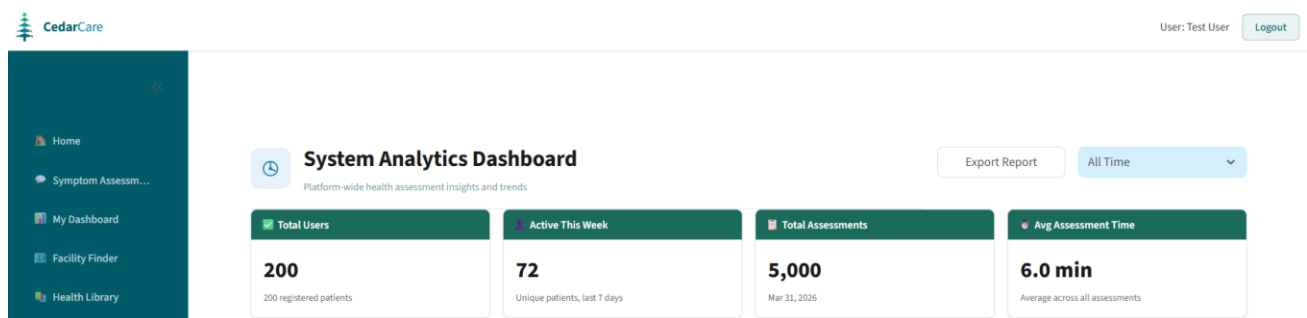
A heart-healthy diet emphasizes fruits, vegetables, whole grains, lean proteins, and healthy fats from sources like fish, nuts, and olive oil. Limit your intake of sodium (aim for less than 2,300 mg per day), processed foods, and sugary beverages. The Mediterranean diet has strong evidence supporting heart health benefits.

Aim for at least 150 minutes of moderate-intensity aerobic activity per week — that is about 30 minutes, five days a week. Activities like brisk walking, cycling, and swimming are excellent choices. Know the warning signs of a heart attack: chest discomfort, pain spreading to the arm or jaw, shortness of breath, cold sweats, and nausea. If you experience these symptoms, call 9-1-1 immediately.

[Visit Source →](#)

B.6 System Analytics

Step 1: Click System Analytics in the sidebar.

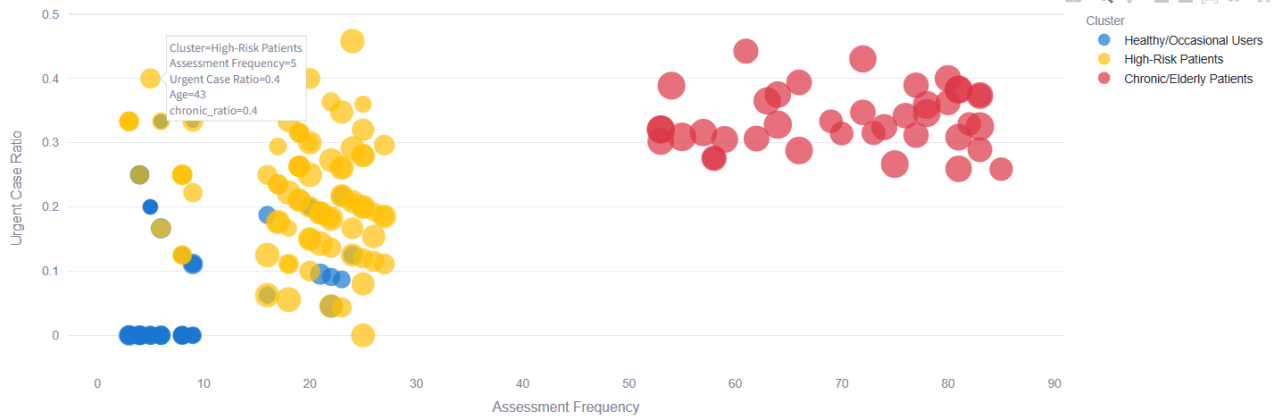


Step 2: Scroll down to the K-Means Patient Cluster Analysis section. The scatter plot shows three patient risk segments.

Patient Cluster Analysis (K-Means)

Unsupervised K-Means clustering automatically groups patients by age and assessment patterns to identify natural health behaviour segments.

Patient Segments by Assessment Frequency and Urgency Profile



Chronic/Elderly Patients

40 patients
Avg age: 73
Avg assessments: 70.5
Urgent ratio: 33.6%

High-Risk Patients

86 patients
Avg age: 47
Avg assessments: 18.8
Urgent ratio: 22.3%

Healthy/Occasional Users

74 patients
Avg age: 29
Avg assessments: 7.6
Urgent ratio: 7.4%

Appendix C: User Evaluation Survey Questionnaire

Survey link: <https://forms.gle/hyE1jGp6wcqjowgGA>

CedarCare User Evaluation Survey

Help us improve CedarCare! Please share your experience using our AI-powered health navigation platform for BC residents. This survey takes approximately 2 minutes to complete.

12 responses collected — April 3, 2026. Participants accessed <https://cedarcare.streamlit.app> prior to completing the survey.

Q1 How easy was it to navigate and use the CedarCare platform overall?

Required · Single choice (1 = Extremely Difficult → 5 = Extremely Easy)

- ☐ 1 — Extremely difficult
- ☐ 2 — Somewhat difficult
- ☐ 3 — Neutral
- ☐ 4 — Somewhat easy
- ☐ 5 — Extremely easy

Q2 Please rate how helpful and accurate the Symptom Assessment chatbot was in guiding your next steps.

Required · Single choice (1 = Not helpful at all → 5 = Very helpful)

- ☐ 1 — Not helpful at all
- ☐ 2 — Somewhat unhelpful
- ☐ 3 — Neither helpful nor unhelpful
- ☐ 4 — Somewhat helpful
- ☐ 5 — Very helpful

Q3 Which feature(s) did you find most useful during your last visit to CedarCare?

Required · Multiple choice (Select all that apply)

- | | |
|--------------------------|---|
| ☐ | Symptom Assessment Chatbot |
| ☐ | My Dashboard (Personalised Health Summary) |
| ☐ | Facility Finder (Locating clinics, hospitals, etc.) |
| ☐ | Health Library (Information and resources) |
| ☐ | System Analytics (Performance data/reports) |

Q4 Did the urgency level recommendation provided by CedarCare seem appropriate for the symptoms you entered?

Required · Single choice (1 = Not appropriate → 5 = Very appropriate)

- | | |
|-----------------------|----------------------------|
| ☐ | 1 — Not appropriate at all |
| ☐ | 2 — Somewhat inappropriate |
| ☐ | 3 — Neutral |
| ☐ | 4 — Somewhat appropriate |
| ☐ | 5 — Very appropriate |

Q5 How likely are you to recommend CedarCare to other residents of British Columbia?

Required · Single choice (1 = Very unlikely → 5 = Very likely)

- | | |
|-----------------------|---------------------------------|
| ☐ | 1 — Very unlikely |
| ☐ | 2 — Somewhat unlikely |
| ☐ | 3 — Neither likely nor unlikely |
| ☐ | 4 — Somewhat likely |
| ☐ | 5 — Very likely |

Q6 What specific improvements or suggestions do you have for CedarCare to enhance its service or features?

Required · Open-ended response

Enter your answer here...

Appendix D: AI Prompt History — Complete Log

This appendix documents all AI tool interactions across the full project lifecycle — Proposal, Progress Reports 1–6, and Midterm Report. 51 sessions in chronological order, fully deduplicated. All architectural decisions, implementation choices, and testing strategies were independently developed by the student.

Live deployment: <https://cedarcare.streamlit.app> | GitHub: https://github.com/desporamon/W26_4495_S2_DesmondC

Session 1: Clarifying Hybrid Approach

January 19, 2026 | Tool: Claude (Anthropic)

Context:

After researching CTAS guidelines, I had the idea to use rule-based logic for critical symptoms and wanted to verify this was a sound approach.

Prompt:

'I'm thinking of using a hybrid approach for my symptom classification — rule-based for critical symptoms like chest pain, and ML for less obvious cases. Is this a valid approach? I got this idea from reading about how CTAS works.'

Summary of Response:

AI confirmed this is a valid approach used in real clinical decision support systems, noting the safety benefit of not relying on ML for critical symptoms and the flexibility of ML for ambiguous cases.

Value I Added:

I had already decided on this approach based on my own CTAS research. The AI response validated my thinking. I then independently designed the specific 14 rules based on CTAS Level 1–2 criteria.

Session 2: Understanding OpenAI API Options

January 17, 2026 | Tool: ChatGPT

Context:

I knew I wanted to use OpenAI for NLP but was not sure about the technical implementation options.

Prompt:

'For my healthcare chatbot, I want to use OpenAI to understand user symptoms. Can you explain how the API integration would work? I've read the OpenAI documentation but want to make sure I understand it correctly.'

Summary of Response:

AI explained how OpenAI can extract structured data from unstructured text, handling variations in how users describe symptoms. It showed a basic example of API call structure.

Value I Added:

I used this understanding to design the integration flow: OpenAI extracts symptoms → rule-based layer checks for critical patterns → ML model classifies urgency. All code logic and prompt engineering was my own work.

Session 3: ML Analytics Techniques for Health Dashboard

January 21, 2026 | Tool: Claude (Anthropic)

Context:

Based on my Power BI experience at VCH, I wanted analytics beyond basic counts and needed to identify which ML techniques were appropriate for health data.

Prompt:

'For my health dashboard, I want to show insights beyond just basic counts. What ML techniques could I use to analyze symptom patterns across users? I'm familiar with clustering from my ML course.'

Summary of Response:

AI mentioned K-means clustering for user grouping, anomaly detection for unusual spikes, time series for trends, and association rules for symptom co-occurrence.

Value I Added:

I selected specific techniques based on my existing knowledge: association rules (learned in CSIS 3275), clustering (ML course), and time series (comfortable from VCH work). I decided which techniques to apply to Feature 2 vs Feature 5 based on individual vs population scope.

Session 4: Data Source Validation for ML Training

January 19, 2026 | Tool: Claude (Anthropic)

Context:

My instructor asked about data sources for training the ML model and I wanted to verify my plan to use public datasets.

Prompt:

'My instructor asked where I would get training data to classify symptom severity. I'm planning to use Kaggle datasets. Is this a reasonable approach for an academic project?'

Summary of Response:

AI confirmed Kaggle symptom-disease datasets are commonly used for academic ML projects and noted that I would need to manually map diseases to urgency levels since datasets typically map symptoms to diseases, not triage urgency.

Value I Added:

I independently researched and identified specific Kaggle datasets. I created my own urgency mapping based on CTAS guidelines (e.g., heart attack symptoms → Emergency, common cold → Self-care).

Session 5: Synthetic Data for Demo Dashboards

January 22, 2026 | Tool: Claude (Anthropic)

Context:

I realised my dashboards would be empty at launch with no user data and asked about standard practice for proof-of-concept applications.

Prompt:

'For my demo, the dashboards will be empty since there's no user data yet. What do developers typically do in this situation for proof-of-concept applications?'

Summary of Response:

AI explained that synthetic seed data is standard practice for demos, mentioning generating realistic fake records that enable ML analytics to show meaningful patterns.

Value I Added:

I decided to generate 5,000 synthetic records. I designed the generation script myself, including realistic symptom patterns and a seasonal flu outbreak trend so the ML analytics would have meaningful signals to detect.

Session 6: Document Formatting for Proposal

January 25, 2026 | Tool: Claude (Anthropic)

Context:

I needed help formatting my proposal document with proper tables and structure. All content was already written by me.

Prompt:

'Can you help me format my proposal into a Word document with proper tables? I have all the content written, I just need help with the formatting.'

Summary of Response:

AI assisted with table formatting and document structure in Word.

Value I Added:

All proposal content — the research, architecture decisions, feature specifications, and justifications — was written entirely by me. AI was used only for formatting assistance.

Session 7: System Flow Diagram Generation

January 25, 2026 | Tool: Google Gemini

Context:

I needed a professional system flow diagram for the proposal showing the 3-layer pipeline visually.

Prompt:

'Create a professional software architecture flowchart titled 'BC Personal Health Management Platform — System Flow' showing: User Input → OpenAI NLP extraction → Rule-Based Safety Check (CTAS) → ML Classifier. Include an Emergency branch at the CTAS layer. Use a clean professional style with teal colour scheme.'

Summary of Response:

Gemini generated a professional diagram based on my detailed specifications.

Value I Added:

I created all diagram content, component definitions, data flow directions, and visual specifications myself. Gemini only rendered the visual based on my detailed prompt.

Session 8: Architecture Overview Diagram Generation

January 25, 2026 | Tool: Google Gemini

Context:

I needed a full architecture overview diagram showing all platform components and their relationships for the proposal.

Prompt:

'Create a professional architecture overview diagram showing the complete tech stack: Streamlit frontend, Python backend, SQLite database, OpenAI API, ML Engine (scikit-learn), and external data sources (Kaggle, HealthLinkBC, CTAS).'

Summary of Response:

Gemini generated a professional architecture overview diagram.

Value I Added:

I designed the complete architecture before this session. Gemini only rendered the visual. Every component, connection, and data flow direction was specified by me based on my own technical design.

Session 9: OpenAI API Authentication and Key Management

February 6, 2026 | Tool: Claude (Anthropic)

Context:

Starting Sprint 2A, I needed to understand how to securely integrate the OpenAI API into a Streamlit application without exposing API keys in source code or GitHub.

Prompt:

'I need to use OpenAI's API in my Streamlit app for symptom extraction. What is the safest way to store and access the API key so it never gets committed to GitHub?'

Summary of Response:

Claude explained Streamlit's secrets management system using `.streamlit/secrets.toml`, access via `st.secrets` in code, and the importance of adding the secrets file to `.gitignore`.

Value I Added:

I configured the complete secrets management setup myself, verified it was properly gitignored, and tested that the key loaded correctly in both local development and Streamlit Cloud environments.

Session 10: OpenAI Chat Completions API Structure

February 6, 2026 | Tool: Claude (Anthropic)

Context:

I needed to understand the structure of the OpenAI Chat Completions API to design the symptom extraction function.

Prompt:

'Can you explain how the OpenAI Chat Completions API works? I need to understand the roles (system, user, assistant), how to structure the messages array, and what parameters like temperature and response_format do.'

Summary of Response:

Claude explained the message roles (system sets AI behaviour, user provides input, assistant captures previous responses), temperature as a creativity dial (0 = deterministic, 1 = creative), and response_format for enforcing JSON output.

Value I Added:

I designed the complete system prompt for medical symptom extraction independently, choosing temperature 0.3 for consistency in medical contexts and structuring the JSON schema with symptoms, duration, and severity fields that my downstream CTAS rules and ML model would need.

Session 11: Random Forest for Healthcare Triage — Literature Research

February 7, 2026 | Tool: Claude (Anthropic)

Context:

Sprint 2C algorithm research phase — I needed to find academic justification for Random Forest before benchmarking.

Prompt:

'Can you help me find academic papers that have used Random Forest for healthcare triage? I need to justify why RF is appropriate for symptom-to-disease prediction before I start training.'

Summary of Response:

Claude identified several relevant papers on RF for clinical decision support, noting RF strengths in handling high-dimensional binary feature spaces common in symptom datasets.

Value I Added:

I independently searched and read the full papers, wrote the complete literature review, and synthesised findings into a research justification mapping each algorithm to its specific purpose in the platform.

Session 12: CTAS Rule Design and Level Mapping

February 7, 2026 | Tool: Claude (Anthropic)

Context:

Designing the 14 CTAS emergency rules — I needed to understand the 5 triage levels to determine which symptom patterns correspond to Level 1 and Level 2.

Prompt:

'Can you explain the 5 CTAS triage levels used in Canadian emergency departments? I need to understand which symptom patterns correspond to Level 1 and Level 2 so I can build a rule-based safety check.'

Summary of Response:

Claude explained the CTAS level structure: Level 1 (Resuscitation) for immediate life threats, Level 2 (Emergent) for potential threats to life or limb.

Value I Added:

I independently selected and defined all 14 critical symptom patterns, mapped each to the correct CTAS level and emergency action, and designed the match_type logic (any vs all) based on clinical reasoning.

Session 13: Prompt Engineering for JSON Medical Extraction

February 7, 2026 | Tool: ChatGPT

Context:

Designing the GPT symptom extraction system prompt — I needed best practices for forcing GPT to return only valid JSON.

Prompt:

'What are the best practices for writing a system prompt that forces GPT to return only valid JSON? I need it to extract medical symptoms from plain English.'

Summary of Response:

ChatGPT recommended explicit JSON format instructions with example output, specifying each field name and type, and a rule that no extra text outside the JSON should appear.

Value I Added:

I wrote the complete system prompt independently, testing with formal, casual, and vague inputs across multiple iterations. The final prompt included CTAS-relevant fields (symptoms, duration, severity, body_location) not suggested by the AI.

Session 14: 70% Confidence Threshold Strategy

February 11, 2026 | Tool: Claude (Anthropic)

Context:

Designing the confidence threshold for the ML fallback — I needed to understand the tradeoffs between different threshold values in a medical context.

Prompt:

'For my healthcare ML model, I need a confidence threshold below which I should fall back to OpenAI rather than using the ML prediction. What are the considerations for choosing a threshold in a medical context?'

Summary of Response:

Claude explained that lower thresholds risk showing low-confidence predictions while higher thresholds increase fallback frequency. In medical contexts, safety favours higher thresholds.

Value I Added:

I independently chose 70% based on testing the model's performance distribution and the OpenAI fallback cost tradeoff. I designed the fallback to pass already-extracted symptoms rather than re-extracting from scratch.

Session 15: Claude Code — Database Module Scaffolding

February 5, 2026 | Tool: Claude Code (VS Code Extension)

Context:

Having designed the complete database schema and all CRUD function signatures on paper, I used Claude Code for initial scaffolding of database.py.

Prompt:

'Create components/database.py for a health platform with SQLite. Tables: users (id, name, email, password_hash, age, gender, region, conditions, created_at) and assessments (id, user_id, symptoms, extracted_symptoms, urgency_level, recommendation, created_at). Include all CRUD functions with proper error handling.'

Summary of Response:

Claude Code generated a 690-line database module with table creation, CRUD functions, and test data insertion based on my schema specifications.

Value I Added:

I reviewed the entire generated code, verified the SQL schema matched my design, tested all functions using SQLite Viewer, and cleaned up test data that was no longer needed. All schema decisions and function signatures were designed by me before the session.

Session 16: SQLite Database Setup for Python

February 3, 2026 | Tool: Claude (Anthropic)

Context:

I was designing the database schema for the platform and wanted to confirm best practices for SQLite with Python, specifically around password storage.

Prompt:

'For my health platform, I need to store user accounts and symptom assessments in SQLite. What is the recommended approach for storing passwords securely? Should I use plain text, MD5, or something else?'

Summary of Response:

Claude recommended bcrypt for password hashing as it includes automatic salting and is computationally expensive to brute-force. It advised against MD5 and SHA-256 for passwords as they are too fast to hash.

Value I Added:

I independently designed the complete database schema (users and assessments tables) with appropriate columns based on my project requirements. The bcrypt recommendation confirmed my initial research. I then implemented the full database.py module with all CRUD functions.

Session 17: Bcrypt Implementation in Python

February 4, 2026 | Tool: Claude (Anthropic)

Context:

While implementing auth.py, I needed to understand the specific bcrypt API in Python to implement the hash_password and verify_password functions.

Prompt:

'How do I implement bcrypt password hashing in Python? I need a hash_password function that takes a string and returns a hash, and a verify_password function that checks a password against a stored hash.'

Summary of Response:

Claude provided the bcrypt.gensalt() and bcrypt.hashpw() pattern for hashing, and bcrypt.checkpw() for verification, including the encode/decode steps for string conversion.

Value I Added:

I implemented the complete auth.py module with additional validation logic (email format checking, password strength requirements) that was not covered in the AI response. I also integrated the auth functions with my database module and Streamlit session management.

Session 18: Understanding Streamlit Session State for Authentication

January 31, 2026 | Tool: Claude (Anthropic)

Context:

While learning Streamlit from the Udemy course, I needed to understand how session_state works for maintaining user login status across page navigation.

Prompt:

'I'm building a multi-page Streamlit app that needs user authentication. How does st.session_state work for keeping a user logged in when they navigate between pages?'

Summary of Response:

Claude explained that session_state persists across Streamlit reruns and page navigation, making it ideal for storing authentication status. It showed the pattern of checking session_state at the top of each page.

Value I Added:

I designed the complete authentication flow myself, including which session variables to store (user_id, user_name, logged_in) and how page protection should work. The session_state pattern was applied to my specific use case.

Session 19: Streamlit Login/Register UI Pattern

February 4, 2026 | Tool: Claude (Anthropic)

Context:

I had designed the login/register mockups in UX Pilot and needed to implement them in Streamlit with tabs and form validation.

Prompt:

'I have a mockup for a login/register page with tabs. How do I create a tabbed interface in Streamlit with form validation? The register form needs to include dropdown selectors and checkboxes for health conditions.'

Summary of Response:

Claude explained Streamlit's `st.tabs()` component and `st.form()` for grouping inputs with a single submit button, along with `st.selectbox` for dropdowns and `st.checkbox` for health conditions.

Value I Added:

I designed the entire form layout to match my UX Pilot mockups, selected the specific BC health regions and health conditions relevant to my project, implemented custom CSS for centering and styling, and added comprehensive validation logic for all fields.

Session 20: Page Protection Pattern with `require_authentication()`

February 5, 2026 | Tool: Claude (Anthropic)

Context:

After implementing login, I needed to protect all pages so unauthenticated users cannot access them directly.

Prompt:

'How do I protect Streamlit pages so that only logged-in users can access them? I want unauthenticated users to see a message telling them to login.'

Summary of Response:

Claude suggested creating a helper function that checks `session_state` and uses `st.warning()` plus `st.stop()` to halt page rendering for unauthenticated users.

Value I Added:

I created the `require_authentication()` function and systematically applied it to all 6 pages. I designed the specific warning messages and added a link back to the login page for better user experience.

Session 21: Streamlit Multi-Section Dashboard Layout

February 25, 2026 | Tool: Claude (Anthropic)

Context:

I had designed the Home Dashboard mockup and needed to understand how to combine Streamlit layout components to create card-style UI sections.

Prompt:

'I'm rebuilding a Streamlit page to match a dashboard mockup with a 3-card row at the top, a 2-column section below, and a full-width banner. How do I structure this layout using `st.columns` and `st.container` with custom CSS cards?'

Summary of Response:

Claude explained nesting `st.columns()` for horizontal layouts, `st.container()` for grouping, and injecting custom CSS via `st.markdown(unsafe_allow_html=True)` for card styling.

Value I Added:

I designed the entire 4-section layout based on my mockup. I wrote the complete page structure, chose all CSS properties, selected the teal colour scheme (#1a6b5c), and integrated database queries for live data.

Session 22: Database Query for Dashboard Statistics

February 26, 2026 | Tool: Claude (Anthropic)

Context:

I needed to display a Most Common Symptom with a percentage on the Home Dashboard but the existing function did not return counts.

Prompt:

'My `get_assessment_stats()` function returns the most common symptom but I need the count to calculate a percentage. What's the best way to add a count to the return dictionary without breaking the existing function signature?'

Summary of Response:

Claude suggested adding the count as an additional key in the returned dictionary while preserving all existing keys for backward compatibility.

Value I Added:

I implemented the modification myself and added the percentage calculation in the dashboard layer. I also designed the fallback display for users with no assessment history.

Session 23: Plotly Chart Types for Streamlit Dashboards

February 28, 2026 | Tool: Claude (Anthropic)

Context:

I needed to understand Plotly chart configuration for the dashboard visualisations — area charts, donut charts, and axis formatting.

Prompt:

'I'm building a personal health dashboard with Plotly in Streamlit. I need an area chart for monthly trends, a donut chart for symptom breakdown, and proper axis formatting. What are the Plotly express functions and key parameters?'

Summary of Response:

Claude explained `px.area()` for the trend chart, `px.pie()` with `hole` parameter for the donut, and key parameters like `dtick`, `textposition`, and `domain`.

Value I Added:

I selected all chart types and designed the data pipeline. I chose the specific colour coding (red=Serious, amber=Infectious, blue=General) and wrote the pandas aggregation logic and urgency scoring algorithm (Emergency=4 to Self-Care=1).

Session 24: Plotly Donut Chart Label Clipping Fix

March 1, 2026 | Tool: Claude (Anthropic)

Context:

The donut chart labels were being clipped at the chart edges in Streamlit and I needed to fix the rendering.

Prompt:

'My Plotly donut chart labels are being cut off at the edges in Streamlit. How do I fix label clipping in a Plotly pie chart?'

Summary of Response:

Claude suggested adjusting the `domain` parameter to leave margin space and using `showlegend=True` to move labels to a separate legend rather than floating outside the chart.

Value I Added:

I identified the specific clipping behaviour through my own testing, chose the legend approach over floating labels as cleaner for small charts, and adjusted the chart dimensions independently.

Session 25: Streamlit Page Redirect After Login

February 25, 2026 | Tool: Claude (Anthropic)

Context:

After a successful login, I needed to redirect users to the Home page automatically.

Prompt:

'After a successful login in Streamlit, how do I redirect the user to a different page automatically after setting `session_state`?'

Summary of Response:

Claude explained `st.switch_page()` for programmatic navigation in Streamlit multi-page apps.

Value I Added:

I implemented the redirect within my authentication flow and tested edge cases including failed login, duplicate registration, and direct URL access while unauthenticated.

Session 26: Claude Code — Dashboard Page Generation

February 27, 2026 | Tool: Claude Code (VS Code Extension)

Context:

Having designed the complete layout specifications for both dashboards based on my UI mockups, I used Claude Code for initial page scaffolding.

Prompt:

'Review the existing 1_Home.py, database.py, and app.py. Outline how 1_Home.py should be structured with 4 sections: Welcome Header using session_state username, Quick Actions row with 3 cards linking to other pages, two-column layout for Health Summary and Recent Activity using get_assessment_stats() and get_user_assessments(), and a BC Health Tip banner. Use the teal colour scheme #1a6b5c.'

Summary of Response:

Claude Code clarified layout patterns, data-flow considerations, and Streamlit dashboard best practices.

Value I Added:

I created all layout specifications, selected chart types, defined the urgency scoring algorithm, and designed all database integrations. I validated every metric against SQLite directly, identified UI issues, and iteratively refined until matching my mockups.

Session 27: Agentic AI Research for Future Enhancement

March 2, 2026 | Tool: Claude (Anthropic)

Context:

After completing the dashboard pages, I researched how agentic AI concepts could further enhance the chatbot beyond the planned multi-turn enhancement.

Prompt:

'I've been reading about agentic AI and autonomous AI agents. How would an agentic approach differ from my current 3-layer pipeline for healthcare triage? Is this something I could realistically integrate into my Streamlit project with 1.5 months remaining?'

Summary of Response:

Claude explained key differences between pipeline-based AI and agentic AI (autonomous planning, tool use, reflection, memory) and noted that agentic healthcare systems carry high regulatory complexity.

Value I Added:

I independently evaluated feasibility and concluded that full agentic implementation exceeded the current timeline. I documented this as exploratory future work, keeping the project scope realistic and deliverable.

Session 28: NLP Dialog System Pipeline for Healthcare Chatbot

March 3–6, 2026 | Tool: Claude (Anthropic)

Context:

After reviewing CSIS 3400 NLP course materials on chatbot theory, I needed to clarify how slot-filling in task-oriented dialog systems could be adapted for healthcare symptom assessment.

Prompt:

'I'm studying dialog system pipelines from my NLP course. My chatbot currently has NLU (OpenAI extracts symptoms) and NLG (returns a recommendation), but no Dialog Manager layer. How does slot-filling work for medical symptom collection?'

Summary of Response:

Claude explained slot-filling as incrementally collecting required parameters until enough information exists to fulfil the intent, describing how medical systems prioritise slots by clinical importance.

Value I Added:

I identified the architecture gap independently from reviewing course material. I decided on the specific 4 slots (symptoms, duration, severity, body_location) and chose the priority order based on how clinicians triage — not from Claude's response. I set the hard cap of 3 turns based on user experience considerations.

Session 29: Stateful Dialog Management Using Streamlit session_state

March 5, 2026 | Tool: Claude (Anthropic)

Context:

Having designed the dialog_state schema on paper, I needed to understand how Streamlit's session_state maintains conversational state across multiple inputs.

Prompt:

'How does Streamlit's session_state behave across multiple user interactions in a chat interface? I'm building a multi-turn chatbot where I need to accumulate slot values (symptoms, duration, severity) across turns.'

Summary of Response:

Claude explained that session_state persists across reruns within a session, and described the pattern of initialising a dialog state dictionary on first load and updating it incrementally on each turn.

Value I Added:

I designed the 8-field dialog_state schema myself, chose which slots to track and how to represent conversation history, and implemented the turn_count hard cap as a safety decision to prevent over-interrogating users.

Session 30: Folium Interactive Map Integration in Streamlit

March 9, 2026 | Tool: Claude (Anthropic)

Context:

Building the BC Facility Finder — I needed to understand how Folium integrates with Streamlit for interactive maps with popup markers.

Prompt:

'How do I use Folium with Streamlit to display an interactive map? I need to add markers for hospitals and clinics with popup HTML showing the facility name, address, and phone number.'

Summary of Response:

Claude explained the streamlit-folium library's `st_folium()` function, the `folium.Marker()` and `folium.Popup()` pattern, and how to set initial map center and zoom.

Value I Added:

I designed the map architecture, colour scheme (blue=hospitals, green=walk-in, orange=urgent care), popup content structure, and city filtering logic. I chose CartoDB Positron tiles for clean aesthetics matching the CedarCare brand.

Session 31: Structuring a Stateful Dialog Backend in Python

March 8, 2026 | Tool: Claude Code (VS Code Extension)

Context:

Having designed the dialog state schema and slot-filling logic on paper, I needed to clarify best practices for structuring the functions in Python before implementation.

Prompt:

'I'm building a multi-turn chatbot in Python with a `dialog_state` dictionary. I want to separate the turn orchestration (incrementing count, running safety checks) from the slot-filling logic. What's the cleanest way to structure these as separate functions?'

Summary of Response:

Claude explained the pattern of a thin orchestrator calling focused helper functions, and described how a shared mutable dictionary can be passed by reference through both.

Value I Added:

I had designed all architectural decisions before this session: the 8-field schema, 4 slots and clinical priority order, emergency circuit breaker requirement, and temperature values. The session clarified the structural pattern; I then wrote all functions myself.

Session 32: Folium Map Filtering and Dynamic Dropdown Patterns

March 9, 2026 | Tool: Claude Code (VS Code Extension)

Context:

Building the Facility Finder filter UI — I needed to implement cascading dropdowns where city options update dynamically based on the selected region.

Prompt:

'I have a Streamlit page with a facility filter. When the user selects a BC Health Authority region, the City dropdown should update to show only cities in that region. How do I implement cascading selectboxes in Streamlit?'

Summary of Response:

Claude explained using `st.session_state` to track the selected region and filtering the city list dynamically before rendering the second selectbox.

Value I Added:

I designed the complete facility data schema, mapped all BC facilities to health authorities and cities, and implemented the full filter pipeline including the Open Now toggle and facility type filter.

Session 33: JSON Knowledge Base Design for Health Library

March 11, 2026 | Tool: Claude (Anthropic)

Context:

Having identified 8 health topic categories and mapped 18 article topics, I needed to clarify the JSON schema so article fields could serve both as search filter targets and UI rendering data.

Prompt:

'I'm building a health information library backed by a JSON file. Each article needs to support keyword search across multiple fields, category filtering, expandable card rendering, and a Visit Source link to the official HealthLinkBC URL. What fields should the JSON schema include?'

Summary of Response:

Claude described a standard knowledge base schema with separate fields for search metadata (tags array), display content (summary vs full content), and source attribution (source_url).

Value I Added:

I independently decided on the 8 categories based on reviewing HealthLinkBC's own topic structure. I added banner_color for the Popular Topics visual design — not suggested by Claude. I required source_url for every article to ensure all content links to a verifiable official source.

Session 34: CSS :has() Selector Pattern for Streamlit Widget Styling

March 13, 2026 | Tool: Claude (Anthropic)

Context:

Building the Health Library — I needed to style the st.pills() chip selection widget to match the CedarCare brand without overriding all Streamlit defaults.

Prompt:

'I need to style a Streamlit st.pills() component to use custom teal colour for the selected chip state. The pills component uses data-testid attributes. How do I target it with CSS injected via st.markdown?'

Summary of Response:

Claude explained the CSS :has() selector approach for targeting Streamlit widget containers by their data-testid attributes without relying on unstable class names.

Value I Added:

I identified the specific data-testid values by inspecting the Streamlit DOM myself, chose which widget states to target, and implemented the complete CSS injection approach with fallback styling for older browsers.

Session 35: Synthetic Data Generation with Stratified Patient Tiers

March 16, 2026 | Tool: Claude (Anthropic)

Context:

Designing the 5,000-record synthetic dataset for System Analytics — I needed to ensure the data would produce meaningful K-Means clusters rather than random noise.

Prompt:

'I'm generating synthetic patient assessment data for a BC health platform analytics dashboard. I need 5,000 records representing 200 patients. How do I structure the generation to ensure K-Means clustering will produce meaningful patient segments?'

Summary of Response:

Claude explained that for meaningful clustering, the data should embed distinct patient archetypes: high-frequency chronic patients, moderate-risk active users, and low-frequency healthy users.

Value I Added:

I independently defined the 3 patient tiers (Chronic/Elderly, Active/Moderate, Healthy/Occasional) and their specific parameter ranges based on BC health statistics. I decided on the seasonal patterns (flu peak December, allergy peak April) to make the analytics narratively meaningful.

Session 36: K-Means Feature Engineering for Patient Segmentation

March 12, 2026 | Tool: Claude (Anthropic)

Context:

Designing the K-Means clustering feature set for the System Analytics Dashboard patient segmentation.

Prompt:

'Why is StandardScaler important before K-Means? My patient features include age (18–85) and assessment count (3–80) — would different scales affect clustering?'

Summary of Response:

Claude explained that K-Means uses Euclidean distance, making features with larger ranges dominate. StandardScaler normalises each feature to zero mean and unit variance, giving each equal weight.

Value I Added:

I selected the four clustering features independently based on clinical relevance (mean_age, assessment_count, urgent_ratio, chronic_ratio). I implemented automatic cluster naming by sorting on combined risk score, and designed the scatter plot with bubble size encoding patient age.

Session 37: Understanding pytest Fixture Patterns for conftest.py Design

March 17, 2026 | Tool: Claude (Anthropic)

Context:

Before writing the 95-test suite, I needed to understand pytest fixture scoping and how to design a confest.py that would provide isolated SQLite test databases.

Prompt:

'I'm designing a confest.py for a pytest suite testing a Streamlit + SQLite app. I need a test_db fixture that creates a fresh :memory: SQLite database for each test, initialises the schema, and tears down cleanly. What's the best fixture scope and teardown pattern?'

Summary of Response:

Claude explained pytest fixture scopes (function, class, module, session), the yield pattern for teardown, and that function scope with :memory: SQLite provides the strongest isolation.

Value I Added:

I decided on the 4-folder test structure (unit/integration/ui/property), the yield teardown pattern, and the choice to use :memory: SQLite rather than a temporary file — providing both isolation and speed.

Session 38: Mocking Streamlit Dependencies for Chatbot Module Testing

March 20, 2026 | Tool: Claude (Anthropic)

Context:

I needed to import chatbot.py in pytest without triggering Streamlit's server connection or @st.cache_resource decorator failures.

Prompt:

'I need to import a Python module (chatbot.py) that has 'import streamlit as st' at the top level and uses @st.cache_resource. When I try to import it in pytest, it fails because Streamlit tries to connect to a server. How can I mock the entire streamlit module in sys.modules?'

Summary of Response:

Claude explained the sys.modules patching approach — installing a MagicMock as sys.modules['streamlit'] before any import of the dependent module.

Value I Added:

I identified which specific Streamlit attributes needed specific mock values by reading chatbot.py line by line. The cache_resource lambda pattern was verified by testing against the actual decorator usage. All mock return value structures were constructed from reading the real function signatures.

Session 39: Property-Based Testing with Hypothesis for Healthcare Validation Functions

March 21, 2026 | Tool: Claude (Anthropic)

Context:

Having designed the 10 fuzz test scenarios, I needed to understand how to compose Hypothesis strategies for inputs satisfying specific structural constraints.

Prompt:

'I'm writing property-based tests with Hypothesis for a `validate_email()` function that checks 6 specific conditions. How do I build a strategy using `st.builds()` that generates emails satisfying all 6 conditions? For strings that must never match any of 40+ medical keywords, what is the most reliable approach?'

Summary of Response:

Claude explained the `st.builds()` strategy for composing structured strings from independently generated parts and `st.from_regex()` for the non-medical keyword filter.

Value I Added:

I used `@settings(max_examples=100)` for exactly 1,000 total fuzz inputs. The `_CTAS_KEYWORDS` set of 40+ keywords was compiled by me from reading `rules.py` directly. The `valid_password` strategy correction (letter + digit, not uppercase + lowercase) was caught by me reading the source code before writing tests.

Session 40: Streamlit AppTest Framework for Headless UI Testing

March 20, 2026 | Tool: Claude (Anthropic)

Context:

Before writing the dashboard and chatbot UI tests, I needed to understand Streamlit's AppTest API for `session_state` injection, chat input simulation, and auth guard testing.

Prompt:

'I'm using Streamlit's AppTest to test a dashboard page that requires authenticated `session_state`. How do I set `session_state` values before calling `at.run()`? What is the AppTest syntax for submitting a chat message? And what happens when `st.switch_page()` is called for unauthenticated users?'

Summary of Response:

Claude explained `AppTest.from_file()`, `session_state` injection via `at.session_state["key"] = value`, the `at.chat_input[0].set_value("message").run()` syntax, and that `st.switch_page()` raises `StreamlitAPIException` in AppTest.

Value I Added:

The `os.chdir(SRC_DIR)` fix for the relative `logo.png` path issue was discovered through my own testing. The dual-path assertion for auth guard tests and the `_submit_chat()` helper with patch re-application across chat reruns were both my own solutions.

Session 41: GitHub Actions CI/CD Pipeline for Python/pytest

April 2, 2026 | Tool: Claude (Anthropic)

Context:

Sprint 5B — setting up the GitHub Actions workflow to run the 79-test suite automatically on every push to main.

Prompt:

'I have a Python/Streamlit project with pytest tests at the root `tests/` folder. The app runs from `Implementation/src/`. Can you explain how GitHub Actions resolves module imports when running pytest from a different directory than the source code?'

Summary of Response:

Claude clarified that running pytest from Implementation/src/ allows Python to find the components module correctly and explained the distinction between the pytest binary PATH and python -m pytest invocation.

Value I Added:

I implemented the cd Implementation/src approach in ci.yml, diagnosed the streamlit.testing module conflict, designed the --ignore=../tests/ui exclusion, and resolved the model.pkl not found error by removing it from .gitignore.

Session 42: Streamlit Session State and Widget Re-evaluation for Chip Bug

April 3, 2026 | Tool: Claude (Anthropic)

Context:

Debugging the chips-on-redirect bug where severity chips appeared after non-medical inputs.

Prompt:

'I have a Streamlit chatbot that shows severity chips after every bot response. The chips should only appear when the bot has collected real symptoms and is asking a follow-up. How does st.rerun() interact with session_state-based widget visibility guards?'

Summary of Response:

Claude clarified how st.rerun() triggers a full script re-execution and that widget visibility controlled by session_state flags persists correctly if the flag is checked before the widget render call.

Value I Added:

I designed the _show_chips = True/False flag approach, implemented the two-tier redirect logic (gentle redirect turn 1, hard end turn 2+), and applied the fix to both the chat input handler and the chip handler. All implementation decisions were independently made.

Session 43: Google Forms Survey Design — Question Type Selection

April 3, 2026 | Tool: Claude (Anthropic)

Context:

Setting up the CedarCare User Evaluation Survey on Google Forms for user testing.

Prompt:

'I'm designing a user evaluation survey in Google Forms. For a 5-point Likert scale question, should I use a Linear Scale question type or a Multiple Choice Grid? What are the tradeoffs?'

Summary of Response:

Claude clarified that Linear Scale is most appropriate for single Likert items, while Multiple Choice Grid works better for rating multiple items on the same scale simultaneously.

Value I Added:

I independently designed all 6 survey questions, chose the question types (Linear Scale for Q1/Q2/Q4/Q5, Multiple Choice for Q3, Paragraph for Q6), wrote all question text and scale labels, and distributed the survey to 12 testers.

Session 44: Landing Page Implementation via `st.components.v1.html()`

April 2, 2026 | Tool: Claude (Anthropic)

Context:

Implementing the CedarCare public landing page — I needed to understand how to render a fully custom HTML/CSS page inside Streamlit without Streamlit overriding the styles.

Prompt:

'I want to build a custom marketing-style landing page inside a Streamlit app with full control over layout and CSS. The default Streamlit layout adds padding and constraints I can't override. Is there a way to render pure HTML/CSS without any Streamlit widget wrappers?'

Summary of Response:

Claude explained `st.components.v1.html()` which renders arbitrary HTML in an iframe, bypassing Streamlit's layout engine entirely. It noted the height parameter must be set explicitly and that JavaScript `postMessage` cannot communicate with the parent Streamlit app due to iframe sandboxing.

Value I Added:

I designed the complete landing page HTML/CSS myself — the hero section, two-tone CedarCare wordmark, chatbot demo mockup card, statistics counter bar, and trust indicator row. I discovered the `href="?show_login=true"` anchor workaround for navigation independently after finding `postMessage` was blocked by the iframe sandbox.

Session 45: Streamlit `st.pills()` for Severity Chip UI

March 8, 2026 | Tool: Claude (Anthropic)

Context:

Sprint 3E multi-turn chatbot enhancement — I needed to implement clickable severity chips as a UI improvement over plain text input for chatbot follow-up questions.

Prompt:

'Is there a Streamlit component that renders clickable pill/chip buttons for a selection interface? I want to show options like Mild, Moderate, Severe as chips the user can tap, similar to chat suggestion chips in messaging apps.'

Summary of Response:

Claude confirmed that `st.pills()` was introduced in Streamlit 1.40 and renders a horizontal row of selectable pill buttons, explaining the `options` parameter, `label_visibility`, and that the return value is the selected option string or `None`.

Value I Added:

I independently designed the chip context logic — severity chips on turn 1, duration chips on turn 2, and body location chips on turn 3. I implemented the `_chip_selected` session state flag for handling chip selection across reruns and the key parameter strategy to prevent widget state conflicts.

Session 46: Health Library Full-Text Search Implementation

March 14, 2026 | Tool: Claude (Anthropic)

Context:

Implementing the search functionality in the Health Education Library — I needed to perform case-insensitive multi-field search across a JSON knowledge base efficiently in Python.

Prompt:

'I have a list of 18 article dictionaries loaded from a JSON file. Each article has title, summary, content, tags, and category fields. How do I implement a case-insensitive search that matches the query against all of these fields and returns articles where any field contains the search term?'

Summary of Response:

Claude explained using `str.lower()` for normalisation combined with the `any()` built-in to check if the search term appears in any specified field, wrapped in a list comprehension for filtering.

Value I Added:

I decided which fields to include in search scope (title, summary, content, tags — excluding category), wrote the complete search pipeline with popular search chip shortcuts, and designed the article count display to give users feedback on search results.

Session 47: Urgency Colour Coding Strategy for Clinical Context

February 10, 2026 | Tool: Claude (Anthropic)

Context:

Designing the urgency badge colour scheme for the Symptom Assessment Chatbot result cards — I needed colours that were both visually distinct and clinically appropriate.

Prompt:

'For a health triage application with 4 urgency levels (Emergency, Urgent, Routine, Self-Care), what colour conventions are used in clinical settings? I want my UI colour coding to align with healthcare conventions.'

Summary of Response:

Claude explained that clinical settings use red for immediate emergency, orange/amber for urgent, yellow for less urgent, and green for non-urgent — mirroring the traffic light convention but with orange distinguishing urgent from emergency red.

Value I Added:

I independently mapped the 4 urgency levels to specific hex colours (`#FF4B4B` for Emergency, `#FFA500` for Urgent, `#FFD700` for Routine, `#00CC66` for Self-Care) and implemented the `urgency_badge()` HTML helper function used consistently across all result cards and dashboard components.

Session 48: joblib Model Serialisation and @st.cache_resource Pattern

February 10, 2026 | Tool: Claude (Anthropic)

Context:

After training the Random Forest model, I needed to understand the best approach for saving and loading the model in Streamlit without reloading it on every page rerun.

Prompt:

'I've trained a Random Forest model with scikit-learn and need to save it for use in my Streamlit app. Should I use pickle or joblib? And how do I prevent Streamlit from reloading the model on every rerun — loading takes 2–3 seconds?'

Summary of Response:

Claude recommended joblib over pickle for scikit-learn models as it is more efficient for large numpy arrays. It explained @st.cache_resource as the correct decorator for singleton resources that should be loaded once per session.

Value I Added:

I implemented load_model() with @st.cache_resource, the MODEL_PATH and METADATA_PATH constants using relative paths via os.path, and the model_metadata.json schema storing the symptom list, severity weights, urgency mapping, class names, and the 70% confidence threshold as a configurable parameter.

Session 49: Pandas DataFrame Filtering for Dashboard Date Range Filter

February 28, 2026 | Tool: Claude (Anthropic)

Context:

Implementing the date range filter on the Personal Health Dashboard — I needed to filter assessment records by a selected time window consistently across all KPI cards and charts.

Prompt:

'I have a pandas DataFrame of assessment records with a created_at datetime column. I need to filter it to show only records from the last 30 days, last 3 months, or all time based on a user dropdown. What is the best pandas pattern for datetime filtering?'

Summary of Response:

Claude explained using pd.to_datetime() to ensure the column is datetime type, then comparing against datetime.now() minus a timedelta for the lookback windows.

Value I Added:

I designed the complete filter pipeline — converting the date column at load time, computing the cutoff date from the dropdown selection, and passing the single filtered DataFrame to all downstream KPI calculations and chart functions so that changing the filter updates all components consistently.

Session 50: Seed Database Design for Streamlit Cloud Deployment

April 2, 2026 | Tool: Claude (Anthropic)

Context:

Sprint 5C deployment preparation — Streamlit Cloud uses an ephemeral filesystem meaning the SQLite database is wiped on each deployment. I needed to design a seed script to create a usable test account automatically.

Prompt:

'My Streamlit app uses SQLite which gets wiped on Streamlit Cloud deployments. I need a seed_db.py script that creates a test user account on fresh deployment. How do I make it idempotent — safe to run multiple times without creating duplicate accounts?'

Summary of Response:

Claude explained using INSERT OR IGNORE or a prior SELECT check to make the insert idempotent, and noted that the script should use the same bcrypt hashing as the production registration flow.

Value I Added:

I designed seed_db.py to call the existing init_db() and create_user() functions from database.py rather than writing raw SQL, ensuring seed data goes through the same validation and hashing pipeline as real user registration. I committed seed_db.py to the repo so Streamlit Cloud runs it as part of app startup.

Session 51: reshape(1,-1) for Single-Sample ML Prediction

February 11, 2026 | Tool: Claude (Anthropic)

Context:

Implementing the _ml_predict() function — when calling model.predict_proba() on a single user input, I encountered a shape error and needed to understand why.

Prompt:

'I'm calling a scikit-learn RandomForestClassifier's predict_proba() method with a single symptom vector but I get a ValueError about the input shape. My feature vector is a 1D array of 131 values. What does reshape(1, -1) do and why is it required?'

Summary of Response:

Claude explained that scikit-learn's predict_proba() expects a 2D array with shape (n_samples, n_features). A 1D array of 131 values has shape (131,) which scikit-learn interprets as 131 samples of 1 feature each. reshape(1, -1) converts it to shape (1, 131) — 1 sample with 131 features.

Value I Added:

I implemented the complete _ml_predict() function independently, including the severity weighting step, the 70% confidence threshold check, the top-3 disease retrieval from predict_proba(), and the graceful exception handler that silently falls back to OpenAI if the model fails at runtime.